



Frameworkx Standard

Information Framework (SID)

Service Overview Business Entities

Information Framework Suite
GB922 Service Overview
Release 16.0.1
September 2016

Latest Update: Frameworkx Release 16	TM Forum Approved Suitable for Conformance
Version 16.0.2	IPR Mode: RAND

Notice

Copyright © TM Forum 2016. All Rights Reserved.

This document and translations of it may be copied and furnished to others, and derivative works that comment on or otherwise explain it or assist in its implementation may be prepared, copied, published, and distributed, in whole or in part, without restriction of any kind, provided that the above copyright notice and this section are included on all such copies and derivative works. However, this document itself may not be modified in any way, including by removing the copyright notice or references to TM FORUM, except as needed for the purpose of developing any document or deliverable produced by a TM FORUM Collaboration Project Team (in which case the rules applicable to copyrights, as set forth in the [TM FORUM IPR Policy](#), must be followed) or as required to translate it into languages other than English.

The limited permissions granted above are perpetual and will not be revoked by TM FORUM or its successors or assigns.

This document and the information contained herein is provided on an "AS IS" basis and TM FORUM DISCLAIMS ALL WARRANTIES, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTY THAT THE USE OF THE INFORMATION HEREIN WILL NOT INFRINGE ANY OWNERSHIP RIGHTS OR ANY IMPLIED WARRANTIES OF MERCHANTABILITY OR FITNESS FOR A PARTICULAR PURPOSE.

TM FORUM invites any TM FORUM Member or any other party that believes it has patent claims that would necessarily be infringed by implementations of this TM Forum Standards Final Deliverable, to notify the TM FORUM Team Administrator and provide an indication of its willingness to grant patent licenses to such patent claims in a manner consistent with the IPR Mode of the TM FORUM Collaboration Project Team that produced this deliverable.

The TM FORUM invites any party to contact the TM FORUM Team Administrator if it is aware of a claim of ownership of any patent claims that would necessarily be infringed by implementations of this TM FORUM Standards Final Deliverable by a patent holder that is not willing to provide a license to such patent claims in a manner consistent with the IPR Mode of the TM FORUM Collaboration Project Team that produced this TM FORUM Standards Final Deliverable. TM FORUM may include such claims on its website, but disclaims any obligation to do so.

TM FORUM takes no position regarding the validity or scope of any intellectual property or other rights that might be claimed to pertain to the implementation or use of the technology described in this TM FORUM Standards Final Deliverable or the extent to which any license under such rights might or might not be available; neither does it represent that it has made any effort to identify any such rights. Information on TM FORUM's procedures with respect to rights in any document or deliverable produced by a TM FORUM Collaboration Project Team can be found on the TM FORUM website. Copies of claims of rights made available for publication and any assurances of licenses to be made available, or the result of an attempt made to obtain a general license or permission for the use of such proprietary rights by implementers or users of this TM FORUM Standards Final Deliverable, can be obtained from

the TM FORUM Team Administrator. TM FORUM makes no representation that any information or list of intellectual property rights will at any time be complete, or that any claims in such list are, in fact, Essential Claims.

Direct inquiries to the TM Forum office:

240 Headquarters Plaza,
East Tower – 10th Floor,
Morristown, NJ 07960 USA
Tel No. +1 973 944 5100
Fax No. +1 973 944 5110
TM Forum Web Page: www.tmforum.org

Table of Contents

Notice	2
Table of Contents	4
Table of Figures	5
1. Business Entities	7
1.1. Service Domain	7
1.1.1. Introduction	7
1.1.2. Approach	11
1.1.3. Overview of “Service”	12
1.1.4. Use Cases	13
1.1.5. Scope	13
1.1.6. A Starting Point – Supporting a Link to Products	14
1.1.7. A Simple Example	18
1.1.8. ServiceSpecification	21
1.1.9. CustomerFacing and ResourceFacing ServiceSpecifications	27
1.1.10. Stand-Alone vs. Aggregate Services and ServiceSpecifications	29
1.1.11. ServiceSpecVersion	32
1.1.12. ServiceSpecificationTypes	33
1.1.13. ServiceRoles and ServiceSpecRoles	35
1.1.14. ServiceSpecCharacteristic and ServiceSpecCharacteristicValue Classes	47
1.1.15. ServiceCharacteristicValue and ServiceSpecCharacteristicValue Classes	50
1.1.16. Linking to QoS – ServicePackage and ServiceBundle	53
1.1.17. Service Order	63
1.2. TIP Service Management ABE	66
1.2.1. Introduction	66
1.2.2. Service and Sap Specifications	66
1.2.3. Service Related Characteristics	68
1.2.4. Service Activation	70
1.2.5. Ethernet Services Example	72
1.3. Interaction With Other SID Business Entities	74
1.3.1. Introduction	74
1.3.2. Policy	74
1.3.3. BusinessInteraction	74
1.3.4. PartyRole	75
1.3.5. Product	76
1.3.6. Interaction with Resources	77
1.3.7. Place	77
1.3.8. Service Level Specification Overview	78
1.4. References	87
2. Administrative Appendix	88
2.1 About this document	88
2.2 Document History	88
2.2.1. Version History	88
2.2.2. Release History	91
2.3 Acknowledgments	92

Table of Figures

Figure SO.01 - Conceptual Relationship of Product, Service and Resource	8
Figure SO.02 - eTOM Level 0 Domains	9
Figure SO.03 - Level One of the Service Domain of the SID Framework	10
Figure SO.04 - Simplified SID Product Model	14
Figure SO.05 - Starting Point for Modeling Service	15
Figure SO.06 - Basic Service Model – A Starting Point	16
Figure SO.07 - Representing BGP as a LogicalResource	19
Figure SO.08 - Service Specifications	23
Figure SO.09 - VPNService – VPNServiceSpecification Example	25
Figure SO.10 - Subclassing ServiceSpecification	28
Figure SO.11 - Modeling Stand-Alone and Aggregate ServiceSpecifications	30
Figure SO.12 - Modeling Stand-Alone and Aggregate Services	31
Figure SO.13 - Modeling the Versioning of a ServiceSpecification	32
Figure SO.14 - The Concept of a ServiceSpecificationType	33
Figure SO.15 - Simple 2547bis MPLS VPN.	35
Figure SO.16 - The Concept of LogicalDeviceRoles and DeviceInterfaceRoles	36
Figure SO.17 - The Concept of PhysicalResourceRoles	39
Figure SO.18 - Interaction Between ServiceSpecRoles and ResourceSpecRoles	40
Figure SO.19 - VPN Roles	42
Figure SO.20 - The Concept of ServiceRoles	43
Figure SO.21 - The Concept of ServiceSpecificationRoles	45
Figure SO.22 - The Concept of ServiceSpecCharacteristic	48
Figure SO.22a – Service and Resource Characteristic Associations	49
Figure SO.23 - ServiceCharacteristicValue and ServiceSpecCharacteristicValue	51
Figure SO.24 - Multiple Command Syntaxes and Programming Models	53
Figure SO.25 - The Concept of a ServicePackage	55
Figure SO.26 - Grouping Together CustomerFacingServiceSpecifications into a ServicePackage	56
Figure SO.27 - ServicePackageSpecs and ServicePackages	57
Figure SO.28 - Relationship Between ServiceBundleSpec and ServicePackageSpec	59
Figure SO.29 - ServiceBundles and ServiceBundleSpecifications	61
Figure SO.30 - Summary Relationships Between ServicePackage and ServiceBundle	62

Figure SOr.01 - Service Order Entities	63
Figure SOr.02 - Relationships Inherited From Business Interaction	64
Figure SOr.03 - Relationships Inherited From BI Item	65
Figure SO.31 - TIP Service Management Service Definition and Service Template	68
Figure SO.32 - TIP Service Management – Service Characteristics	69
Figure SO.33 - TIP Service Management – Service Activation	71
Figure SO.33a - TIP Service Management - CommonServiceInfo	72
Figure SO.34 - Example of Ethernet Service CFS/RFS	73
Figure SO.35 - Interaction Between PartyRole and Service	75
Figure SO.36 - Interaction between Service and Place	77
Figure SO.37 - ServiceLevelSpecification Overview	78
Figure SO.38 - Products, Services, and ServiceLevelAgreements	79
Figure SO.39 - Associating ServiceLevelSpecifications With Products and Resources	80
Figure SO.40 - Service Level Specification Parameters and Objectives	81
Figure SO.41 - KPIs, KQIs, Products, and Services	82
Figure SO.42. - ServiceLevelSpecConsequences	83
Figure SO.43 - ServiceLevelSpecApplicability	84
Figure SO.43a - ServiceLevelSpecificationExpression	85

1. Business Entities

1.1. Service Domain

1.1.1. Introduction

One of the goals of the SID model is to enable a seamless transition from business concepts and definitions to other domains, such as networking. This addendum covers the definition of services.

In looking at “Service”, the first problem is one of terminology. Service means many things to many people. For example, many people use the words “Product” and “Service” interchangeably. In addition, though Resources are usually not confused with Services, the relationships between different types of Resources and a Service are confusing to many people.

Business entities in the Product domain represent what is offered to the market. This domain deals with such things as the development, promotion, pricing, and retirement of product offerings, as well as instances of offerings procured by customers and others. Business entities in the Service and Resource domain represent (among other concepts) how products offered to the market are implemented. Services are procured by the market, but their representation is via a ProductOffering. This gives rise to a fundamental difference between services that are “customer facing” and services that are “resource facing”. As we will see, there is a marked difference between services that are used internally by the enterprise offering a product versus services that are represented as product offerings. It is therefore very important to differentiate between these two different types of services (as well as their relationship with Products) in our Service model.

From a definition perspective, a Product (Offering) is an externally facing representation of a Service and/or Resource procured by the market; for example, a real-time broadcast of a football match. In this example, the Customer buys the Product, which may require dedicated hardware to run the Services that make up the Product. In this view, a Service is an intangible realization of a Product or something provided in support of a Product; for example, a video streaming connection used to transmit the football match to a customer’s PDA, an installation service provided for a cable modem, or specific Quality of Service (QoS) settings that are required to provide an acceptable end-user experience watching the football match. A Resource is part of an enterprise’s infrastructure utilized by a Service or a good procured by the market in the form of a Product. In the above example, a wireless network utilized by a video streaming connection is an example of a Resource.

This can be conceptually thought of as illustrated in figure below.

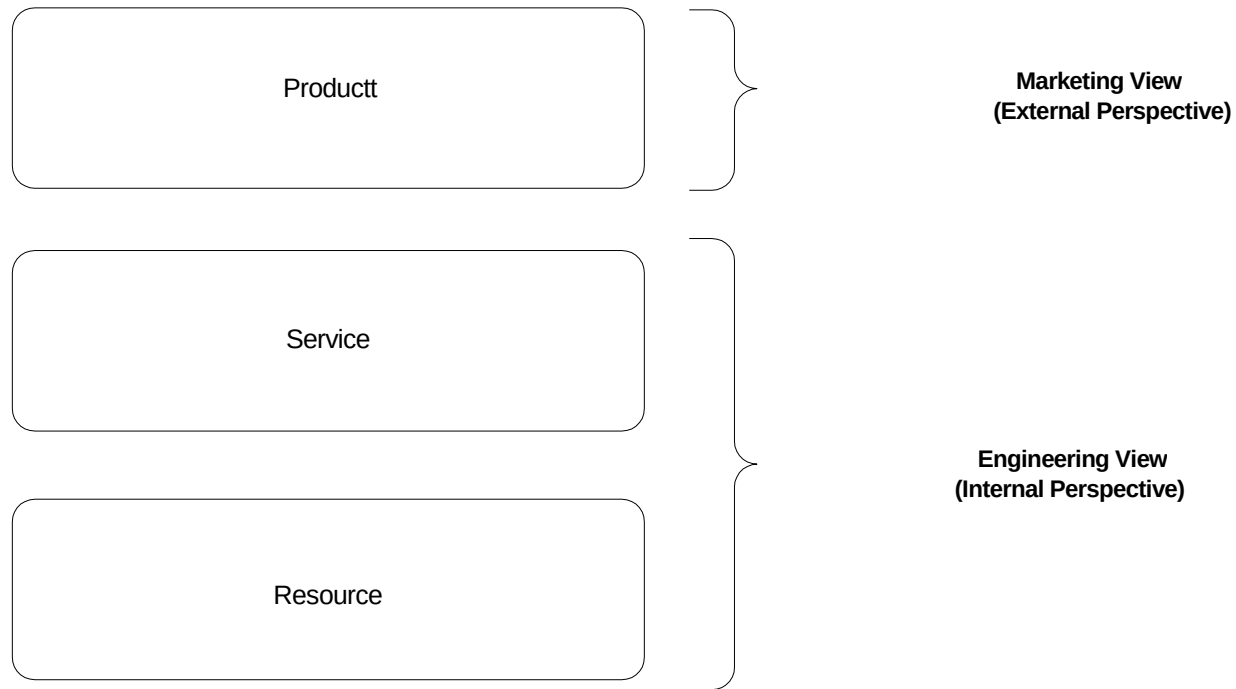


Figure SO.01 - Conceptual Relationship of Product, Service and Resource

The eTOM [eTOM] shows this separation in its Level 0 Domain map, shown in Figure SO.02 below.

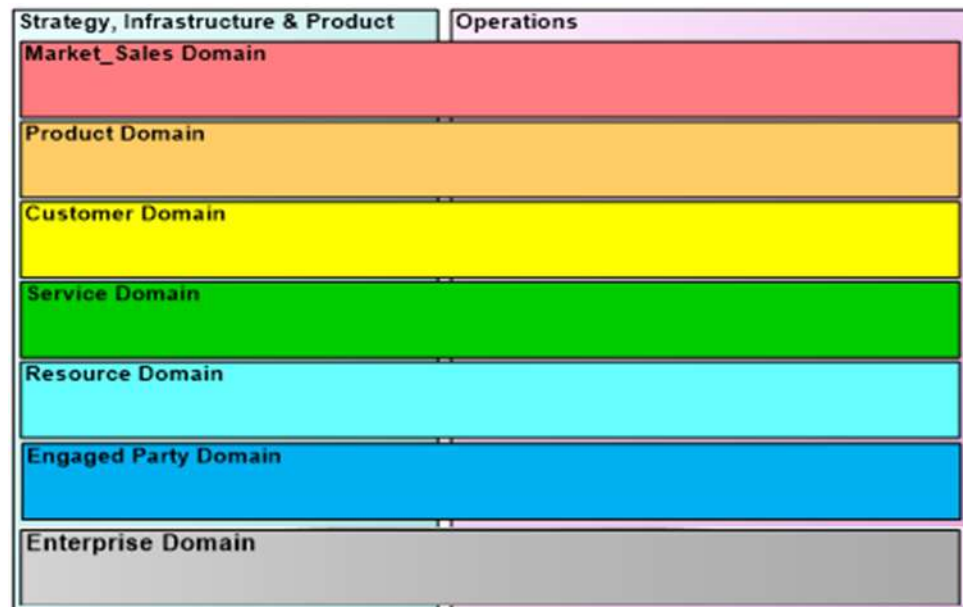


Figure SO.02 - eTOM Level 0 Domains

The reader is encouraged to refer to the eTOM for an explanation of the various business processes that are defined to support the concepts in the above domains.

The SID uses the SID Framework to link the eTOM to the SID. Level 1 of the Service Domain of the SID Framework is shown in Figure SO.03 below. Please note that this is subject to change over time and represents this domain's ABEs in release 15.5 of the SID. This Addendum will concentrate on defining the Service and ServiceSpecification Aggregate Business Entities (ABEs). Note that the Service Strategy and Plan is filled in with gray, which indicates it has not been fully developed.

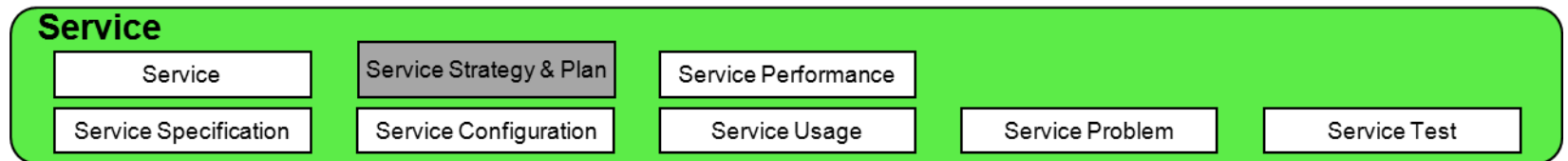


Figure SO.03 - Level One of the Service Domain of the SID Framework

1.1.2. Approach

As can be seen in Figure SO.03, the first level of the Service Domain is large and complex. The nature of this complexity lies both in the nature of each of the Aggregate Business Entities (ABEs) that are in the Service Domain in Figure SO.03 as well as in the interaction between entities of different ABEs in the Service domain. A third area of complexity is the interaction between entities in the Service Domain and entities in other Domains. However, the Service and ServiceSpecification ABEs, and the entities that they contain, form a “core” part of a Service model that, once designed, can anchor the other Service ABEs. Therefore, the focus of this Addendum is on the Service and ServiceSpecification ABEs.

The other focus of this Addendum is on the relationship between Service and Product entities; the relation to other major SID entities (e.g., Party) is also included in this release of this Addendum.

This Addendum is built with the express purpose of serving as a foundation for other Service (sub-) models to fit into. Conceptually, this Addendum is a framework, with other Service models (e.g., an MPLS VPN model) “plugging into” it.

This model reflects service entities from a business-oriented point-of-view, and uses the best of standard modeling patterns for this area to build in extensibility. For example, patterns are used to capture common relationships and occurrences of services as they relate to other services, and also Products, Resources, and other entities in other Domains. This helps make the model inherently extensible. Three important patterns utilized in this document are:

- Composite-Atomic pattern, which provides a common structure for stand-alone and aggregate objects of a given type
- Entity-EntitySpecification pattern, which separates the common characteristics of an object from its individual characteristics
- Entity-EntityRole pattern, which uses the notion of roles to statically or dynamically extend an object by defining roles it can play.

1.1.3. Overview of “Service”

This section outlines the approach taken to build the SID Service model. The approach used in this document is to build up pieces of the model gradually due to the complexity of the model.

An important goal of the SID effort is to reuse standards (as opposed to reinventing the same concepts) wherever possible. Since this model is a federated model that draws from different standards, we need a starting point for resolving the naming conflicts that will inevitably arise.

Consistent terminology in a federated model is critical. This terminology, as the model itself, will be introduced in stages. This enables more complex ideas to build on simpler concepts, thereby enabling both an easier as well as a more thorough understanding of the Service Domain as well as its relationship to other domains.

We will establish a mapping of terminology. For example, some people might think of a “VPN Service” without regard to how that Service is implemented. Other users of this Addendum might think of the architectural differences in how a VPN is constructed, and prefer more specific terminology such as a Layer 2 VPN or even an RFC 2547bis VPN. The framework developed in this Addendum must be able to accommodate either of these representations, as well as enable the decomposition of the Service, so that its constituent components can be appropriately represented.

We will over the course of this document define two fundamentally different types of Services. One is called a CustomerFacingService, while the other is called a ResourceFacingService. This enables us to model a wide variety of services in a common class hierarchy while differentiating between Services that are obtained as a Product by a Customer versus those that aren't. As we will see, a CustomerFacingService is one that is obtained as a Product by a Customer. Therefore, the Customer may have specific control over this Service via its associated Product. In contrast, the Customer never knows explicitly which ResourceFacingServices are being used to support a CustomerFacingService. More importantly, the Customer shouldn't have to know which ResourceFacingServices are being used, since the Customer hasn't explicitly obtained them. This fundamental concept is crucial to understanding the core of the Service model, and will be referenced throughout the remainder of this document.

1.1.4. Use Cases

The primary use case that drove the design of this model is to support Service creation and activation, as defined in the eTOM. A very important related use case is to set the groundwork for defining how devices can be configured in a standard way to support service creation, activation, measurement, and other processes defined by the eTOM.

There are countless examples of dependent use cases that come into play when considering the above use case. For example, one important use case is to be able to understand how different services present in a system support each other. Another example is to relate the physical infrastructure and topology to the logical topology of the system.

Another important dependent use case is the ability to describe how other standards can be implemented using this Service framework. For example, Differentiated Services and MPLS as defined by the IETF are prevalent in the Service Provider industry. Therefore, it is important to see how these Services can be mapped onto the SID Service Framework.

There are many other use cases that require a detailed or partial knowledge of specific Services and technologies.

1.1.5. Scope

This Addendum will focus on defining a framework for defining Services. Specifically, the set of classes defined in this Addendum will be able to be used as a foundation for two important activities:

- First, to define Services that can interact with other SID entities defined in other Addenda of GB922¹
- Second, to define ServiceLevelSpecifications, so that the work being done in other TMF teams on Service can be better associated with the work of the SID team

¹ This version of this Addendum is focused on defining the Service and ServiceSpecification ABEs of the SID Framework, as well as establishing links to the Product and Resource domains

1.1.6. A Starting Point – Supporting a Link to Products

Let's start by categorizing the key entities that need to be modeled:

- A Service that is a realization of a Product, that is obtained by a Customer – we'll call this a CustomerFacingService
- A Service that is indirectly part of a Product, but is invisible to the Customer – it exists to support one or more CustomerFacingServices. We'll call this a ResourceFacingService

The following is a simple model of how a Product is modeled.



Figure SO.04 -Simplified SID Product Model

The reason that Services are linked to Products is that the definition and use of a Product is essential to the overall SID model. Customers obtain Products, not Services. Hence, they need to be able to express their needs in plain English in order to determine which ProductOfferings support their requirements. Likewise, Service Providers and Enterprises need to be able to understand these requirements in order to realize the ProductOffering. A Product is in reality a “procured” instance of a ProductOffering; this is why the cardinality of the ProdOfferDescribes association is * on the Product side in Figure SO.04 above.

The above argument implies that Services can’t exist “in the ether” – rather, they must be bound to a particular Product. Logically, this makes sense – VPNs aren’t spontaneously created and freely handed out! This enables us to define a simple starting model, as follows (remember, this is not the complete model, and will be refined):

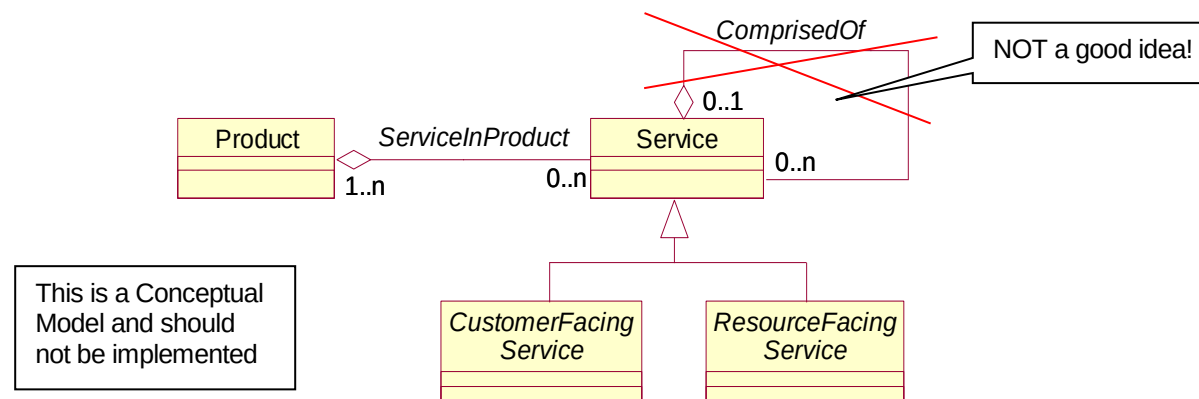


Figure SO.05 - Starting Point for Modeling Service

Figure SO.05 shows that zero or more Services are optionally used to realize a Product. The cardinality is zero or more in order to allow for the development of Services in a Service Provider or Enterprise before they are offered as a Product or ProductOffering.

The *ComprisedOf* aggregation is tempting to add, because it defines the ability to aggregate subordinate Services into a higher-level Service. However, this aggregation is a bad idea, as it enables ResourceFacingServices to aggregate CustomerFacingServices, which destroys the point of separating them in the first place. Hence, it will not be defined in this model.

The next step is to apply some more precise semantics to Figure SO.05. Specifically, a CustomerFacingService is what is bound to a Product, not a Service. ResourceFacingService is not linked directly to Product; rather, it is linked to Resource. In addition, an association between ResourceFacingService and CustomerFacingService should be shown. These changes yield the following starting model:

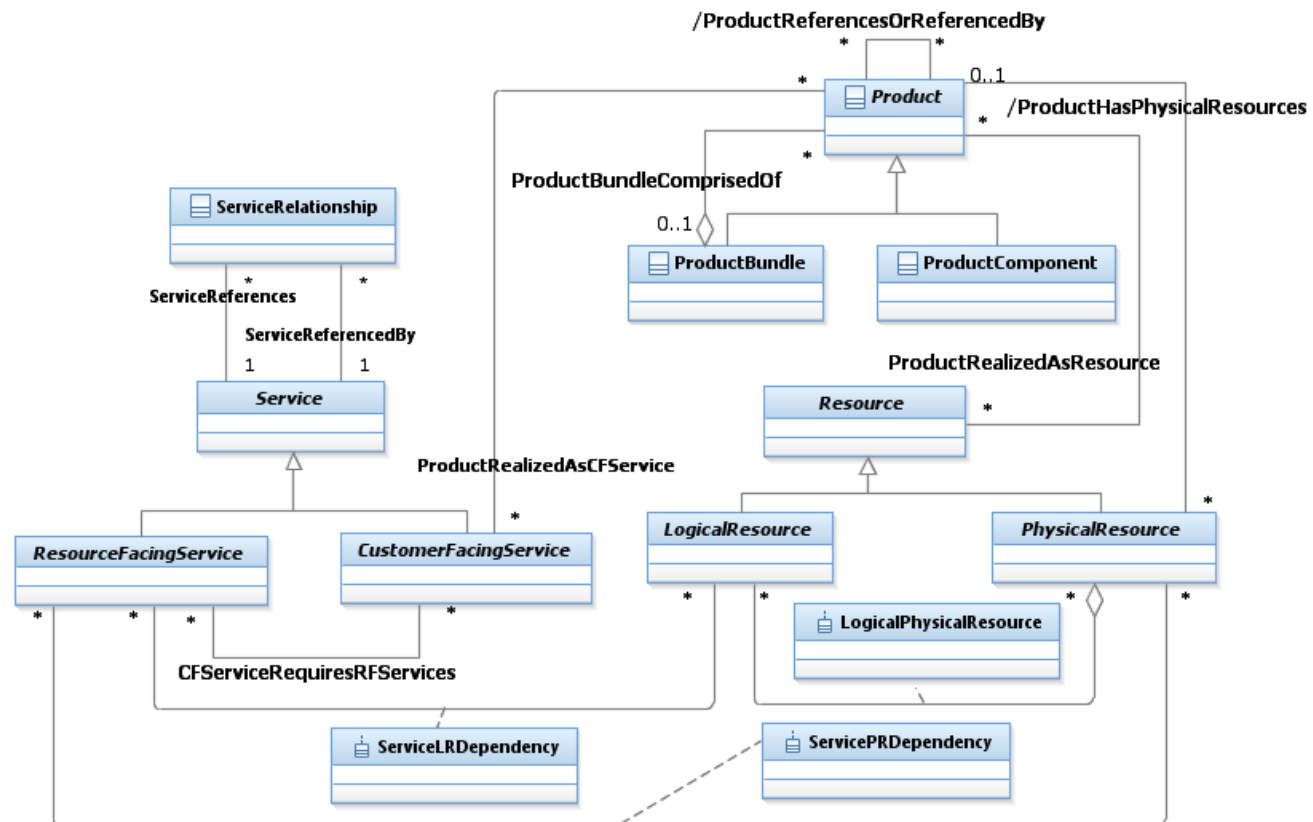


Figure SO.03 - Basic Service Model – A Starting Point

The *CFServiceRequiresRFServices* association defines the set of *ResourceFacingServices* that are required by a particular *CustomerFacingService* in order for that *CustomerFacingService* to operate correctly. A *ResourceFacingService* is related to *LogicalResources* and *PhysicalResources* through two association classes, *ServiceLRDependency* and *ServicePRDependency*. These association classes define the set of *LogicalResources* and *PhysicalResources*, respectively, which are required for this *ResourceFacingService* to function correctly. The *ServiceLRDependency* is used to define the set of *LogicalResources*, such as memory and *DeviceInterfaces*, which are required for this particular *ResourceFacingService* to function correctly. The *ServicePRDependency* have a slightly different connotation. This defines the set of *PhysicalResources*, such as *PhysicalDevices* and *PhysicalPorts*, which are required for this particular *ResourceFacingService* to function correctly. Thus, the former association class defines the *operational* aspects of the *ResourceFacingService*, while the latter defines its *hosting* aspects. This separation is, of course, facilitated by the design of the Service (*CustomerFacingServices* vs. *ResourceFacingServices*) and Resource (*PhysicalResource* vs. *LogicalResource*) models.

The *LogicalPhysicalResource* association class represents the binding between a *PhysicalResource* and a *LogicalResource*. In other words, it defines the set of *LogicalResources* that a particular *PhysicalResource* supports. For example, in order for a Service to be instantiated on a Resource, it might need to operate on a *PhysicalPort* and utilize some amount of memory and other resources residing in the Resource (these latter are represented by the concept of a *LogicalResource*).

Note that a Product has two subclasses, *ProductComponent* and *ProductBundle*. A *ProductComponent* represents a stand-alone Product, while a *ProductBundle* represents Products formed from other Products. Clearly, either of these can serve as the instance that is being realized through a combination of *CustomerFacingServices* and Resources.

The model shown above in Figure SO.03 was built to distinguish between two types of Services – those that are used to realize a Product, and those that are required to implement the functionality provided by a Product. These are called *CustomerFacingServices* and *ResourceFacingServices*, respectively.

In addition, note the strong link between infrastructure entities and business entities, such as a Product. In particular, this model ensures that Customers access Resources and Services through their packaging in a Product. This particular modeling of the complex abstractions and relationships between Customer, Product, Service and Resource provides some of the key underpinnings that support what the SID is all about.

1.1.7. A Simple Example

Let's consider the simple example of a Customer purchasing a VPN. The VPN itself may be both a Product and a CustomerFacingService, since the VPN provides a number of functions (e.g., connectivity) that people commonly refer to as Services. Suppose the VPN is a particular type of VPN, defined by RFC2547bis. This specification mandates that BGP (the Border Gateway Protocol, which is an inter-domain routing protocol that is used to exchange reachability information with other BGP systems) is used for route advertisement. The Customer doesn't buy BGP services, yet BGP is required for this type of VPN to function. Therefore, BGP is a ResourceFacingService.

Figure SO-07 takes an extract from the LogicalResource model and shows that BGP is a particular type of RoutingProtocol. This model defines several different types of Protocols. A RoutingProtocol is a particular type of protocol that accomplishes routing² through the implementation of a specific algorithm. Protocols are modeled as a type of LogicalResource.

(To connect this properly, we also need the concept of a LogicalDevice. This is the logical analog to the PhysicalDevice class.) A LogicalDevice is an abstract base class for all logical resources that are inherently manageable and make up a Product. It is used to describe different types of logical features and services that constitute a Product. It has two main purposes: (1) to collect common attributes and relationships for all logical resources, and (2) to provide a convenient, single point where relationships with other managed objects can be defined.)

Thus, a LogicalDevice *uses* a particular type of protocol to accomplish a given logical function. In this example, the LogicalDevice represents the “thinking” part of the router – the part that can use a routing protocol to connect a source to a destination. This is modeled by the *DevicePVRoutingDetails* association class in Figure SO-07.

² Routing is defined as the process of finding a path to a destination.
© TM Forum 2016. All Rights Reserved.

Thus, BGP is associated with a ResourceFacingService through the *RFServicesUsesProtocol* aggregation defined in Figure SO.07, as follows:

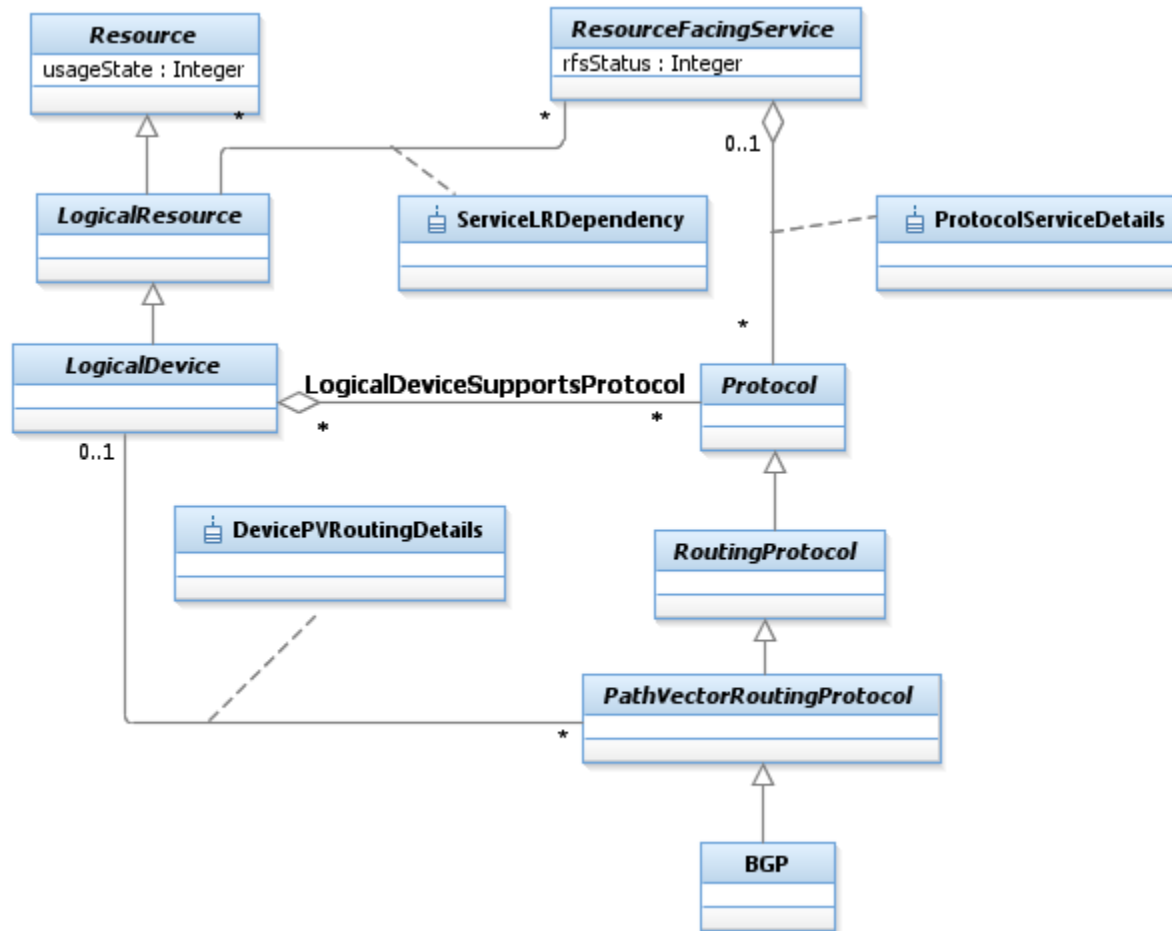


Figure SO.07 - Representing BGP as a LogicalResource

The *ProtocolServiceDetails* association class defines the Protocols that are required by a particular ResourceFacingService.

The *LogicalDeviceSupportsProtocol* aggregation defines the set of Protocols that this LogicalDevice can properly support. This helps define how these Protocols are configured..

The important point of this example is that the VPN Product requires a set of Services. Specifically, our VPN Product requires a combination of the BGP ResourceFacingService (and others, but let's keep it simple for the time being) and the VPN CustomerFacingService (please refer to Figure SO.03). The utility of Figure SO.03 is that it provides the ability to “package” multiple heterogeneous Services together to realize a Product. This models a CustomerFacingService requiring a set of ResourceFacingServices through the CFServiceRequiresRFServices association.

Continuing the example, the VPN Product that the Customer bought may already have a router that the Customer can use to access the VPN, or the Customer may need to purchase one. This is represented using the *ProductHasPhysicalResources* association defined in Figure SO.03. The *ServiceLRDependency* and *ServicePR Dependency* association classes in that figure model a particular ResourceFacingService as depending on, or being supported by, a set of LogicalResources and PhysicalResources, respectively.

Equally clearly, a Service may depend on one or more specific PhysicalResources. For example, the Customer connects to the VPN through a PhysicalPort on a Card of a PhysicalDevice. This is represented using the *RFSERVICEConfiguredUsing* association, whose purpose is to bind Resources to Services without identifying what type of Service or Resource is being bound. This is shown in Figure SO.08

Error! Reference source not found., which is in the next section, titled REF_Ref43438502 \h ServiceSpecification. For example, this enables services to be “reserved” on a PhysicalResource for resource planning and consumption purposes, without specifically identifying the low-level system resources required.

The above discussion has been conducted in terms of “instances” of these objects. For the most part we will talk about these concepts more generically using the concept of a “specification”. This will be discussed in the following section.

1.1.8. ServiceSpecification

The Product guide book refers to the concept of a ProductSpecification. As a reminder, ProductOfferings are tangible or intangible entities that organizations, such as Service Providers and Enterprises, market, sell or lease to Customers. A ProductSpecification is a detailed description of a tangible or intangible object made available externally in the form of a ProductOffering to Customers or other Parties playing a particular PartyRole. The information described by a ProductSpecification is a set of invariant data that is common to all realizations of that ProductSpecification. A ProductSpecification may exist as a single ProductSpecification, or consist of a set of ProductSpecifications supplied together as a collection.

Similarly, one can define the concept of a ServiceSpecification. ServiceSpecification is an abstract base class for representing a generic means for implementing a particular type of Service. In essence, a ServiceSpecification defines the common portions of a set of Services, while Service defines a specific instance that is based on a particular ServiceSpecification. This relationship is represented by the *SpecifiesService* dependency.

The foundation of the ServiceSpecification model is represented in Figure SO.08**Error! Reference source not found.** below. Note that a direct association between Product and Service does not exist. Instead, the *ProductSpecRealizedAsCFSSpec* association is defined between ProductSpecification and CustomerFacingServiceSpec. Remember that the purpose of a Specification is to serve as a template. This enables specific instances of a desired entity to be more easily and consistently created. The *ProductSpecRealizedAsCFSSpec* association enables a given ProductSpecification to define a set of CustomerFacingServiceSpecs that will be used to realize it. This enables multiple Products to be created from the same ProductSpecification, where each Product uses the same set of CustomerFacingServices. This is because the same set of CustomerFacingServiceSpecs (which of course define the same set of CustomerFacingServices) are defined by the ProductSpecification used to define the set of Products.

Note in particular that there is no association defined between a ProductSpecification and a ServiceSpecification. This is because such an association would include ResourceFacingServiceSpecs, and that violates the effort to separate CustomerFacingServices from ResourceFacingServices. Thus, the *ProductSpecRealizedAsCFSSpec* is defined to *enforce* this separation.

The *ProductSpecRealizedAs* association enables a ProductSpecification to define zero or more ResourceSpecifications. This is analogous to the *ProductSpecRealizedAsCFSSpec* association.

This is shown in Figure SO.08**Error! Reference source not found.** below.

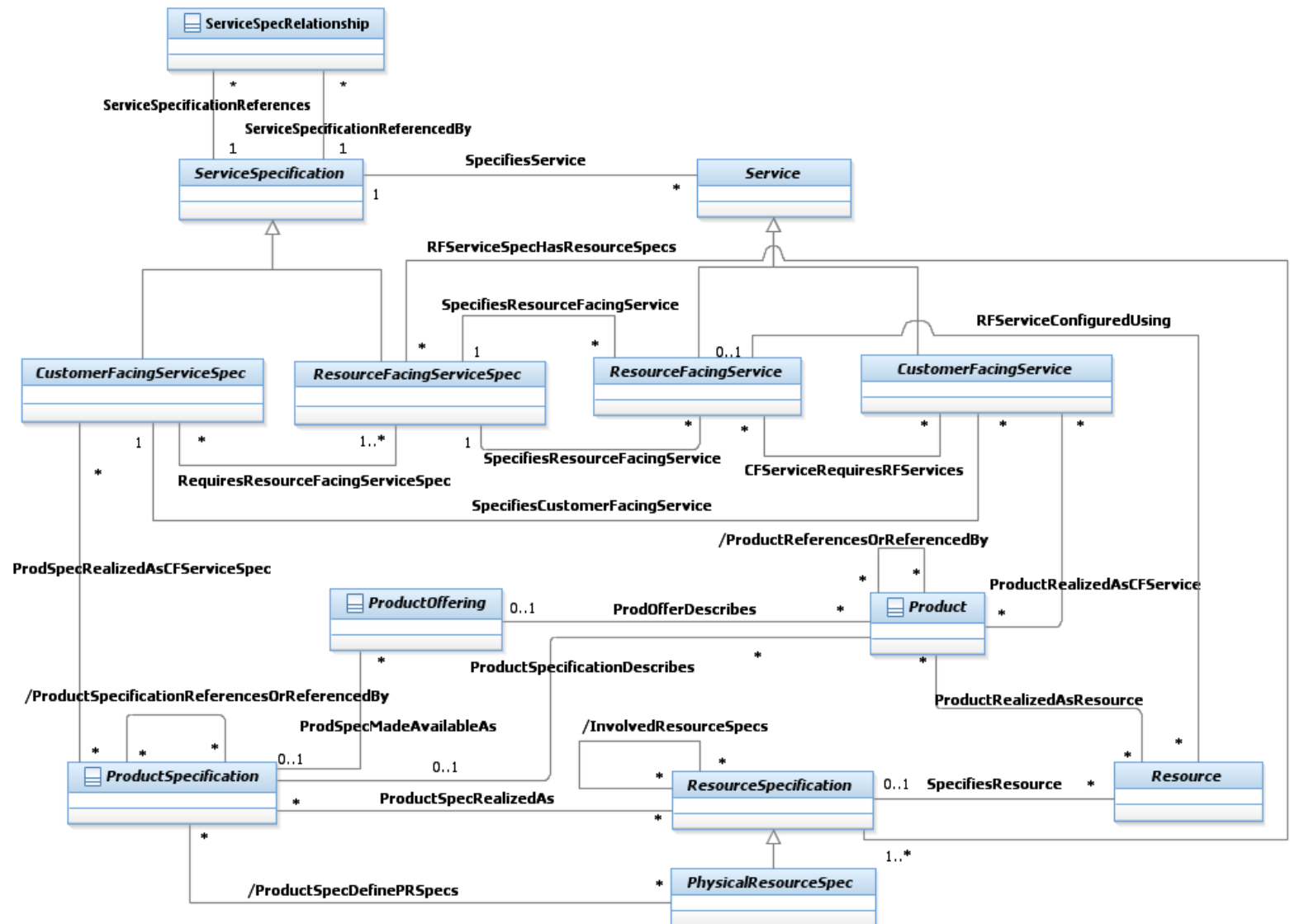


Figure SO.08 - Service Specifications

Conceptually, a ServiceSpecification defines the attributes and relationships for defining a set of Services. A Service uses these invariant characteristics and adds instance-specific characteristics to them such that, by building from a base of standard capabilities, it becomes possible to realize unique ProductOfferings that are specifically targeted at different users, market segments, and so forth.

ServiceSpecifications are generic in nature, just as Services are. The two subclasses of ServiceSpecification, CustomerFacingServiceSpec and ResourceFacingServiceSpec, are defined in an analogous manner to the two subclasses of Service, CustomerFacingService and ResourceFacingService. The *SpecifiesCustomerFacingService* association defines a set of CustomerFacingServices that can be created from a particular CustomerFacingServiceSpec. Similarly, the *SpecifiesResourceFacingService* association defines a set of ResourceFacingServices that can be created from a particular ResourceFacingServiceSpec.

In Figure SO.08**Error! Reference source not found.** the *ProductSpecRealizedAsCFSSpec* association defines the set of CustomerFacingServiceSpecs that realize a particular ProductSpecification. A ServiceSpecification can classify a set of Services using the *SpecifiesService* association. More specifically, CustomerFacingServiceSpecs and ResourceFacingServiceSpecs are both derived from a ServiceSpecification, and represent more specific types of Services. They each *specialize* the *SpecifiesService* association by defining their own associations, called *SpecifiesCustomerFacingService* and *SpecifiesResourceFacingService*, respectively. The CustomerFacingServiceSpec and ResourceFacingServiceSpec will define shared characteristics of CustomerFacingServices and ResourceFacingServices, which are inherited by CustomerFacingServices and ResourceFacingServices using the *SpecifiesCustomerFacingService* and *SpecifiesResourceFacingService* associations, respectively. Each CustomerFacingService and ResourceFacingService can then add its own specific characteristics to the CustomerFacingService and ResourceFacingService *instances*.

Note that the *ProductSpecRealizedAsCFSSpec* association serves a similar purpose as the *ProductSpeciRealizedBy* association to ResourceSpecification, except it is targeted at defining a set of ResourceSpecifications. Similarly, a ResourceSpecification can classify a set of Resources using the *SpecifiesResource* association, so its two subclasses, PhysicalResourceSpec and LogicalResourceSpec, inherit this association. Thus, the combination of the *ProductSpecRealizedAsCFSSpec* and the *ProductSpecRealizedBy* associations enables a ProductSpecification to define the set of CustomerFacingServiceSpecs and ResourceSpecifications that it is realized by. This in turn means that different Products defined by the same ProductSpecification automatically inherit the set of CustomerFacingServices and Resources specified by the associated set of CustomerFacingServiceSpecs and ResourceSpecifcaitons that are attached to that ProductSpecification.

A Product is realized by zero or more CustomerFacingServices through the *ProductRealizedAsCFService* association. This enables a Product to in effect customize a CustomerFacingService already defined by a CustomerFacingServiceSpec. In other words, the *ProductSpecRealizedAsCFSSpec* association is used to define the set of CustomerFacingServiceSpecs for a given ProductSpecification, which means that all Products based off of that ProductSpecification will use that set of CustomerFacingServiceSpecs. Different Products have the ability to add their own features to these CustomerFacingServices, and do so through the *ProductRealizedAsCFService* association.

Similarly, a Product realizes zero or more Resources through the *ProductRealizedAsResource* association. This is explained in and shown in Figure SO.03. Note that the cardinality on the Product side of both of these associations is optional, which means that a Product doesn't have to implement these two associations. This is because not all Products are realized through CustomerFacingServices and Resources.

CustomerFacingServices cannot be implemented without ResourceFacingServices to support them. This is reflected by the *CFServiceRequiresRFServices* association. This relationship is templated by the *RequiresResourceFacingServiceSpec* association, which defines the set of ResourceFacingServiceSpecifications that are required by a particular CustomerFacingServiceSpecification. Again, note that the cardinality on the ResourceFacingServiceSpec side is 1,*, meaning that a CustomerFacingServiceSpec cannot be instantiated unless it is bound to at least one ResourceFacingServiceSpec. Similarly, both a CustomerFacingServiceSpecification as well as a ResourceFacingServiceSpecification can define a set of CustomerFacingServices and ResourceFacingServices that can be created from them. This is done using the *SpecifiesCustomerFacingService* and *SpecifiesResourceFacingService* associations, respectively.

Let's now examine a practical example. A ProductSpecification will be used to define a Service offering. A ServiceSpecification will define how to build a VPN to implement the Service offering defined in the ProductSpecification. Now, there are many different types of VPNs. For example, there are network-based VPNs, VPNs based on tunneling, VPNs based on encryption, hybrid VPNs such as MPLS VPNs, and others. All of these VPNs have certain things in common, which can be represented by a VPNServiceSpecification. This is shown in Figure SO.09 below.

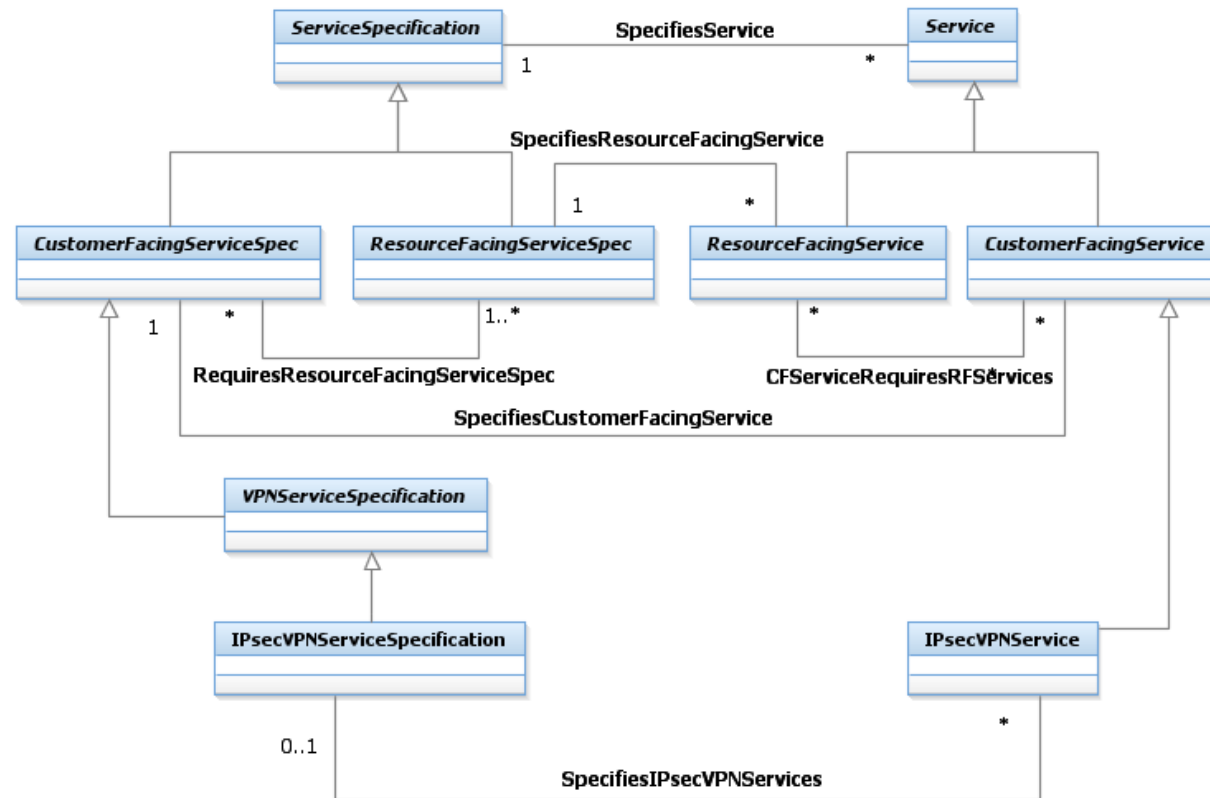


Figure SO.09 - VPNService – VPNServiceSpecification Example

An MPLS VPN is implemented in a significantly different manner than other types of VPNs. Thus, an MPLSVPNService is a different type of object than (for example) an IPsecVPNService. However, they have three important things in common. First, they are both defined through a ProductSpecification and realized as a Product. Second, they can both be related to the same CustomerFacingServiceSpec. This is useful if, for example, the Service Provider is defining multiple types of VPNs that the Customer could use. Third, it is possible that they could both use the same CustomerFacingServices and/or ResourceFacingServices. This is shown above in Figure SO.09.

(Note that a more complete picture would also show that the two VPNs could use the same PhysicalResources and/or LogicalResources as well. However, this complicates the above figure and is out of scope for this Addendum.

The difference between the MPLSVPNService and the IPsecVPNService is in the protocols and architecture used, which in turn cause additional system-specific differences that don't show up in a business model. Rather, the SID identifies these two CustomerFacingServices and relates them to zero or more ResourceFacingServices through the *CFServiceRequiresRFServices* association (as well as the *RequiresResourceFacingServiceSpec* association from the template point-of-view).

A ResourceFacingServiceSpec is related to a ResourceSpecification by the *RFSpecHasResourceSpecs* association. Furthermore, a ResourceSpecification can be used to define the set of Resources that it requires using the *SpecifiesResource* association. This combination of associations means that conceivably, the same Resource could be used to support both types of VPNServices, assuming that it has the software and/or hardware to do so.

Note that in Figure SO.09, the VPNServiceSpecification is an abstract class that contains the basic attributes and relationships and constraints for defining a VPNService without specifying the specific type and technology used (e.g., Ipsec vs. MPLS) to implement the VPNService. For example, the VPNServiceSpecification can be used to define the set of SLSSs that will be used to measure performance of the VPN. In Figure SO.09, both an MPLS VPN as well as an Ipsec VPN specification are derived from the VPNServiceSpecification class.

The MPLSVPNServiceSpecification and IPsecVPNServiceSpecification are related to the MPLSVPNService and the IPsecVPNService through the *SpecifiesMPLSVPNServices* and the *SpecifiesIPsecVPNServices* associations, respectively. This enables different MPLSVPNServices (as well as IPsecVPNServices) to be specified by the same MPLSVPNServiceSpecification (or IPsecVPNServiceSpecification, respectively). This enables the Service Provider to define a number of characteristics for each of the CustomerFacingServices and thereby templatize the definition of each. Combined with the LogicalResource and QoS Addenda, a complete roadmap will be developed for configuring this templatized definition.

1.1.9. CustomerFacing and ResourceFacing ServiceSpecifications

There are three important additional aspects to consider regarding ServiceSpecifications. They are the notions of CustomerFacing and ResourceFacing ServiceSpecifications, types of ServiceSpecifications, and versioning of ServiceSpecifications. These concepts have already been introduced; the purpose of this section is to define them in greater detail.

Consider an MPLS VPN built to the RFC2547bis specification. The MPLS VPN is an example of a CustomerFacingService. This particular type of MPLS VPN is characterized by having three different types of routers. Each of these – the CustomerPremise, ProviderEdge, and Provider routers – can run a different set of protocols, perform a different function, and be managed in a different way. Remembering that our ServiceSpecification is being used by multiple CustomerFacingServices and ResourceFacingServices. Therefore, in order to better define similar characteristics for these types of routers, the ServiceSpecification class can be subclassed to capture these characteristics.

There are many different types of CustomerFacingServices. These can be grouped together into similar Services that each are related to a particular CustomerFacingServiceSpec. A CustomerFacingServiceSpec can therefore be used to define the common characteristics for a set of CustomerFacingServices. This scales by enabling multiple CustomerFacingServiceSpecs to be defined to model the common characteristics of different groups of CustomerFacingServices.

Similarly, there are many different types of ResourceFacingServices. Similar logic is used to justify how ResourceFacingServiceSpecs are used to characterize different groups of ResourceFacingServices. This is shown in Figure SO.4 below.

It is important to remember that the Specification classes are used to define attributes shared by each associated Entity, whether it is a Service, a Resource, a Product, or something else. This is a generic *pattern* that is used throughout the SID. Similarly, the Entities themselves, whether they are Services, Resources, Products, or something else, represent the *instances* that are *derived* from their associated Specification classes. This is why the cardinality on the “specification” side (e.g., CustomerFacingServiceSpec) of each association (e.g., *SpecifiesCustomerFacingService*) is defined to be “1” – the SID spec *mandate* the use of Specifications.

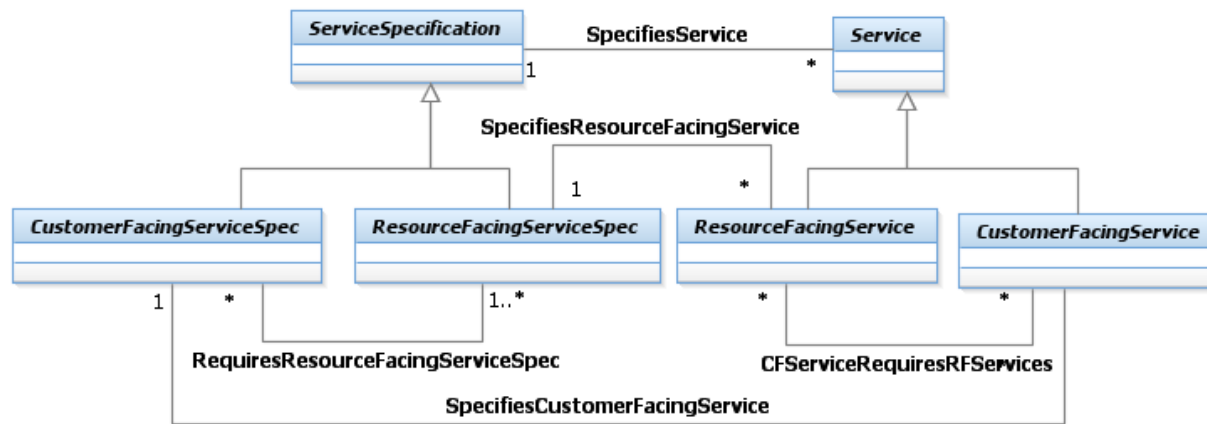


Figure SO.4 - Subclassing ServiceSpecification

1.1.10. Stand-Alone vs. Aggregate Services and ServiceSpecifications

There are two basic ways to define and use a ServiceSpecification. One is to define a stand-alone ServiceSpecification, while the other is to define an aggregate ServiceSpecification. The difference is straightforward – aggregate ServiceSpecifications are used to define collections of ServiceSpecifications, whereas stand-alone ServiceSpecifications define all the functionality that is needed by themselves.

If we did this, then the ServiceSpecAtomic would be defined as an abstract base class for defining ServiceSpecifications that do not have any subordinate ServiceSpecifications. These ServiceSpecifications are stand-alone in definition and use, and don't require any supporting ServiceSpecifications to define the common characteristics of Services that it serves as a template for.

Similarly, a ServiceSpecComposite class would be defined as an abstract base class for defining ServiceSpecifications that are formed by aggregating other ServiceSpecifications. The types of ServiceSpecifications that are aggregated may be ServiceSpecAtomic and/or ServiceSpecComposite instances. A ServiceSpecComposite collectively defines all of the common characteristics of Services that it serves as a template for.

However, we already have defined two subclasses of ServiceSpecification, called CustomerFacingService and ResourceFacingService. We can just as easily apply the composite pattern to these classes instead of defining a ServiceSpecAtomic and a ServiceSpecComposite class. This is exactly what is done in the model shown in Figure SO.5 below. We can reserve the right to define generic ServiceSpecAtomic and ServiceSpecComposite classes that are **not** CustomerFacingServices or ResourceFacingServices in the future. Note that at this time, no examples of such ServiceSpecifications exist. Therefore, we will proceed with the definition of CustomerFacingServiceSpecs and ResourceFacingServiceSpecs using the composite pattern to model this, as shown in Figure SO.5 below.

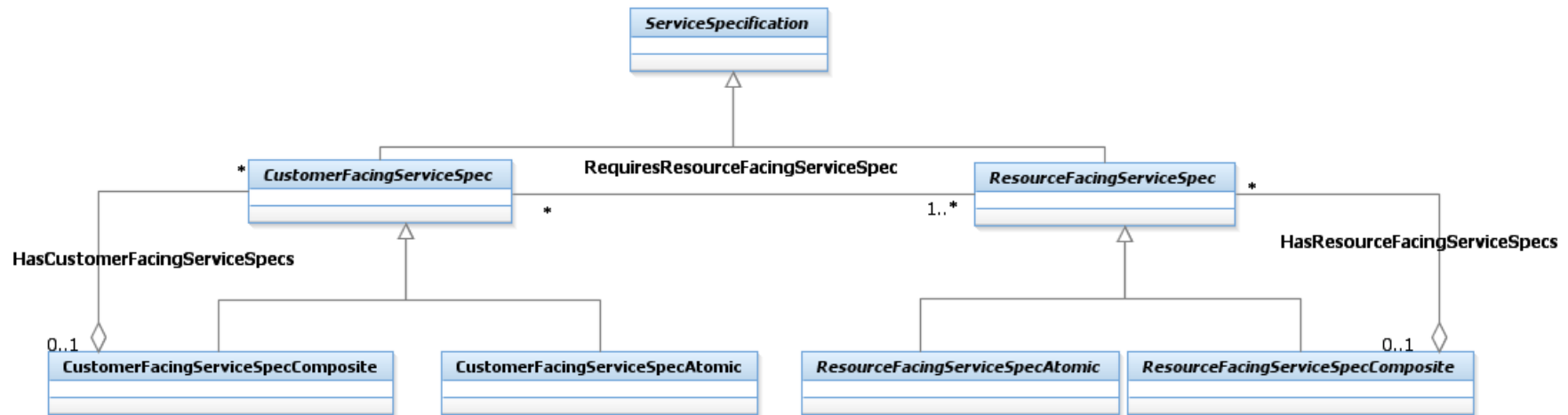


Figure SO.5 - Modeling Stand-Alone and Aggregate ServiceSpecifications

Similarly, CustomerFacingService and ResourceFacingService both use this same composite pattern. The difference is that since Services are in reality *instances* defined by a ServiceSpecification, the atomic and composite services are melded into CustomerFacingServices and ResourceFacingServices. In other words, generic Services are CustomerFacingServices and ResourceFacingServices themselves. While other types of Services could certainly be defined, their associated ServiceSpecification needs to be defined first. Figure SO.6 below shows the composite pattern applied to CustomerFacingServices and ResourceFacingServices.

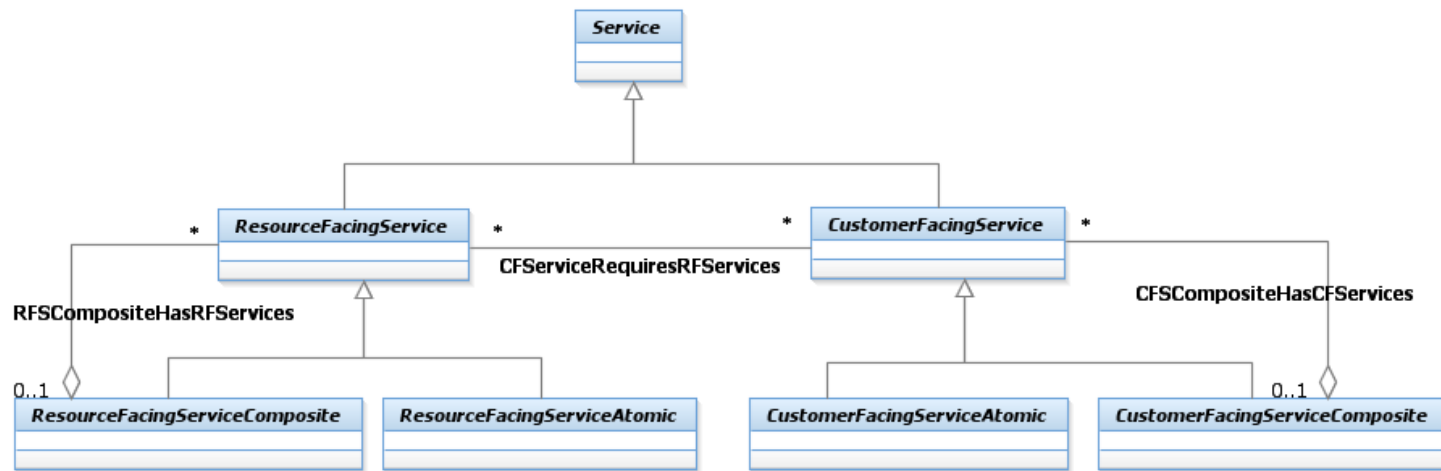


Figure SO.6 - Modeling Stand-Alone and Aggregate Services

The models in Figure SO.5 and Figure SO.6 are not adequate for representing more complex usages, such as a tiered Service Provider offering. In such a case, we need to enforce a better separation between which Services a given Service Provider has access to. This will be modeled after we include one additional concept – a ServiceRole. This is explained in the Section titled ServiceRoles and ServiceSpecRoles.

1.1.11. ServiceSpecVersion

Applications often require the ability to distinguish between different versions of objects in general. This Addendum applies this principal to ServiceSpecifications. The ServiceSpecVersion class represents a particular form or variety of a ServiceSpecification that is different from others or from the original. The form represents differences in attributes, relationships, and/or constraints that characterize this particular ServiceSpecification, but which are not enough to warrant creating a new ServiceSpecification.

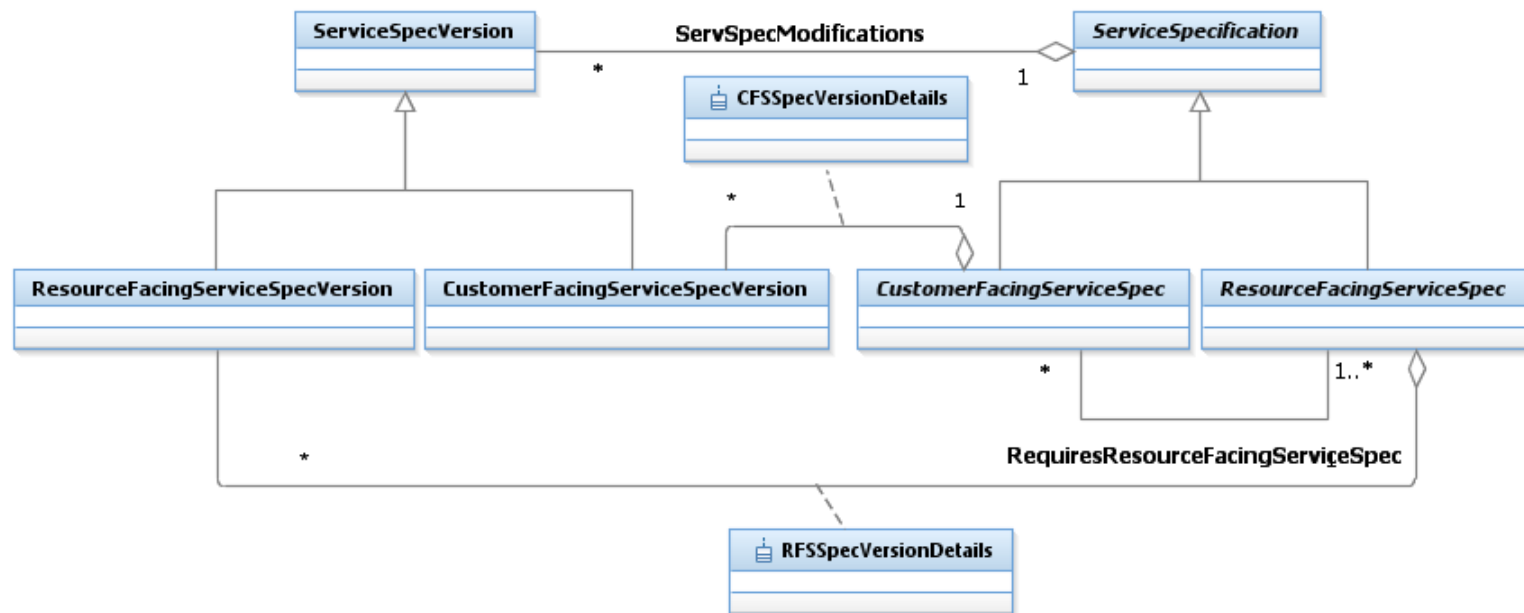


Figure SO.7 - Modeling the Versioning of a ServiceSpecification

The CustomerFacingServiceSpecVersion and ResourceFacingServiceSpecVersion subclasses are defined to enable an application to track different versions of CustomerFacingServiceSpecs and ResourceFacingServiceSpecs independently.

The *ServSpecModifications* aggregation represents the set of versions of this ServiceSpecification. The semantics of this aggregation are implemented with the ServiceSpecVersionDetails class.

The two version details association classes are both derived from the *ServSpecModifications* aggregation. They add additional detail to the generic concept of defining a version of a ServiceSpecification.

The same exact principle applies to tracking different versions of the more generic ServiceSpecAtomic and ServiceSpecComposite objects. This is not shown because the focus of this Addendum is on the definition of CustomerFacingServices and ResourceFacingServices.

1.1.12. ServiceSpecificationTypes

The preceding discussion has shown that multiple ServiceSpecifications can share common characteristics. This can be more efficiently represented using the concept of a ServiceSpecificationType, as shown in Figure SO.8 below:

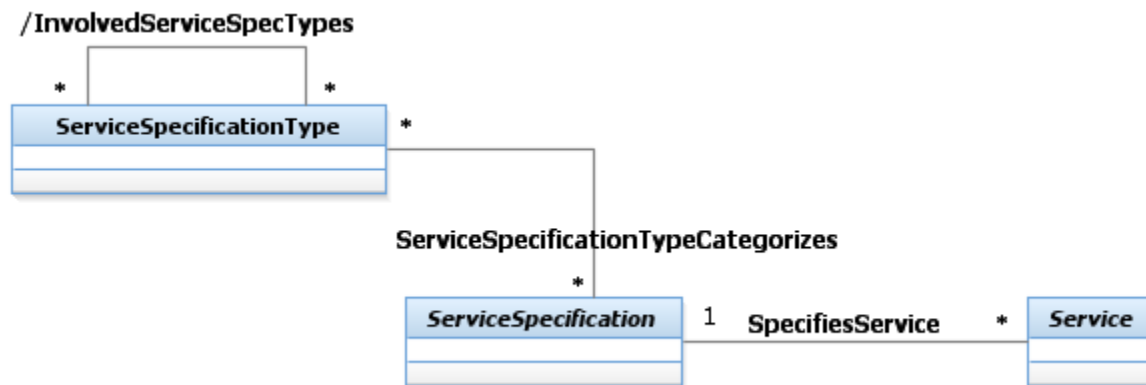


Figure SO.8 - The Concept of a ServiceSpecificationType

The `ServiceSpecificationType` class defines a generic category of `ServiceSpecifications`. Each `ServiceSpecificationType` serves to group a set of particular `ServiceSpecifications` that can be used together. The result of this is a more efficient ability to define a set of related `Services` that can be grouped together to form a higher-level `Service`. For example, a given higher-level `Service` might include VPN and QoS `Services`. If these `Services` are always used together, then they can be categorized using a common type.

The *`ServiceSpecificationTypeCategorizes`* association defines how `ServiceSpecificationTypes` categorize different `ServiceSpecifications`. The *`InvolvedServiceSpecTypes`* is a derived association as indicated by the “/” prefix. It is derived from an association between `EntitySpecificationTypes` which enables hierarchies of specification type entities to be defined.

Note that in principle, `ServiceSpecificationType` could be subclassed, so that the subclasses of `ServiceSpecificationType` could be used to categorize different subclasses of `ServiceSpecification`. This is not recommended for the purposes of this document, as the `ServiceSpecificationType` object is used solely to help categorize objects. The only reason a `ServiceSpecificationType` object should be subclassed is if that subclass adds additional semantics (in the form of attributes, relationships, and/or constraints). Thus, it makes perfect sense for applications that are based on the SID to subclass `ServiceSpecificationType`.

1.1.13. ServiceRoles and ServiceSpecRoles

Previously issued SID Addenda, such as the PhysicalResource Addendum and the Party Addendum, have made extensive use of roles. Roles can once again be used to simplify the modeling of different types of Services and to make the model inherently more extensible.

Instead of trying to either define many subclasses (one for each different type of Service) or introduce multiple inheritance (so that a new Service would derive from a set of existing Services), we can instead define a set of *roles* that a Service is meant to play. The purpose of the ServiceRole class is to define a Service in terms of a set of roles. The roles are then associated for both resource- and customer-facing services.

For example, consider the following simple 2547bis MPLS VPN in more detail.

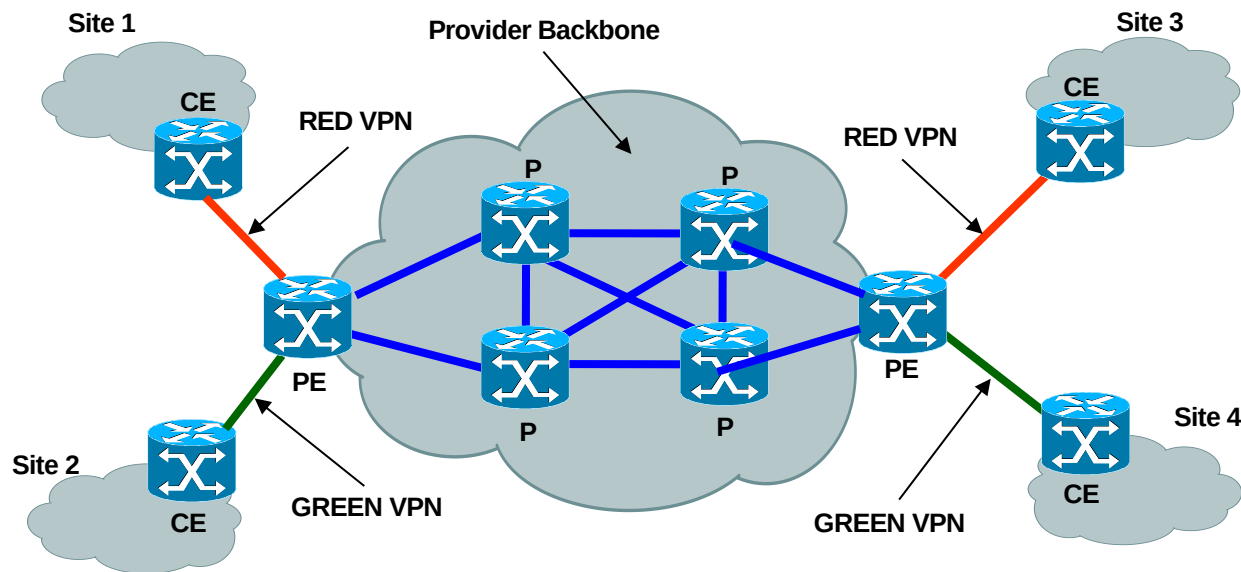


Figure SO.9 - Simple 2547bis MPLS VPN.

Figure SO.9 shows two VPNs. The “Red” VPN connects Sites 1 and 3 through the Provider backbone, while the Green VPN connects Sites 2 and 4 through the same Provider backbone. Note that these are two distinct VPNs that share a common network.

Three types of routers (CustomerEdge, or CE; ProviderEdge, or PE; Provider (core), or P) are shown in Figure SO.9. They are drawn in the above figure using the same icon to emphasize the fact that they are all routers, and need some other way to differentiate them. A preferred way of doing this is to use the concepts of roles.

Since the networking industry has shown itself to be ever-evolving, inheritance is not appropriate for capturing this behavior, as the behavior itself changes over time. In addition, by capturing the key concepts as roles, we can model present and future Devices as entities that can play many roles at one time.

The concept of a DeviceRole was introduced as a Physical Resource to represent complex behaviors such as these in an extensible manner. Figure SO.10 below shows an example of how LogicalDeviceRoles and DeviceInterfaceRoles can be used.

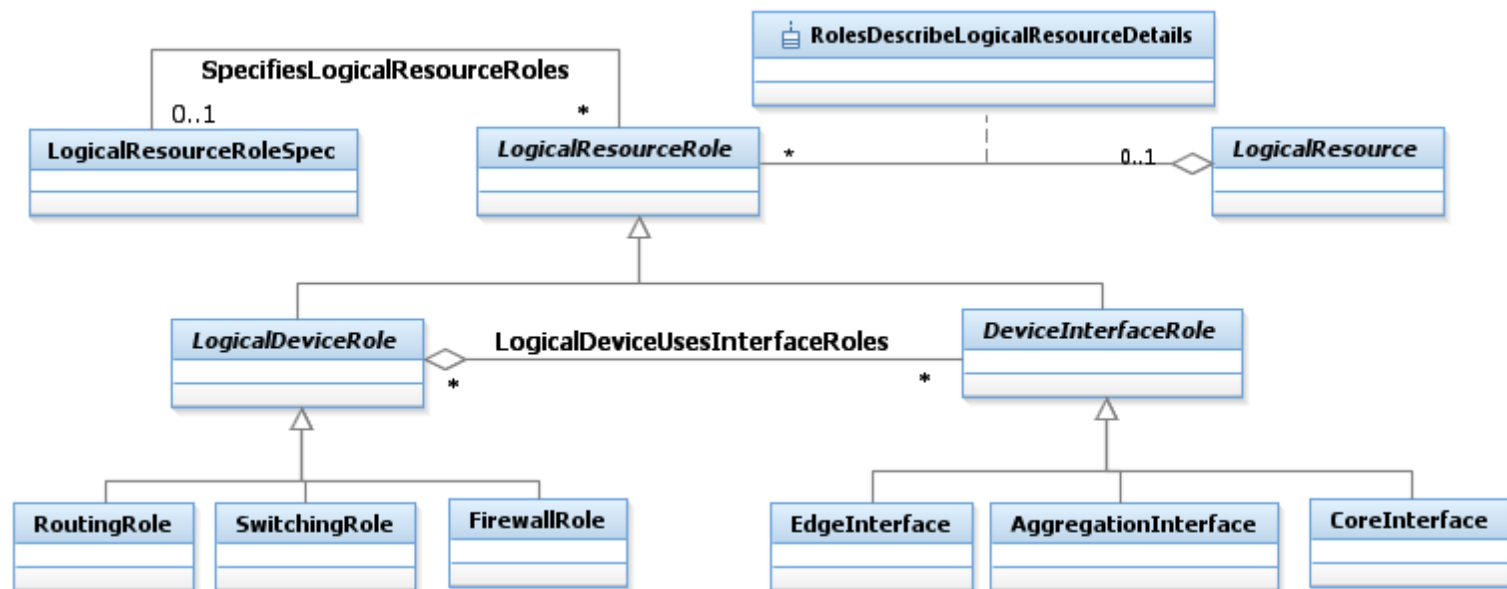


Figure SO.10 - The Concept of LogicalDeviceRoles and DeviceInterfaceRoles



Two types of LogicalResourceRoles are defined – LogicalDeviceRoles and DeviceInterfaceRoles. LogicalDeviceRoles abstract the functionality required of a device (such as a router). Some network devices can have a very large number (e.g., thousands) of DeviceInterfaces. However, most of the time, multiple DeviceInterfaces play the same set of roles. Thus, the DeviceInterfaceRole is used to abstract common functionality of multiple DeviceInterfaces.

Note that the three subclasses of LogicalDeviceRole as well as the three subclasses of DeviceInterfaceRole are exemplary only, as they represent commonly occurring functionality. They are not meant to be inclusive.

The *LogicalDeviceUsesInterfaceRoles* aggregation defines the set of DeviceInterfaceRoles that a particular LogicalDeviceRole can take on. For example, in Figure SO.10 above, consider the functionality required of a CE Device. On the one hand, it needs to connect to the Service Provider. This can be conveniently modeled as one DeviceInterface of a CE Device playing an EdgeInterface role connecting to one DeviceInterface of a PE Device. However, suppose that the CE Device has a need to aggregate multiple hosts in its Site (e.g., using VLANs). This DeviceInterface of the CE Device is not an EdgeInterface at all – rather, it is an AggregationInterface. Thus, we see that the CE Device router has one LogicalDevice that contains and supports two different DeviceInterfaceRoles.

It should be noted that Figure SO.10 does not define all of the different types of roles that a Device can take on. In particular, it focuses on LogicalResourceRoles.

Roles are a critical concept for modeling different types of devices, both physically as well as logically. For example, one can say that a CE router must run a particular set of protocols. This has both physical and logical implications. Physically, it means that the router must have the capability to run a particular protocol on a given media. This can be linked directly to the specification of a particular type of Card. Logically, it identifies a set of protocols to run and the software needed to process and execute those protocols. This is shown in Figure SO.11 below.

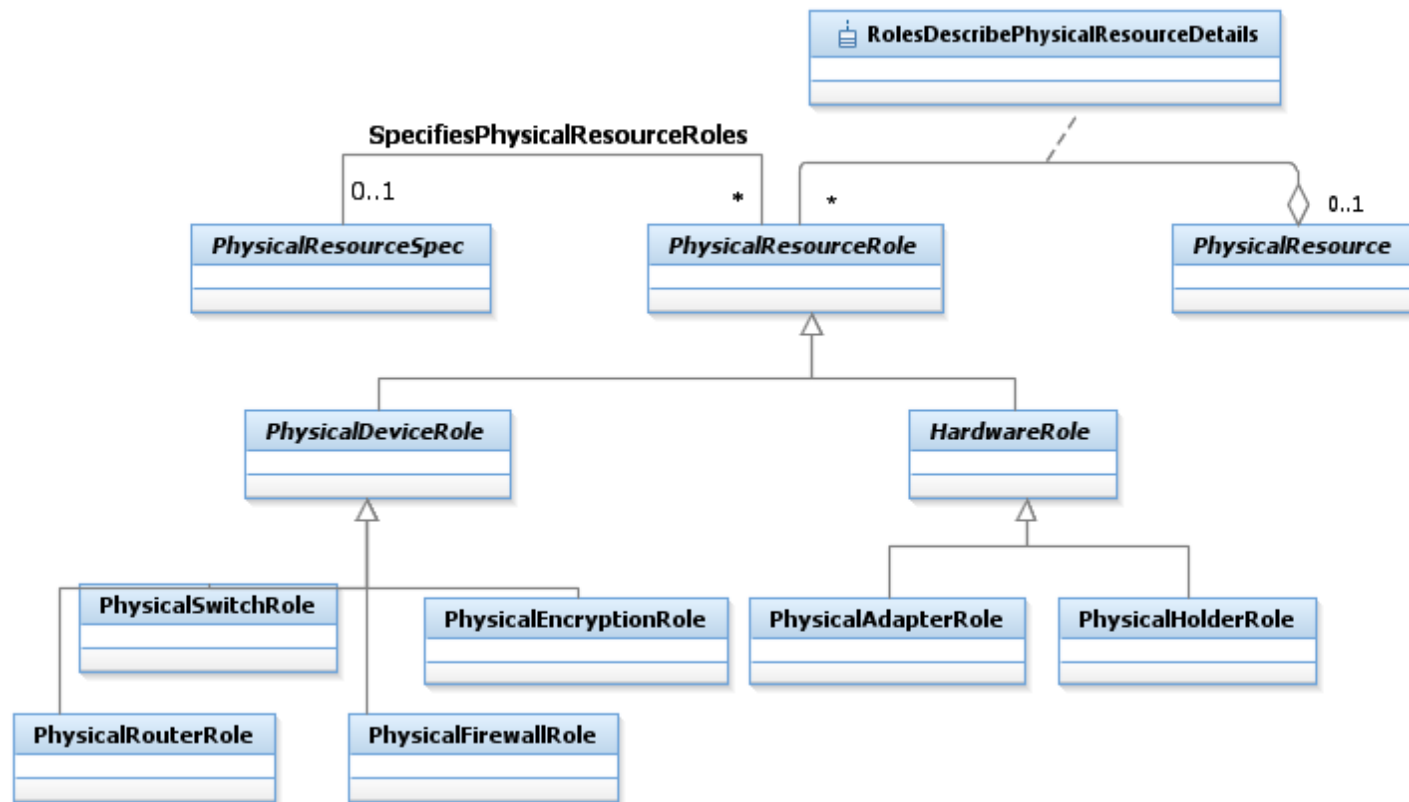


Figure SO.11 - The Concept of PhysicalResourceRoles

Clearly, there is a marked similarity between the PhysicalResourceRoles shown in Figure SO.11 and LogicalResourceRoles shown in Figure SO.10. This is indicative of a well-designed model that uses the same *patterns*.

Now, we need to merge the concept of a ResourceRole into the concept of a ServiceRole. This is shown in Figure SO.12 below for ServiceSpecRoles and ResourceSpecRoles.

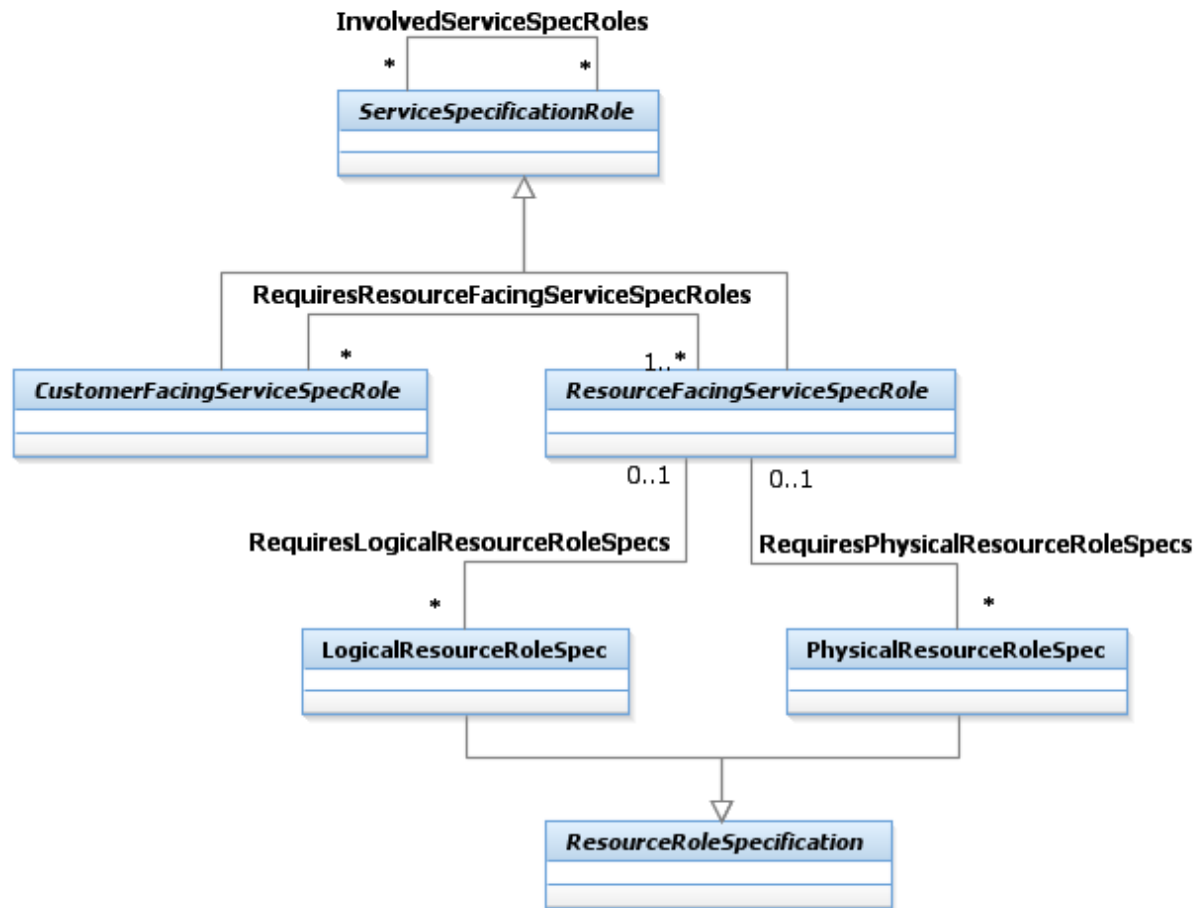


Figure SO.12 - Interaction Between ServiceSpecRoles and ResourceSpecRoles

The interaction between ServiceRoles and ResourceRoles (e.g., the instances derived from the templates) are similar in nature. Figure SO.13 on the next page shows a more complete (but still high-level) model of how roles are used to make a VPN design more scalable.

For our purposes, we will model the three types of roles that we have identified – CE, PE, and P routers – as three types of VPNRoles. A VPNRole is the superclass for various types of VPN role classes. For example, MPLS VPNs will use the CPELogicalDeviceRole, PELogicalDeviceRole, and PlogicalDeviceRole subclasses of this class to abstract functionality required for the CPE, PE, and P roles of an MPLS VPN. Other types of VPNs will use other subclasses of the VPNRole class.

The advantage of this class is that it enables different types of VPN roles to be specified by an MPLSVPNServiceSpecification. The other reason for specifying a VPNRole is because each of the subclasses of a VPNRole combines different LogicalDeviceRoles as well as DeviceInterfaceRoles. For example, a PELogicalDeviceRole also uses a policy to decide what routes to advertise. Thus, a PELogicalDeviceRole does routing and forwarding of traffic as well as filter traffic and apply QoS to traffic that it forwards.

The *RequiresHardwareSupportFor* association defines the PhysicalDeviceRole that a particular LogicalDeviceRole depends on. For example, if we want to use a statically defined route on a Serial connection to connect a CE router to a PE router, we can use this association to relate a CPELogicalDeviceRole to a CPEPhysicalDeviceRole.

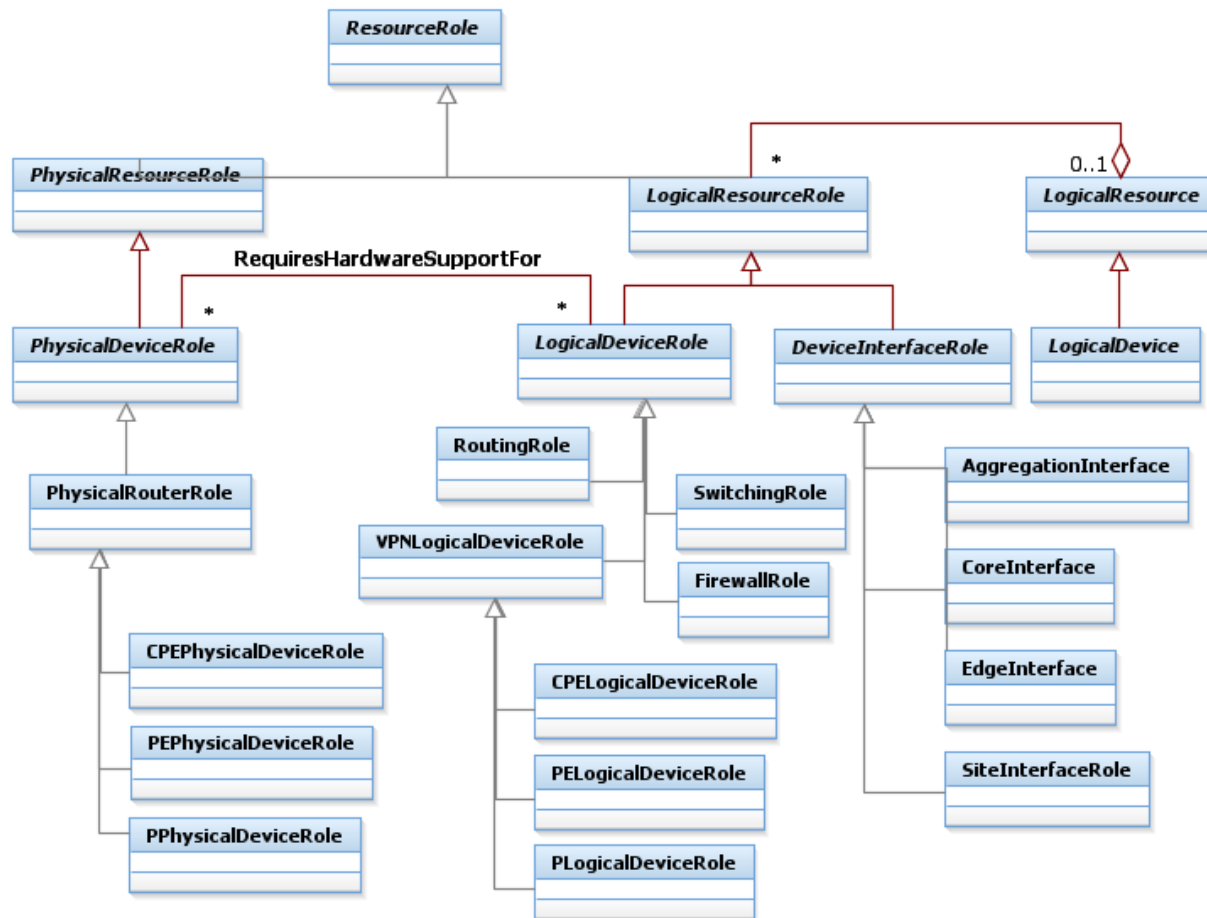


Figure SO.13 - VPN Roles

Figure SO.13 shows how different physical and logical roles can be defined to support the definition of a VPN. However, we still haven't linked similar concepts to associate ServiceRoles and ServiceSpecificationRoles to a VPN. This can be done as shown in Figure SO.14 below.

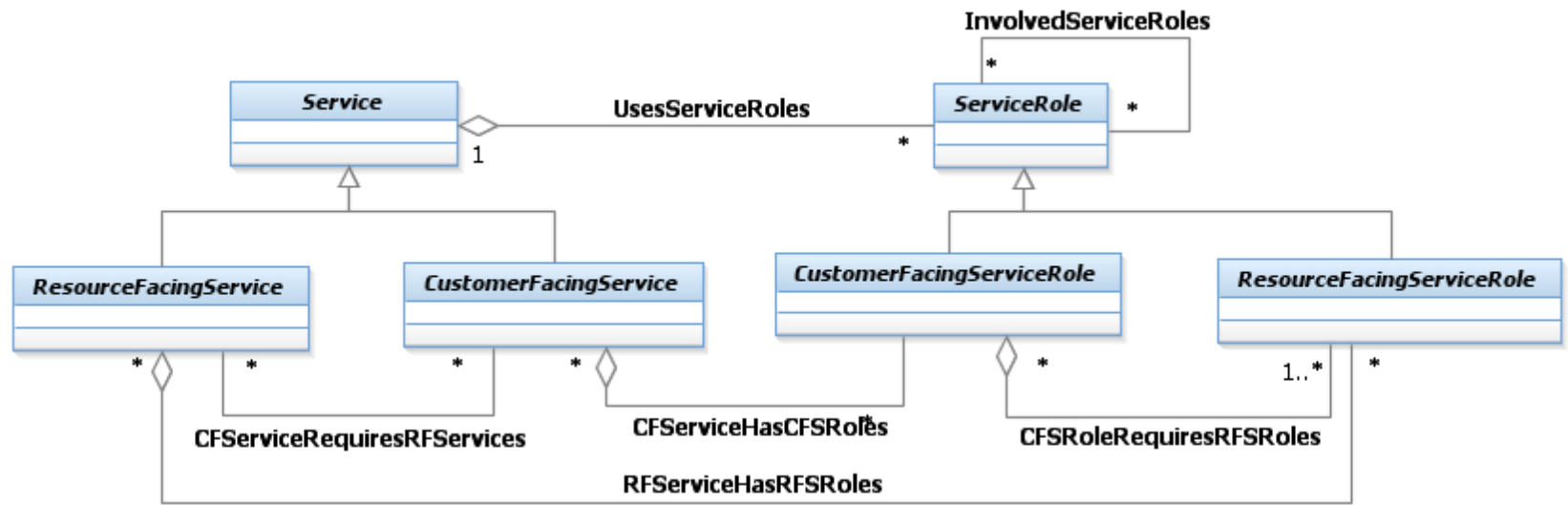


Figure SO.14 - The Concept of ServiceRoles

The `ServiceRole` class defines a `Service` in terms of a set of roles. The roles are then associated for both `CustomerFacing` and `ResourceFacing` services through the `CustomerFacingServiceRole` and `ResourceFacingServiceRole` subclasses. The utility of the `ServiceRole` class is to present a single point for accumulating common relationships that its subclasses can inherit.

The `CFSRoleRequiresRFSRoles` aggregation is used to define the set of `ResourceFacingServiceRoles` that are required by a particular `CustomerFacingServiceRole`. Conceptually, this enables roles to be used to define the common characteristics of a set of `ResourceFacingServices` that are required by a particular `CustomerFacingService`.

Note that the cardinality of the `CFSRoleRequiresRFSRoles` is `*` on the `CustomerFacingServiceRole` (aggregate) side and `1..*` on the `ResourceFacingServiceRole` (component) side. This is because a `ResourceFacingServiceRole` can exist without being bound into a `CustomerFacingServiceRole` (e.g., in testing the network), but a `CustomerFacingServiceRole` requires at least one `ResourceFacingServiceRole` to function. Note also that this cardinality is symmetric with the cardinality defined in the `CFServiceRequiresRFServices` association.

The `InvolvedServiceRoles` association is used to define a set of `ServiceRoles` that are involved with, or related to, each other in order to build a particular type of `Service`. This association applies equally to `CustomerFacingServices` and `ResourceFacingServices`. For example, consider the definition of a set of “QoS packages” that each consist of a set of different `Classes of Service`. Each `Class of Service` definition is associated with different types of traffic, and defines how each type of traffic should be conditioned independently of other types of traffic. The higher-level QoS package is clearly a `CustomerFacingService`, since it is obtained via a `Product` by a `Customer`. Equally clearly, the lower-level `Class of Service` definitions are each `ResourceFacingServices`, since the `Customer` has obtained a higher-level package and doesn’t have explicit knowledge of these sub-services. Now, assume that there is a dependency that exists between a sub-service and either another sub-service or the QoS package itself, such as a limit as to how many instances of one type of `Service` can be created, or that each sub-service must first be instantiated before a QoS package is completed. These are examples of the `InvolvedServiceRoles` association.

Continuing the example, the `CustomerFacingServiceRole` (e.g., “Gold Service”) is *built*, or *constructed from*, a set of `ResourceFacingServiceRoles` (e.g., “Rate Limiting Service” and “Remarking Service”). The former ensures that the `Customer` sends only the amount of traffic that he or she is contractually allowed to send, and the latter prioritizes different types of traffic. The Gold Service simply won’t work unless these two `Services` are built and active, but the `Customer` doesn’t need to explicitly know about them. These and other QoS functions are discussed in the QoS Service guide book.

Conceptually, ServiceRoles represent the variable characteristics of a Service from the viewpoint of functions (i.e., roles) that the Service provides to the rest of the environment. One example is that VPNs need to control how routing information is exchanged between the CE and PE routers at the edges of the Service Provider's backbone and between the PE routers across the Service Provider's backbone. ServiceRoles can be defined that represent how routing is constrained, depending if the router is a CE, PE, or P router.

This enables the Service to be managed abstractly using ServiceRoles. It also helps define the Service in terms of the functions that it has or provides. This is important, as it makes the management independent of the particular type of VPN and, more importantly, type of device and protocol used.

The CustomerFacingServiceRole and ResourceFacingServiceRole subclasses are used to represent the variable characteristics of CustomerFacingServices and ResourceFacingServices, respectively, in terms of roles that these services play. The set of roles define the Service in terms of the functions that it performs. This enables it to be modeled independently of the protocols, devices, and other dependent entities, as well as the environment in general.

ServiceSpecificationRoles can be defined in an analogous manner to how ServiceRoles were defined, and are shown in Figure SO.15 below.

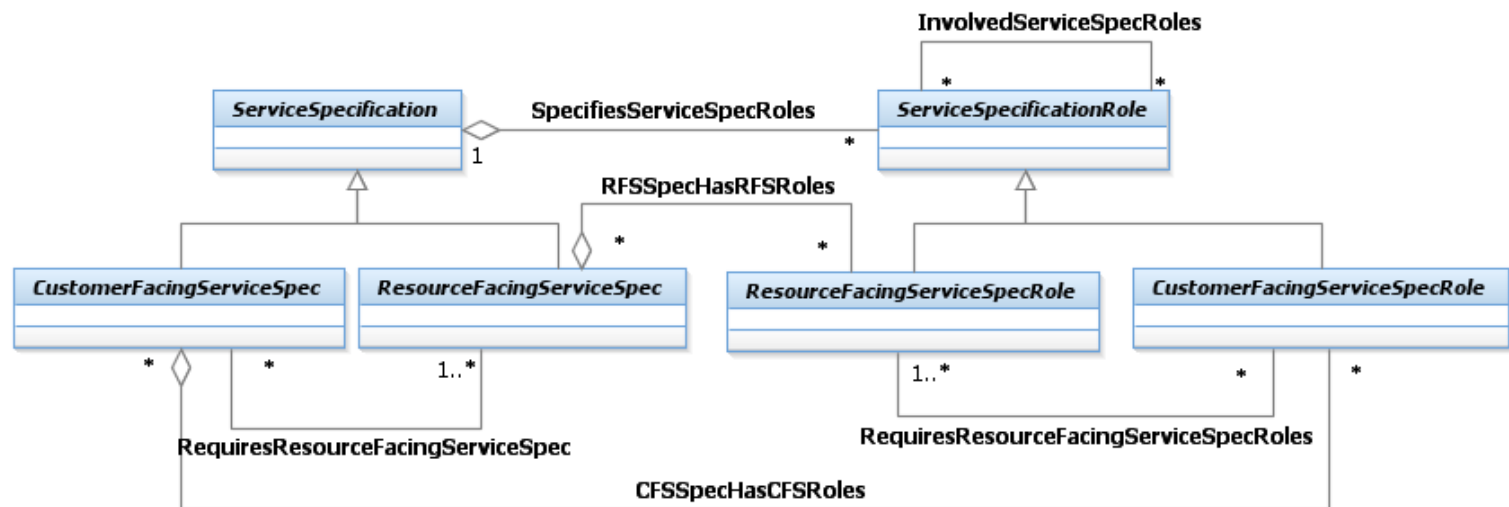


Figure SO.15 - The Concept of ServiceSpecificationRoles

The rationale for defining `ServiceSpecificationRoles` is exactly analogous to that of defining `ServiceRoles`. Specifically, a `ServiceSpecificationRole` defines a `ServiceSpecification` in terms of a set of roles. `ServiceSpecificationRoles` represent the invariant characteristics of a `ServiceSpecification` that make up a given `Service`. This enables the `ServiceSpecification` to be managed abstractly using `ServiceSpecificationRoles`. It also helps define the `ServiceSpecification` in terms of the functions that it has or provides.

This *`RequiresResourceFacingServiceSpecRoles`* aggregation defines the set of `ResourceFacingServiceSpecRoles` that are required to implement the functionality defined by this particular `CustomerFacingServiceSpecRole`. This aggregation can be thought of as enabling the `ResourceFacingServiceSpecRoles` to serve as reusable templates that, when combined, realize a `CustomerFacingServiceSpecRole` (which is, of course, also a template). The cardinality of the *`RequiresResourceFacingServiceSpecRoles`* aggregation is * on the `CustomerFacingServiceSpecRole` (aggregate) side and 1..* on the `ResourceFacingServiceSpecRole` (component) side. This is because a `ResourceFacingServiceSpecRole` can exist without being bound into a `CustomerFacingServiceSpecRole` (e.g., in testing the network), but a `CustomerFacingServiceSpecRole` requires at least one `ResourceFacingServiceSpecRole` to function.

1.1.14. ServiceSpecCharacteristic and ServiceSpecCharacteristicValue Classes

A major focus of marketing programs is to represent the key features of a Product. This was documented in the Product domain guide book as ProductSpecCharacteristics. Since a Service is the realization of a Product, it makes sense to reuse this concept and define the notion of a ServiceSpecCharacteristic. This is shown in Figure SO.16 below.

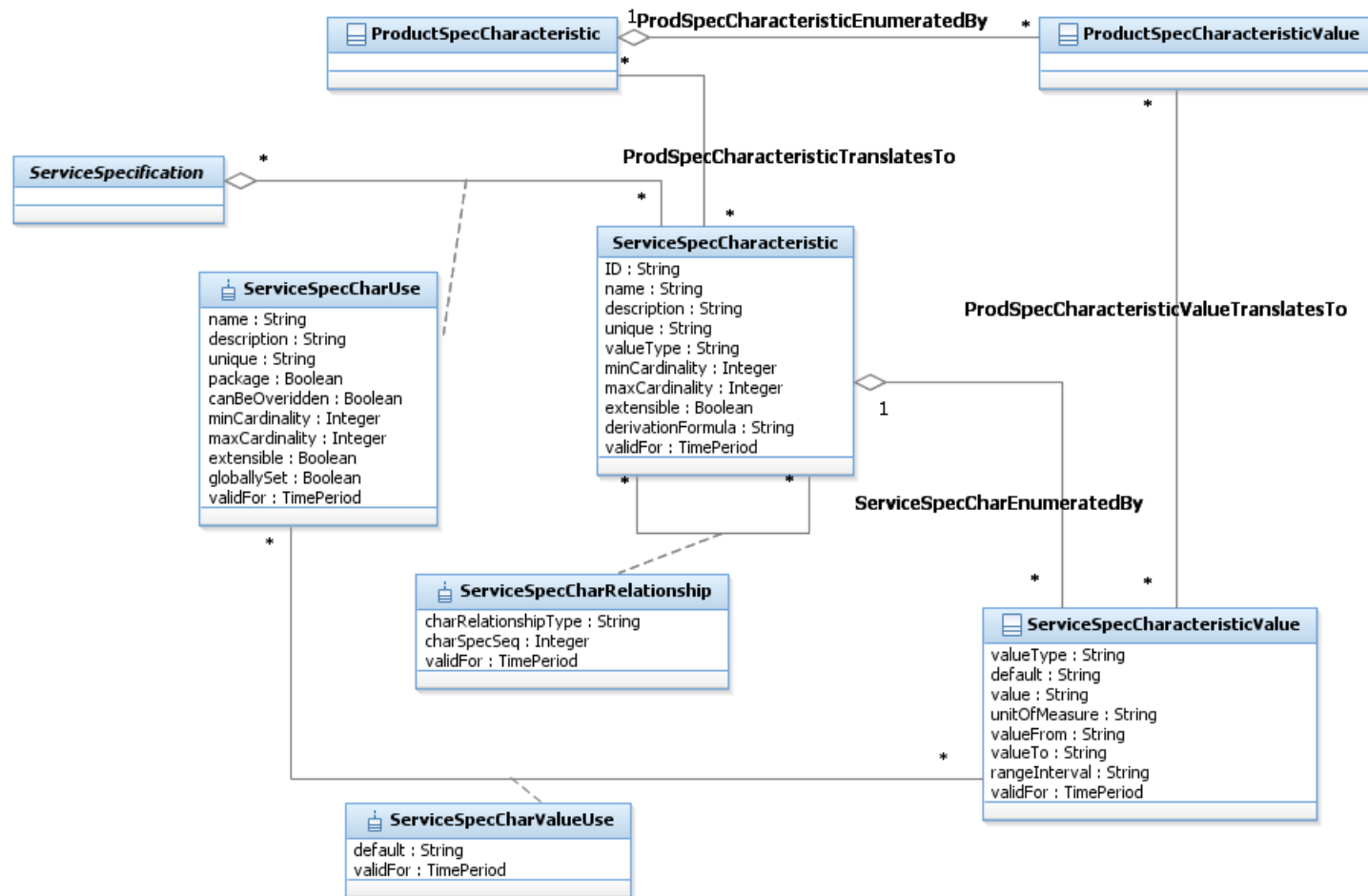


Figure SO.16 - The Concept of ServiceSpecCharacteristic

The concept behind a ServiceSpecCharacteristic and a ServiceSpecCharacteristic Value is similar in nature to those of a ProductSpecCharacteristic ProductCharacteristic Value, respectively. Services have a large variety of important properties (which we call characteristics), such as bandwidth, type of QoS, and so forth. Each of these characteristics is used at the business level to characterize a Service. This Addendum models these characteristics as a set of entities, attributes, and relationships that help characterize a ServiceSpecification and Service, respectively.

Additional information about the CharacteristicSpecification/CharacteristicValue pattern can be found in the Root Business Entities guide book.

ServiceFeatures and ServiceParameters are two other synonyms for a ServiceSpecCharacteristic, and are used in models from other TM Forum member companies.

ServiceSpecCharacteristics and ServiceSpecCharacteristicValues capture the differentiating features of a particular Service. However, ServiceSpecCharacteristics are not versionable objects. This is because ServiceSpecCharacteristics do not *add* any information in and of themselves – rather, they are used to emphasize key features of a ServiceSpecification.

There are cases when an instance of a ServiceSpecCharacteristic and/or a ServiceSpecCharacteristicValue must be translated to a corresponding ResourceSpecCharacteristic and/or a ResourceSpecCharacteristicValue. For example, there may be a ServiceSpecCharacteristic that represents the size of an email box as defined by associated 10MB instance of ServiceSpecCharacteristicValue. This value may be translated into the actual size of 10.5MB as defined by the ResourceSpecCharacteristicValue associated to an email box size ResourceSpecCharacteristic. Figure SO.22a shows the associations between characteristics in the Service and Resource domains that enable this translation.

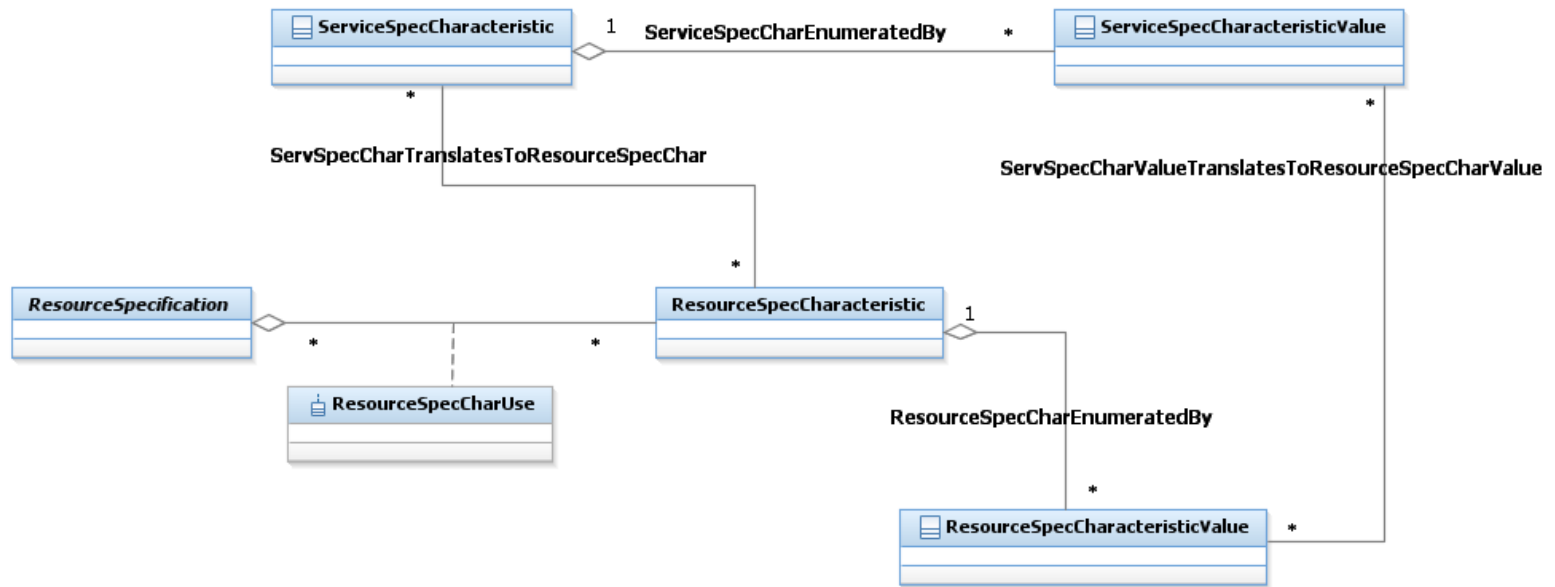


Figure SO.22a – Service and Resource Characteristic Associations

1.1.15. ServiceCharacteristicValue and ServiceSpecCharacteristicValue Classes

The purpose of the ServiceCharacteristicValue and ServiceSpecCharacteristicValue objects is to define how instances of these objects can be populated. Again, it was influenced from the earlier work done on similar Product objects. The ServiceCharacteristicValue and the ServiceSpecCharacteristicValue classes, shown in Figure SO.17, define this concept.

A ServiceSpecCharacteristicValue object is used to define a set of attribute values, each of which can be assigned to a corresponding attribute in a ServiceSpecCharacteristic object. This is represented by the *ServiceSpecCharEnumeratedBy* association. The values of the attributes in the ServiceSpecCharacteristicValue object describe the values of the attributes that a corresponding ServiceCharacteristicValue object can take on.

Note that we have elected to use classes instead of OCL to model these constraints. Therefore classes can be used to create reusable “constraints”, which are in effect what the ServiceSpecCharacteristicValue are.

Figure SO.23 also shows how a ProductCharacteristicValue has been translated to a ServiceCharacteristicValue.

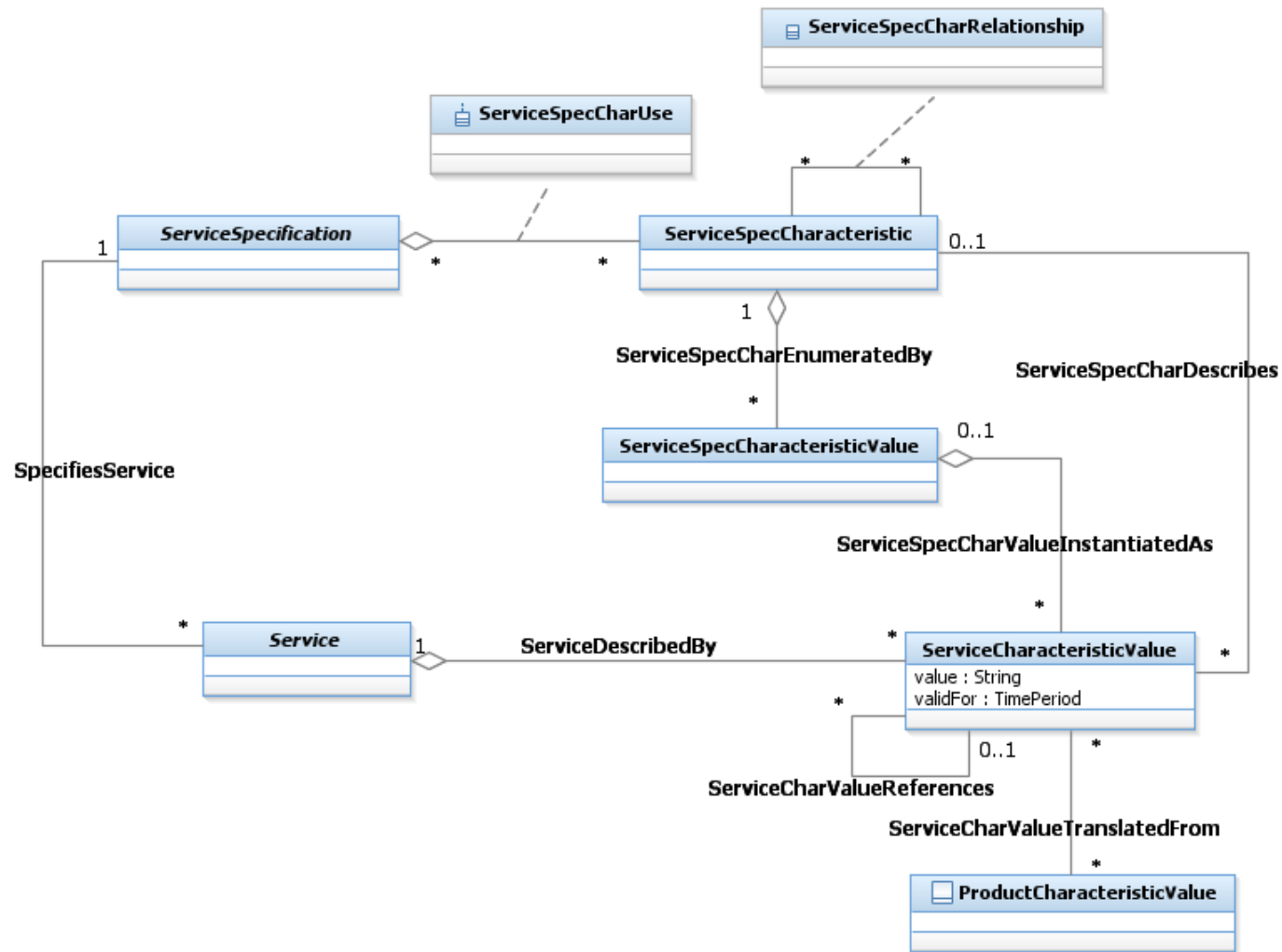


Figure SO.17 - ServiceCharacteristicValue and ServiceSpecCharacteristicValue

For example, consider the definition of a VPNService, as previously shown in Figure SO.09. A set of ServiceSpecCharacteristicValue could, for example, be used to define the types of VPN that are being offered. A corresponding ServiceCharacteristicValue would then represent the salient features of a particular instance of a Service VPN. For example, a Service Provider may be constrained to offer either an MPLS or an Ipsec VPN (as opposed to other types) of VPNs. These would be examples of ServiceSpecCharacteristicValues. Similarly, perhaps the MPLS VPN could only support a defined upper limit of CE devices. This upper limit would be an example of a ServiceCharacteristicValue.

1.1.16. Linking to QoS – ServicePackage and ServiceBundle

The Service QoS guide book defines of the QoS model. The purpose of this model is to satisfy one of the use cases of the Service Domain in general, by describing how QoS mechanisms on a device are configured. Here, “mechanisms” refers to modeling the end-user features of the device.

Conceptually, what makes this a daunting task is that Services involve multiple devices which can have heterogeneous commands and programming models, as is shown in Figure SO.18 below.

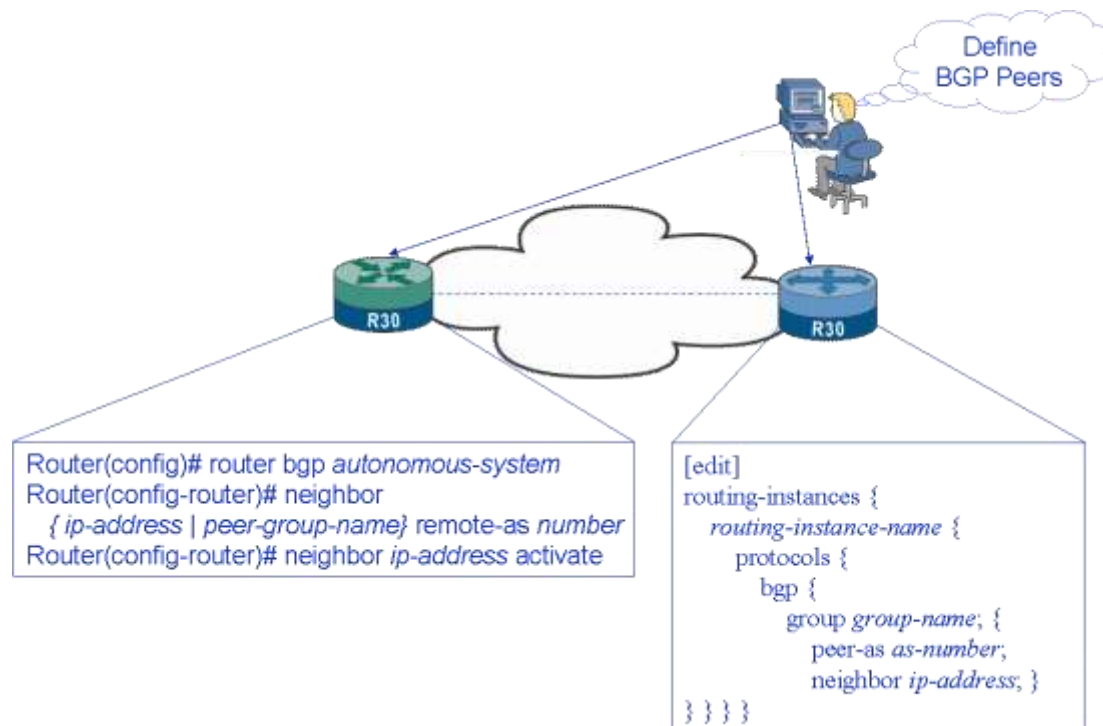


Figure SO.18 - Multiple Command Syntaxes and Programming Models

Current network devices have different programming models using different commands. This makes it very difficult for common tasks to be performed, because in effect the network operator must know different languages. If we also take into account the

fact that commands change per OS version, the analogy grows to a single person having to know multiple dialects of multiple languages. This cannot scale.

Figure SO.18 illustrates the use of different commands and programming models to perform the same high level task. The device on the left has configuration *modes*, each mode defining a set of appropriate commands. The device on the right has no such modes. Furthermore, the syntax used to perform the same task is drastically different.

Note that each network device supports a Service through configuring a DeviceInterface and/or its LogicalDevice. This is worse for two reasons. First, the thought of doing this manually, for thousands of networks, is no longer viable, due to both the complexity of networks, the complexity of services that networks support, and the interaction between services supported on a network. Second, since each Device in general has a different programming model, there is no way to tell in advance which way (or ways) a Device can be programmed.

Clearly, manually configuring each Service that can run on a given device is not a viable process. This Addendum introduces two innovative classes – ServicePackageSpec and ServiceBundleSpec – that serve to link the business requirements of QoS to its implementation. Note that this is done at the *specification* level, to enable this to be templated. Of course, this is yet another instance of the Entity-EntitySpecification pattern, which is used throughout the SID. (This in and of itself is important, because it shows how the existing Service model can be easily extended to bridge new concepts using the same set of standardized mechanisms. This is an indicator of the robustness and inherent extensibility of the SID.)

A ServicePackageSpec defines the concept of bundling a set of different CustomerFacingServiceSpecs to meet the functionality specified by one or more ProductSpecifications. This enables the common characteristics of these CustomerFacingServices to be specified, so that multiple Products can be built from their associated ProductSpecification.

Treating this set of CustomerFacingServiceSpecs as a single object is very important for building complex Services, such as a VPN. This enables a single Product (whether it is a ProductComponent or a ProductBundle, as shown in Figure SO.04, is irrelevant) to be offered to the Customer (or another type of PartyRole), even though in reality the Product consists of a set of different CustomerFacingServices that must work together to provide the functionality that the Customer needs.

The composite pattern is used to make the ServicePackageSpec object extensible. A ServicePackageSpecAtomic object models different ServicePackageSpecs as a set of different instances of individual, independent CustomerFacingServiceSpecs. This is fundamentally different than the ServicePackageSpecComposite object, which is used to model one ServicePackageSpec as the combination of other existing ServicePackageSpecs (as well as providing its own extensions).

The ServicePackageSpec Hierarchy is shown in Figure SO.19 below.

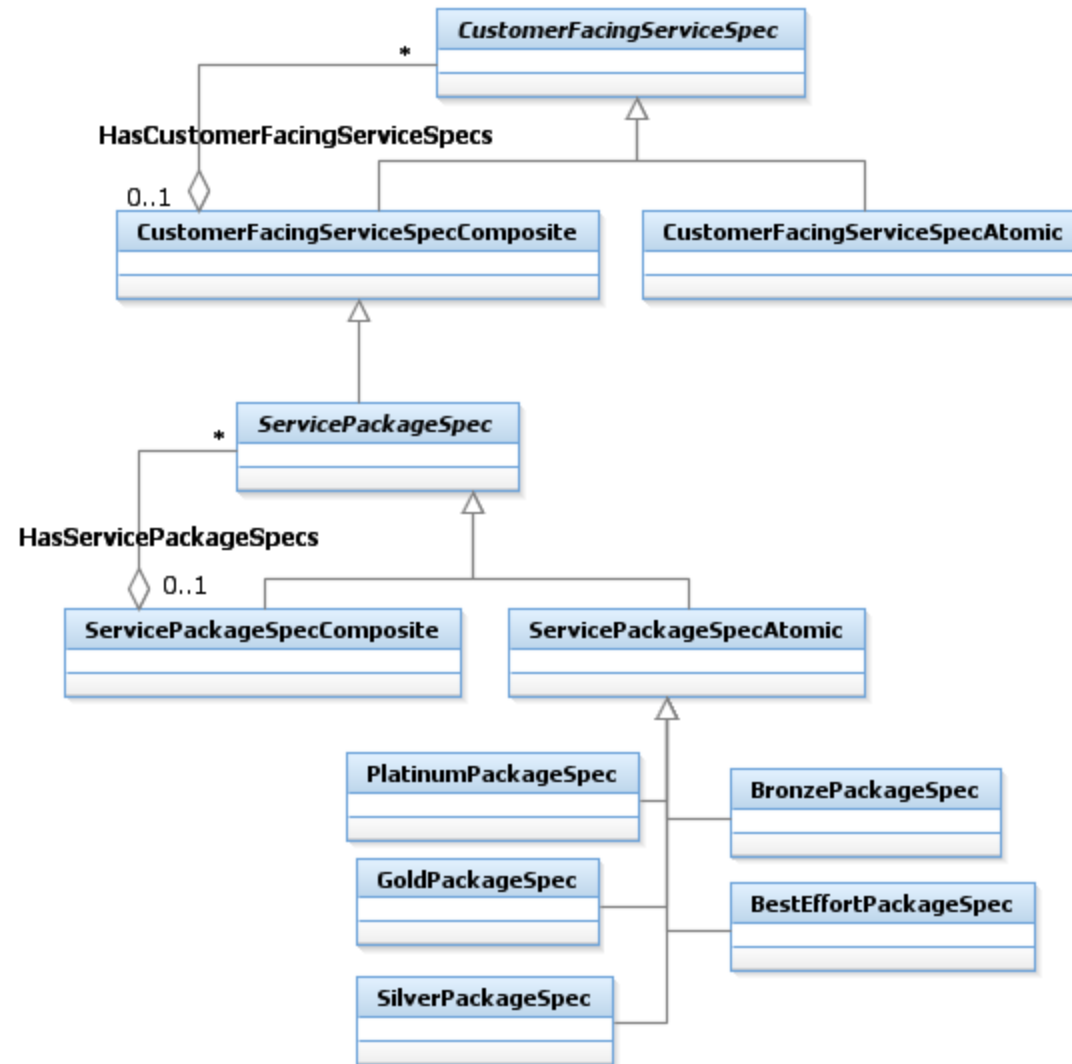


Figure SO.19 - The Concept of a ServicePackage

For example, GoldPackageSpec is an individual packaging of services, and is therefore an instance of the ServicePackageSpecAtomic class. If there was a service offering that combined the services defined by the GoldPackageSpec with those defined by the PlatinumPackageSpec, then that would be an instance of the ServicePackageSpecComposite class.

Figure SO.19 shows five exemplary subclasses of the ServicePackageSpecAtomic class – PlatinumPackageSpec, GoldPackageSpec, SilverPackageSpec, BronzePackageSpec, and BestEffortPackageSpec. Ignoring the latter for the moment, each of these classes represents a collection of CustomerFacingServiceSpecs that work together to collectively offer a set of Services. This enables these Services to be compared as a group to (for example) a ServiceLevelAgreement. Conceptually, this can be represented as shown in Figure SO.20 below:

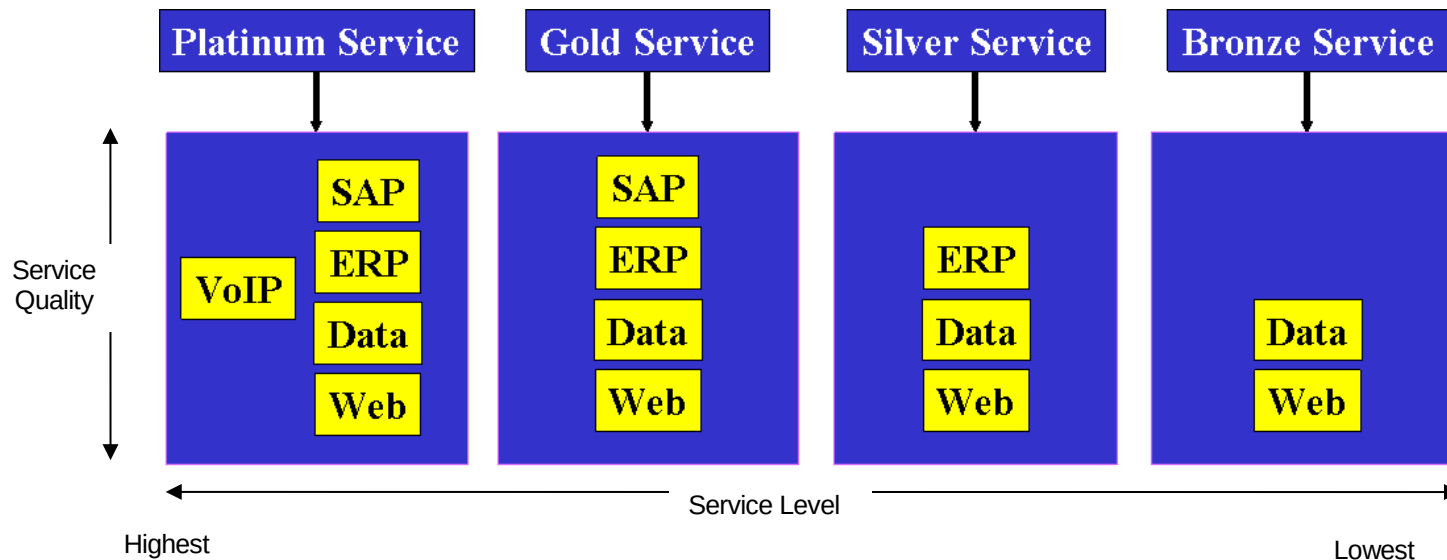


Figure SO.20 - Grouping Together CustomerFacingServiceSpecifications into a ServicePackage

There are two important differences between Platinum, Gold, Silver, and Bronze Service. The first is obvious from Figure SO.20 above – the better the Service Level, the more CustomerFacingServices that a user has access to. The second is more subtle, and involves the *quality* of CustomerFacingService. For example, all four ServicePackages shown in Figure SO.20 above provide Web and Data access. The difference between these applications is that the quality of Web and Data access for

PlatinumService is better from an end-user's point-of-view (i.e., faster downloads, less waiting, etc.) than that for GoldService, and so forth.

The link between a ServicePackageSpec and a ServicePackage is shown in Figure SO.21 below.

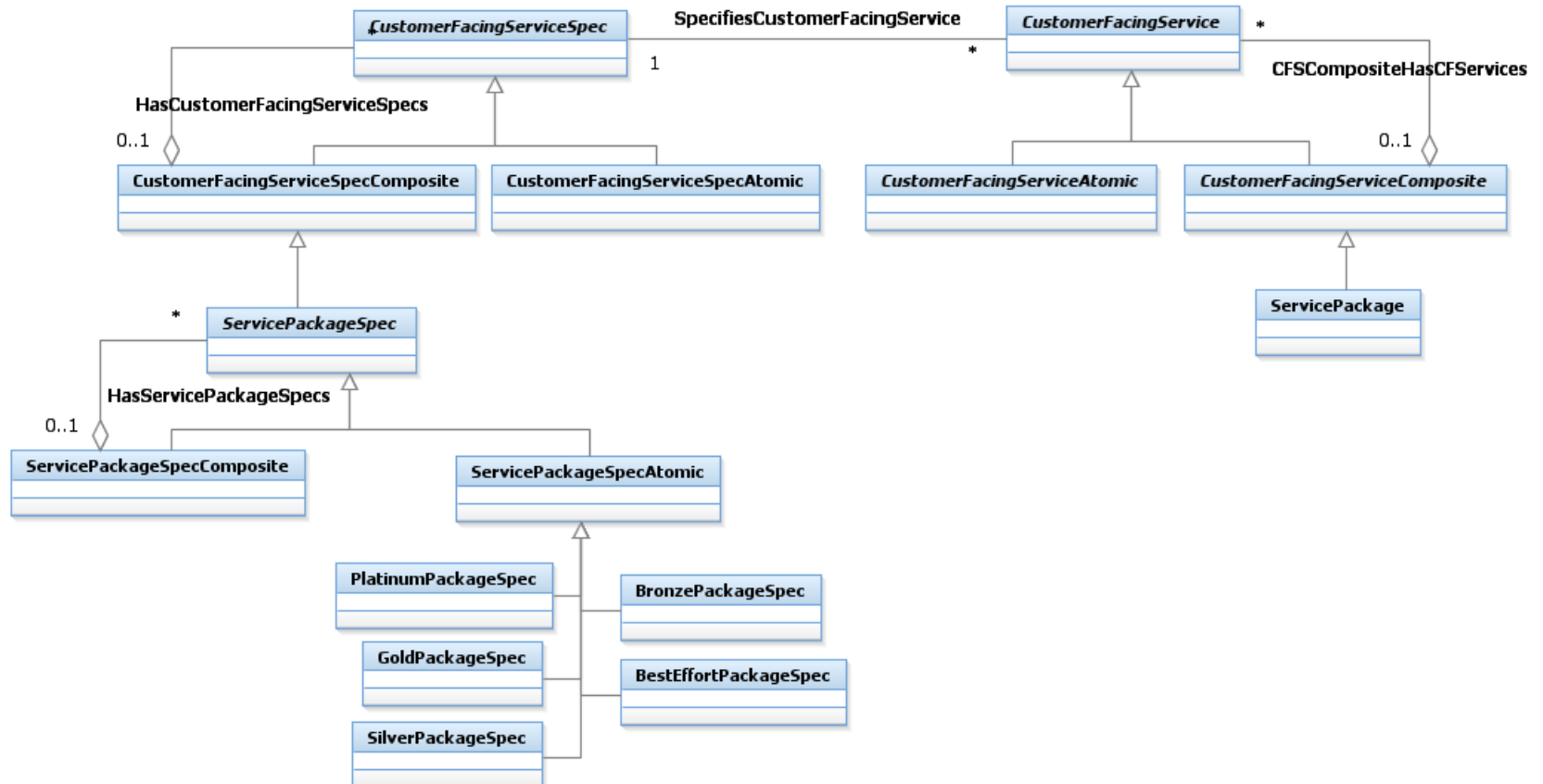


Figure SO.21 - ServicePackageSpecs and ServicePackages

A ServicePackage is derived from an associated ServicePackageSpec. The ServicePackageSpec defines the invariant attributes, methods, relationships, and constraints for all ServicePackage instances that are derived from it. This enables each individual ServicePackage to add its own application-specific changeable characteristics.

Note that there is no specific association used to relate a particular ServicePackage to the ServicePackageSpec that it is derived from. This is because the ServicePackageSpec and ServicePackage both inherit the SpecifiesService association.

Finally, while the composite pattern could be applied to ServicePackage, there is no perceived need to do so. Multiple ServicePackages will simply be related to a ProductBundle, and appear as separate ProductComponents.

Thus, we see that ServicePackage is a way to define the set of CustomerFacingServiceSpecifications that are to be used by a particular ProductSpecification. The next concept to introduce is that of ServiceBundles.

Just as ServicePackageSpecs and ServicePackages define the templates and instances of a set of CustomerFacingServices, ServiceBundleSpecs and ServiceBundles define the templates and instances of a set of ResourceFacingServices. Note the symmetry – this is once again indicative of an extensible, robust information model. The extensibility is, of course, in the use of the ServicePackageSpec and ServiceBundleSpec entities. Once the desired specifications have been defined, it is relatively easy to instantiate those templates (thereby producing ServicePackage and ServiceBundle entities). Furthermore, this enables different instances to be easily differentiated while ensuring that their important characteristics remain constant. This also greatly simplifies their definition and management.

Figure SO.22 shows the relationship between a ServicePackageSpec and a ServiceBundleSpec.

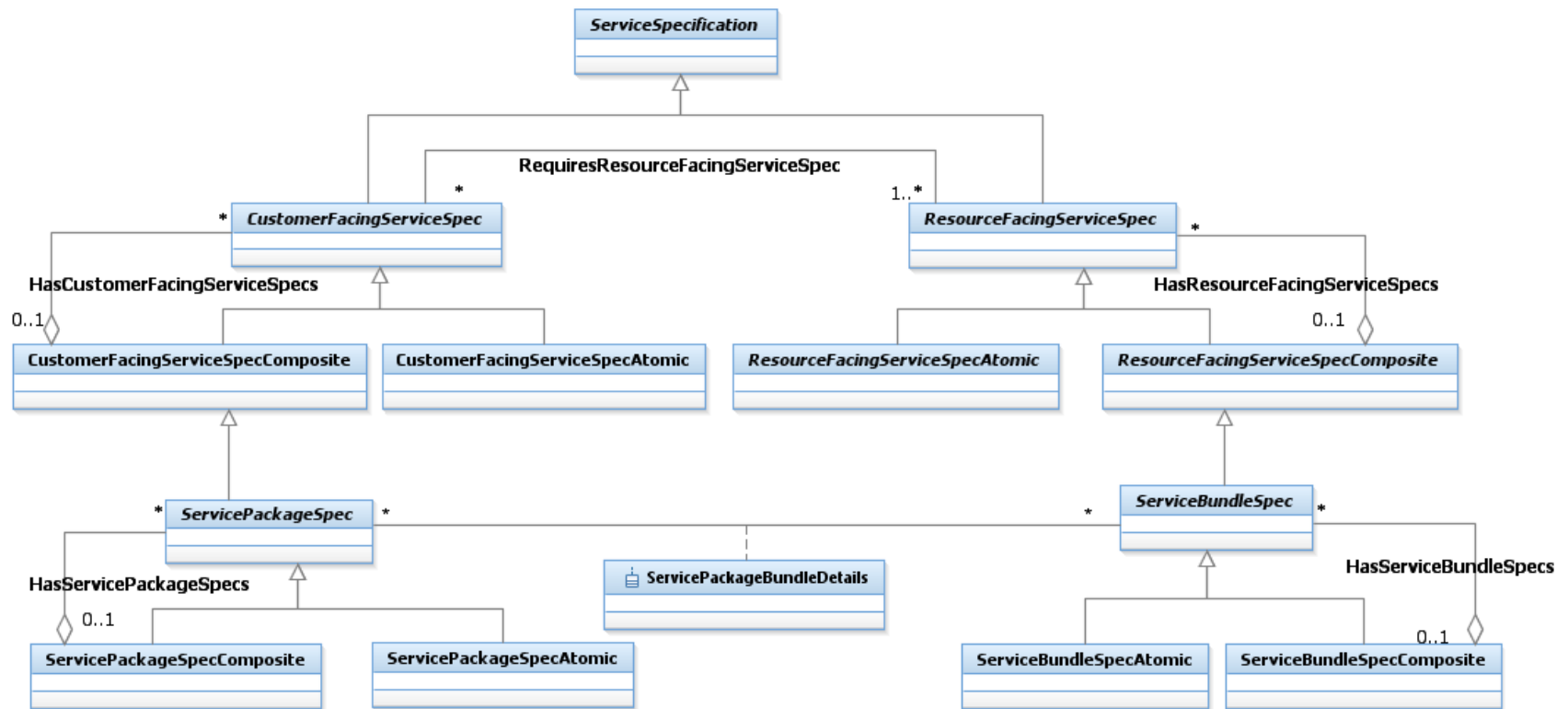


Figure SO.22 - Relationship Between ServiceBundleSpec and ServicePackageSpec

A **ServiceBundleSpecification** is the base class for defining the different classes of bundled **ResourceFacingServiceSpecifications** that a Customer can obtain via a Product. The preferred way to represent a Customer subscription to a Product of this nature is by using the association class **ServicePackageBundleDetails** to define the set of specific **ServiceBundleSpecifications** that are used by a particular **ServicePackage**.

Conceptually, a **ServiceBundle** is thought of as a collection to enable the needs of different sets of **ResourceFacingServices** to be grouped together. The “bundle” conveys the concept of grouped Services that are related. Since these are

ResourceFacingSpecifications, they define reusable templates for implementing the ResourceFacingServices that are required by a particular CustomerFacingService (as represented by a ServicePackage).

Figure SO.23 shows four exemplary subclasses of ServiceBundle for defining four different groups of Class of Service (CoS). The idea is to use each CoSBundle class to represent the Class of Service settings for a specific ServiceBundle. This enables different CoS settings to be associated with the different CustomerFacingServices that are defined by a particular ServicePackage, which is what the Customer obtains via a Product. For example, CoS1 could be used for VoIP, CoS2 could be used for mission-critical applications, CoS3 could be used for all other applications that are not mission-critical but require better than best-effort delivery, and CoS4 could define best effort service delivery.

The combination of ServicePackage and ServiceBundle enables a Service Provider to abstract the notion of different classes of service that are sold to subscribers. For example, the troublesome use case of provisioning a Customer's request to upgrade to a higher-level Service (or downgrade to a lower-level Service) can now be much more easily and efficiently handled. More importantly, this enables a single link to be defined between the low-level QoS model (which controls the mechanisms used in the devices to condition traffic appropriately) and this higher-level, business-oriented model.

Specific ResourceFacingServices are related to a ServiceBundleSpecification as shown in Figure SO.23 below:

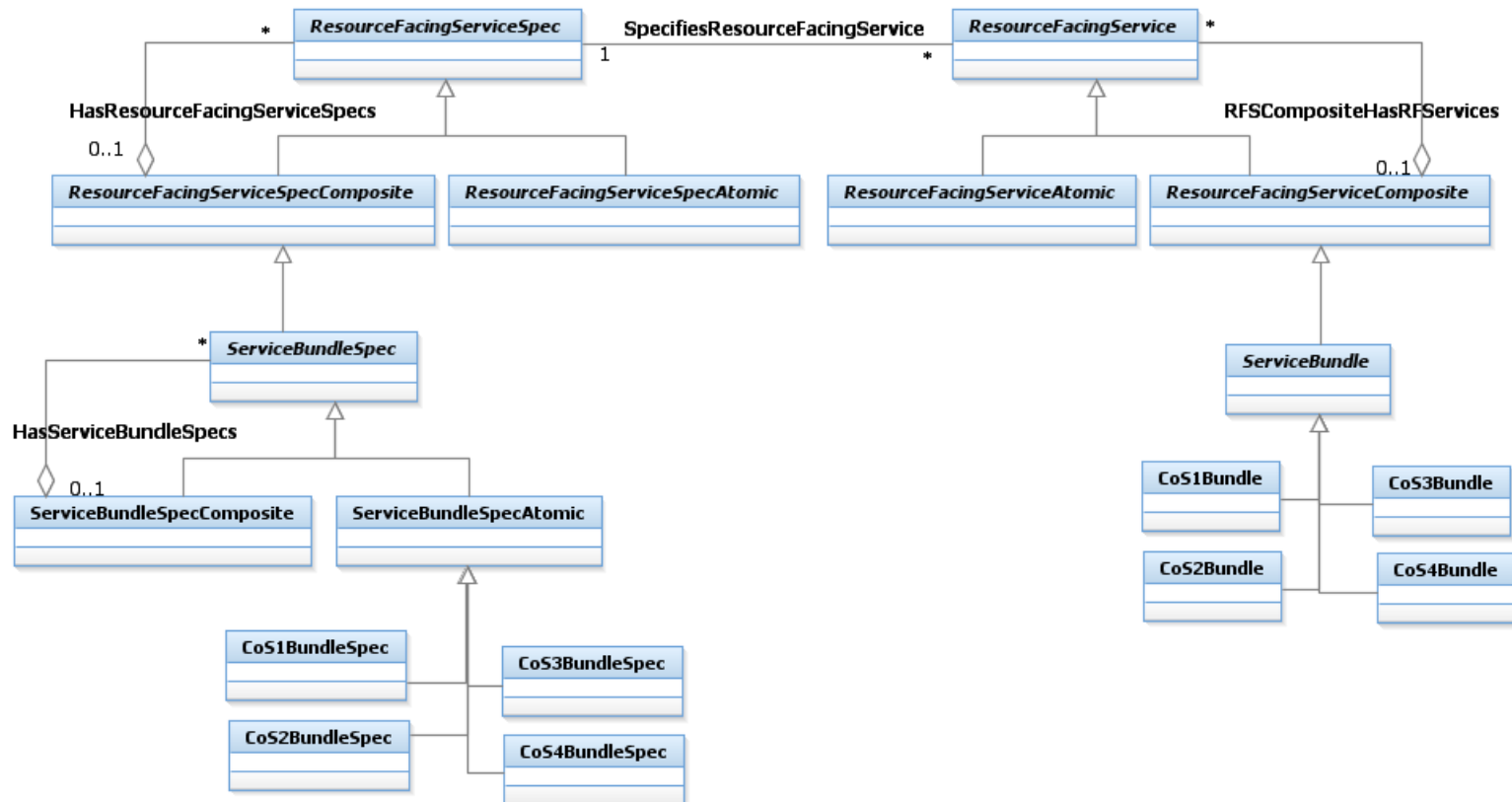


Figure SO.23 - ServiceBundles and ServiceBundleSpecifications

A summary view of the important (not all!) relationships between ServicePackageSpec, ServicePackage, ServiceBundleSpec, and ServiceBundle, and their parent classes (again, not all!) is shown in Figure SO.24 below. This figure was drawn in order to emphasize the symmetry that exists between ServiceSpecifications and Services.

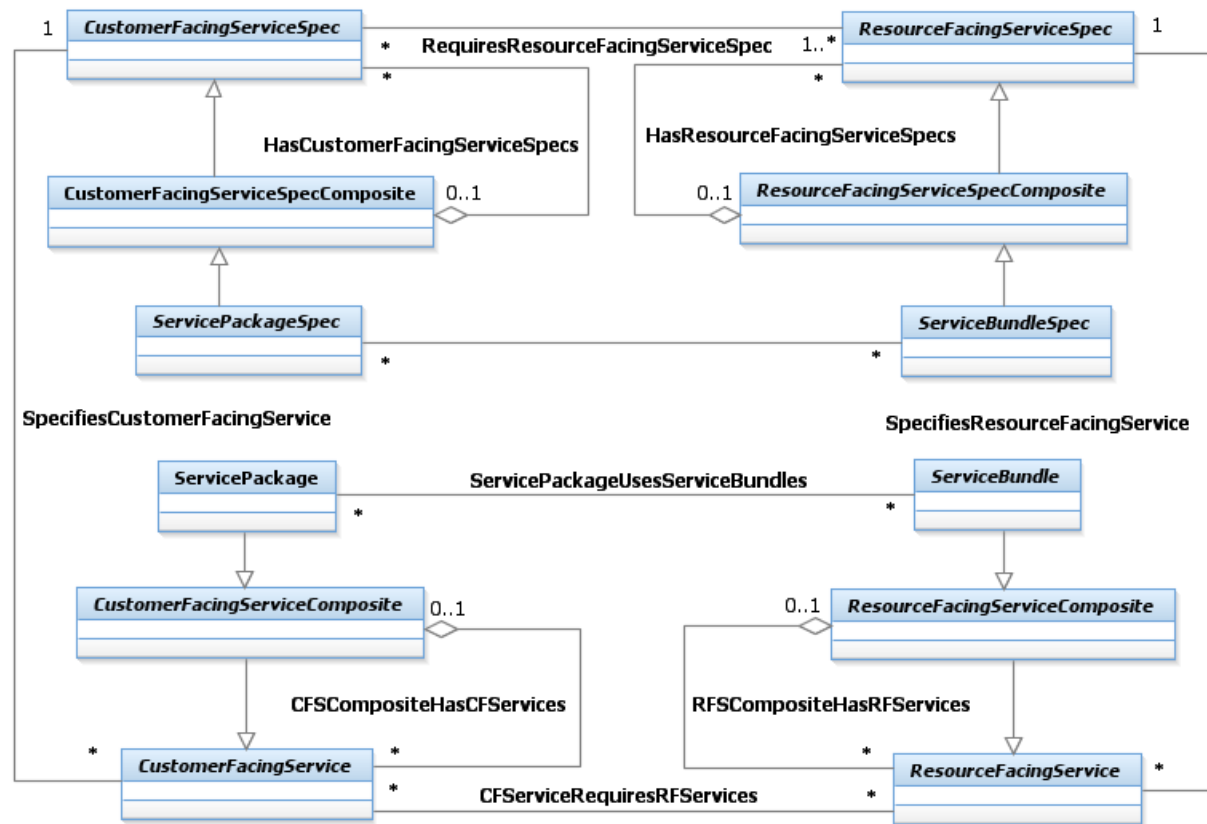


Figure SO.24 -Summary Relationships Between ServicePackage and ServiceBundle

1.1.17. Service Order

ServiceOrders, which are used to provision services, can be initiated in a number of ways, such as by a CustomerOrder to provision Services that realized the Product(s) on the order or to reconfigure/reprovision Services as a result of a problem with them.

Figure SOr.01 shows the basic model and how ServiceOrder and ServiceOrder item inherit from entities in the Business Interaction ABE.

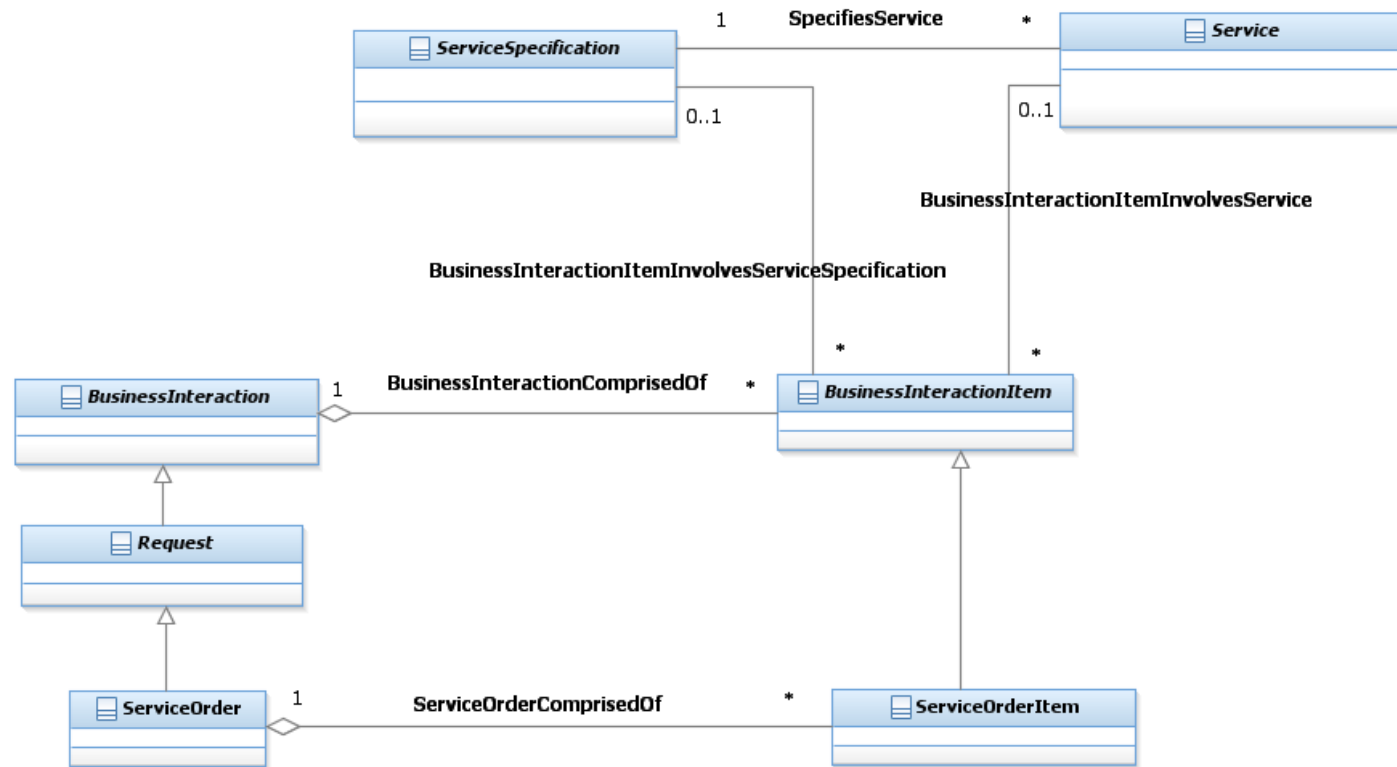


Figure SOr.01 - Service Order Entities

ServiceOrder inherits a number of associations that entities in the Business Interaction ABE have with entities in other ABEs as shown in Figure SOr.02.

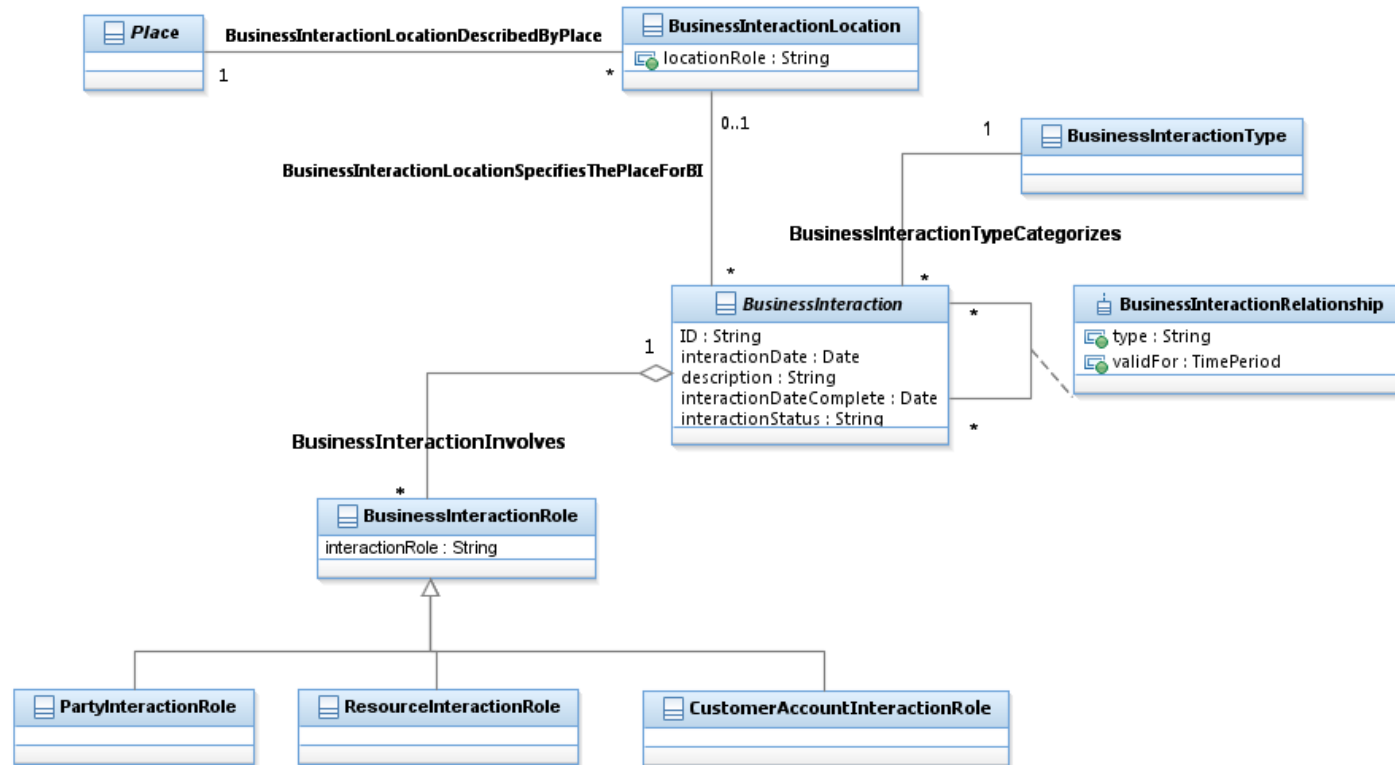


Figure SOr.02 - Relationships Inherited From Business Interaction

ServiceOrderItem also inherits a number of associations that entities in the Business Interaction ABE have with entities in other ABEs as shown in Figure SOr.03.

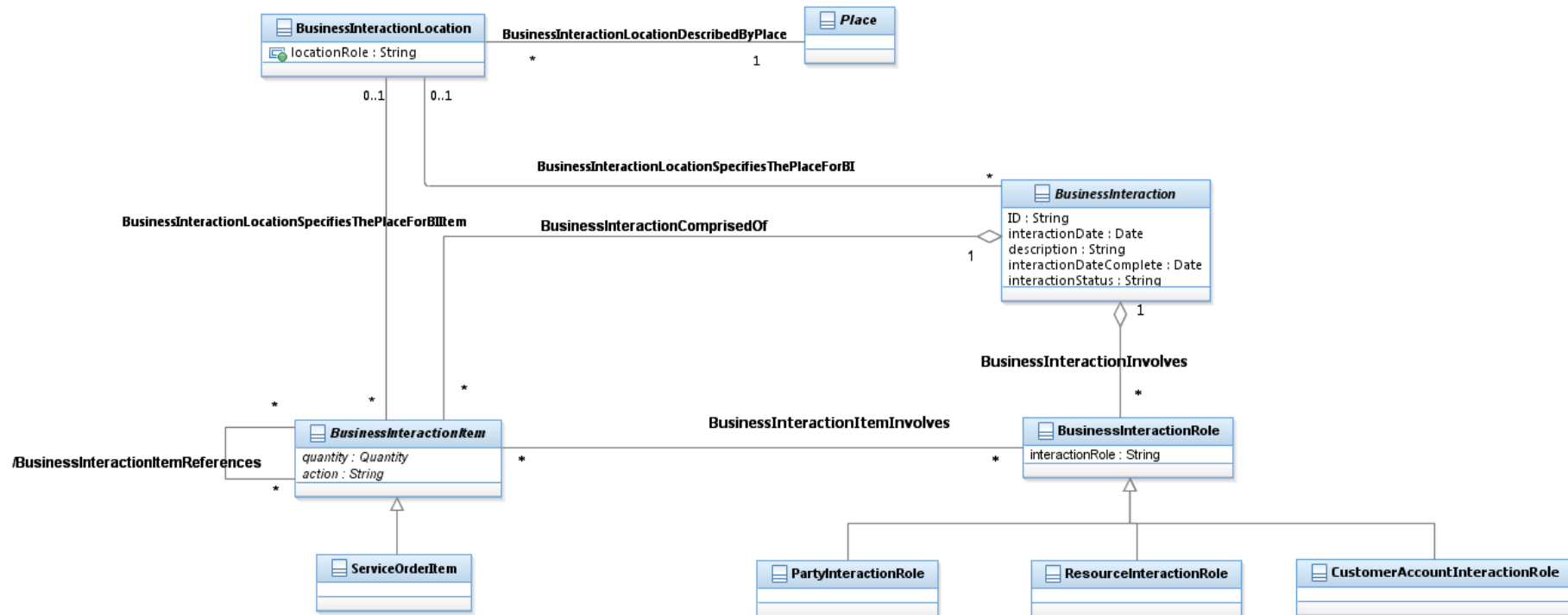


Figure SOr.03 - Relationships Inherited From BI Item

1.2. TIP Service Management ABE

1.2.1. Introduction

The TMF Interface Program introduced in the Multi-Technology Operations System Interface (MTOSI) 2.0, Service Management interfaces based upon extensions to the SID service model. The resulting solution set introduced two new service management activation interfaces, and a service inventory interface:

- The **Service Activation interface (SAI)** between the Customer Relationship Management (CRM) and Service Management and Operations (SM&O) layers allows for the activation of Customer Facing Services (CFS)
 - Interactive Service Activation
 - *Best-effort Transaction interface* Service requests entailing multi-task transaction execution. In this type of interface, the entire transaction execution is completed despite single task failures.
 - *Atomic Transaction interface* This interface has the same operations as the previous one; however, the failure of any single task will incur the termination of the transaction execution and will invoke a roll-back of the preceding task.
 - *Service Order Management interface* Add-on interface to both the Best-effort Transaction or Atomic Transaction interfaces. Provides full exposure and control of the service order associated with the service request over the interface. It depends on the functions provided by the previous interfaces.
- The **Service Component Activation interface (SCAI)** is an internal SM&O layer interface. It allows for the activation of Service Components, also known as Resource Facing Services (RFSs), given specific service related information defined in a Service Template. The SCAI, contrary to the SAI, has no knowledge of the product domain.
- The **Service Inventory** interface supports the retrieval of service related entities from a Service Inventory repository. The TIP Service Management ABE provides the extensions made to the SID model in support of the new Service Management interfaces introduced. The basic *service* and *service specification* concepts as defined in this addendum remain unchanged, and are merely enhanced to support the TIP Service Management interfaces.

1.2.2. Service and Sap Specifications

The abstract class **ServiceSpecification** has been extended to support two new concepts introduced in the TIP Service Management interfaces:

- **ServiceDefinition** used during service design time to define the characteristics of a service, their associated data types, and default values. These characteristics, known as *ServiceSpecCharacteristics* (SSC), may be *globally set*, which implies they can be defined in a service template, or otherwise only passed in real-time over an activation interface
- **ServiceTemplate** defines specific *ServiceSpecCharacteristicsValues* (SSCVs) for the SSCs that were defined in the Service Definition as being globally settable. The Service Template is checked against its associated Service Definition by verifying the presence of the SSCs and the validity of the corresponding assigned SSCVs. Only Service Templates that contain SSCs/SSCVs that are in conformance with their specification in the Service Definition are considered valid.

Both Service Activation interfaces make use of the Service Definition and Service Template concepts in the activation of *CustomerFacingServices* and *ResourceFacingServices*.

The *ServiceDefinition* and *ServiceTemplate* further define the Service Specification through their respective associations, with the versions being identified through the *ServiceSpecVersion*. The association class *ServiceSpecRelationship* allows for Service Specifications to contain other Service Specifications, which thus holds true for Service Definitions and Service Templates. For example, there may be a type of Service Definition for Ethernet Services, and contained within that type are E-LINE, E-LAN, and E-TREE Service Definitions.

The *ServiceAccessPoint* (SAP) represents a unique (logical and/or physical) resource where the Service can be accessed. The *Sap Specification* defines a set of attributes and associated values that may be applied to one or more SAPs. The relationship between the *Service Specification* and the *Sap Specification* illustrates that for every Service Specification, there may be associated multiple *Sap Specifications*. For example, given an Ethernet E-LINE Service Specification, there may be multiple *Sap Specifications* that correspond to various Bandwidth Profile Specifications that can be applied to the Sap – one for GOLD, one for SILVER, and one for BRONZE. Likewise, there may be a single *Sap Specification*, or in this case Bandwidth Profile Specification, that is used for multiple Ethernet Service Specifications – E-LINE and E-LAN. A Bandwidth Profile Specification defines Service Level Agreement guarantees and may be applied per UNI, per EVC, or per CoS Id. They contain frame delivery per Service Level Objectives (Committed Information Rate, Committed Burst Size), and amount of excess frame delivery allowed (Excess Information Rate, Excess Burst Size).

A *ServiceCatalog* represents a grouping of Service Specifications and SSCs. For example a catalog could group all internet related Service Specifications. The following figure illustrates the *Service Definition*, and *Service Template* concepts, and the relationship between the Service Catalog and the Service Specification.

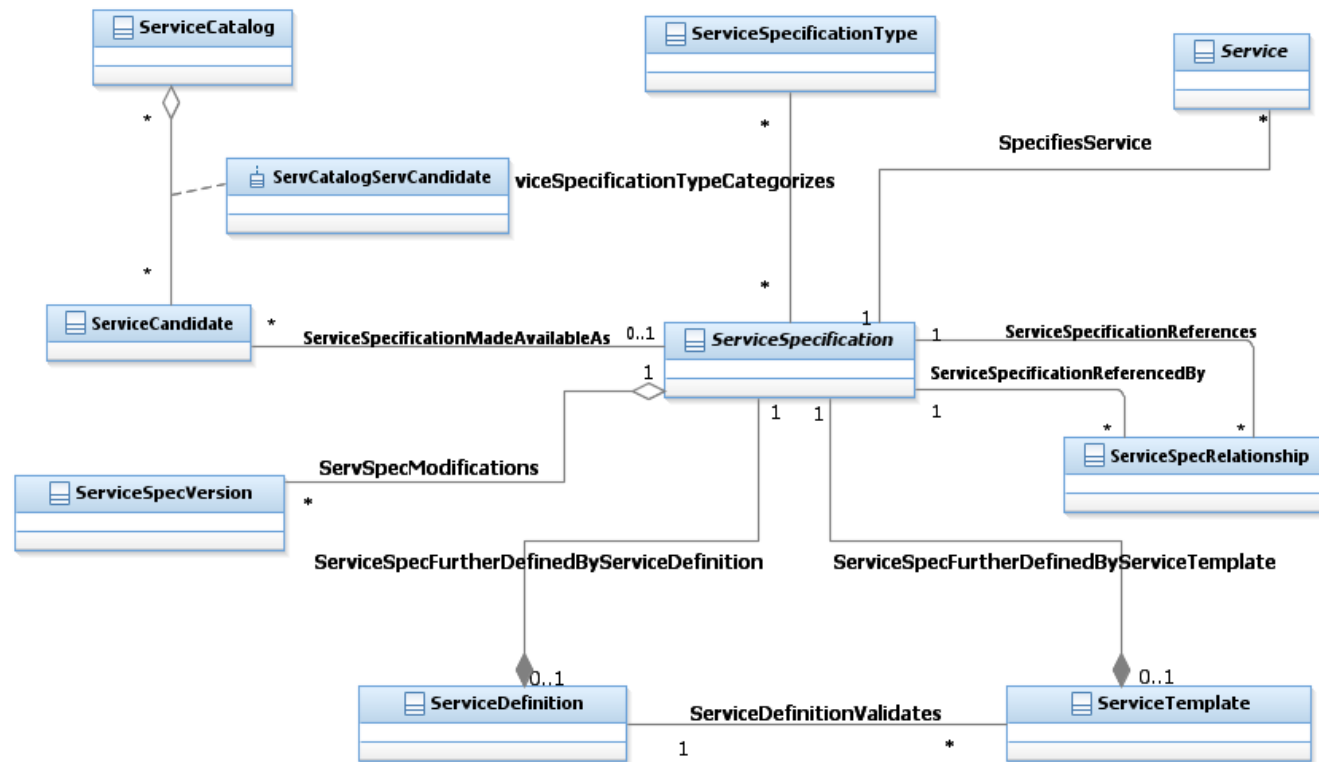


Figure SO.25 - TIP Service Management Service Definition and Service Template

1.2.3. Service Related Characteristics

The *ServiceSpecCharacteristic* and the associated *ServiceSpecCharacteristicValues* pairs are used in both the Service Definition and the Service Template to describe, or characterize, a Service. The *CharacteristicSpecification* and the *CharacteristicSpecValues* pairs are used to characterize the Service Access Point through the association of the *SapSpecification* with the *EntitySpecCharUse*. These provide the actual characteristic / values pairs that would be found, for example, in a Bandwidth Profile Specification for an Ethernet Service User Network Interface (UNI) or Ethernet Virtual Connection (EVC) endpoint. The Bandwidth Profile Specification would contain the following characteristics: committed

information rate (CIR), committed burst size (CBS), excess information rate (EIR), excess burst size (size), and possibly color marking (CM) and color forwarding (CF) indicators. By defining this specification, including the associated characteristic values, as an independent entity, the specification could be associated to a Sap or an EVC.

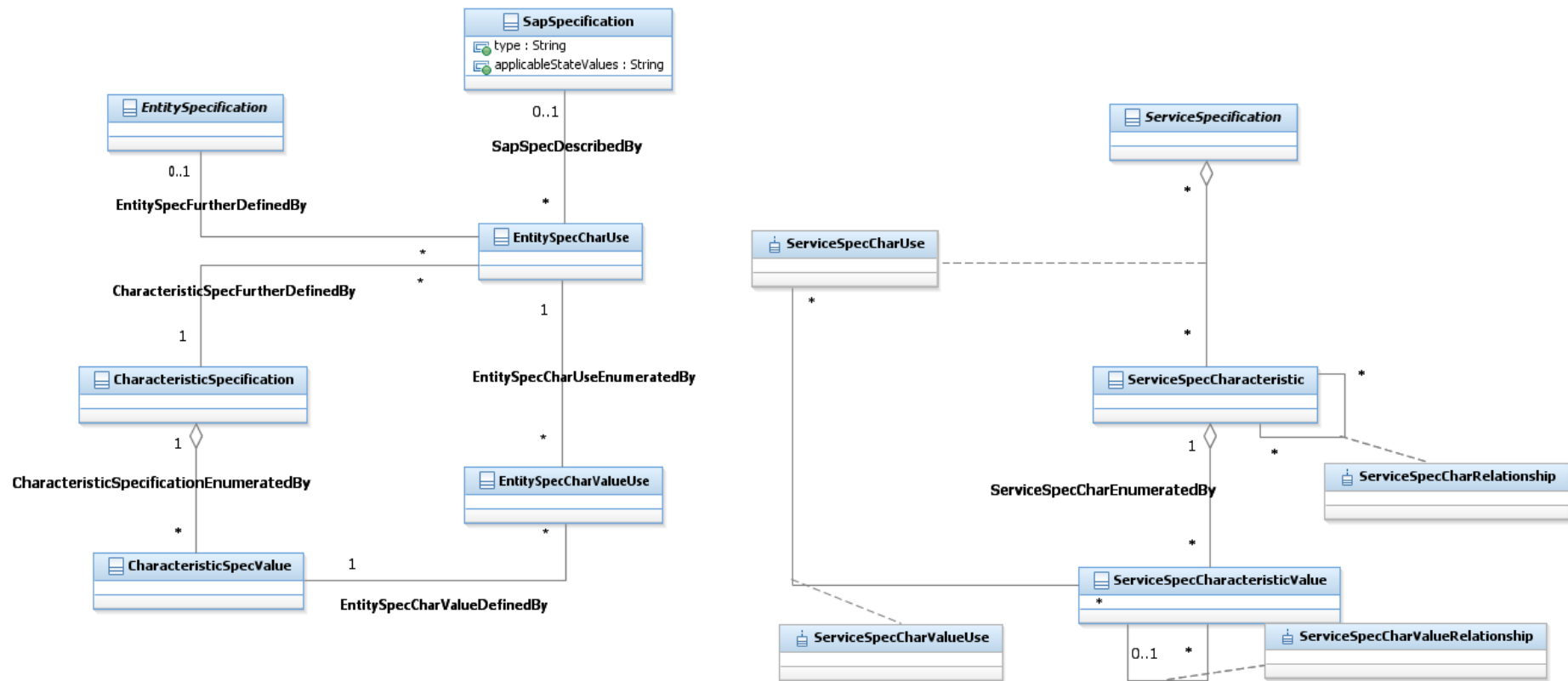


Figure SO.26 - TIP Service Management – Service Characteristics

1.2.4. Service Activation

The service configuration and activation process involves the instantiation of both Customer Facing Services, and associated Resource Facing Services. The CFS and RFS are constructed based upon their respective Service Definitions, ServiceTemplates, and real-time *ServiceCharacteristicValues* passed over the activation interface. The interface context will determine whether the Service Template being referenced is for a CFS (SAI) or an RFS (SCAI).

The CFS and RFS are accessed via one or more *ServiceAccessPoints* (SAPs) that are characterized by *SapSpecifications*. There is a relationship between the SAPs and the underlying Resources serving as access points. The following diagram illustrates the classes and relationships as required for Service Activation:

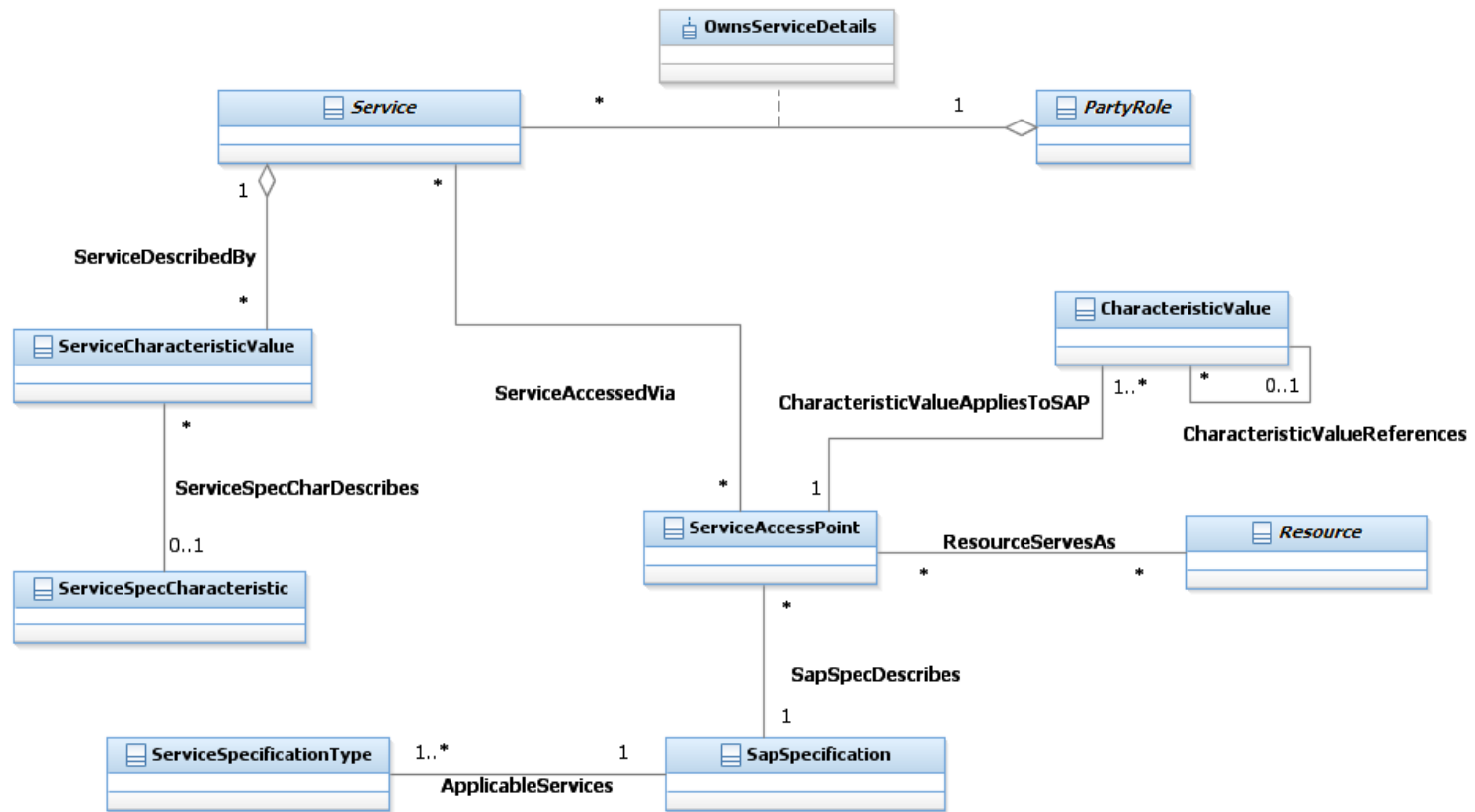


Figure SO.27 - TIP Service Management – Service Activation

A number of the Service Management entities are extended via their association with the CommonServiceInfo entity and via inheritance with the CommonObjectInfo entity as shown in Figure SO.33a.

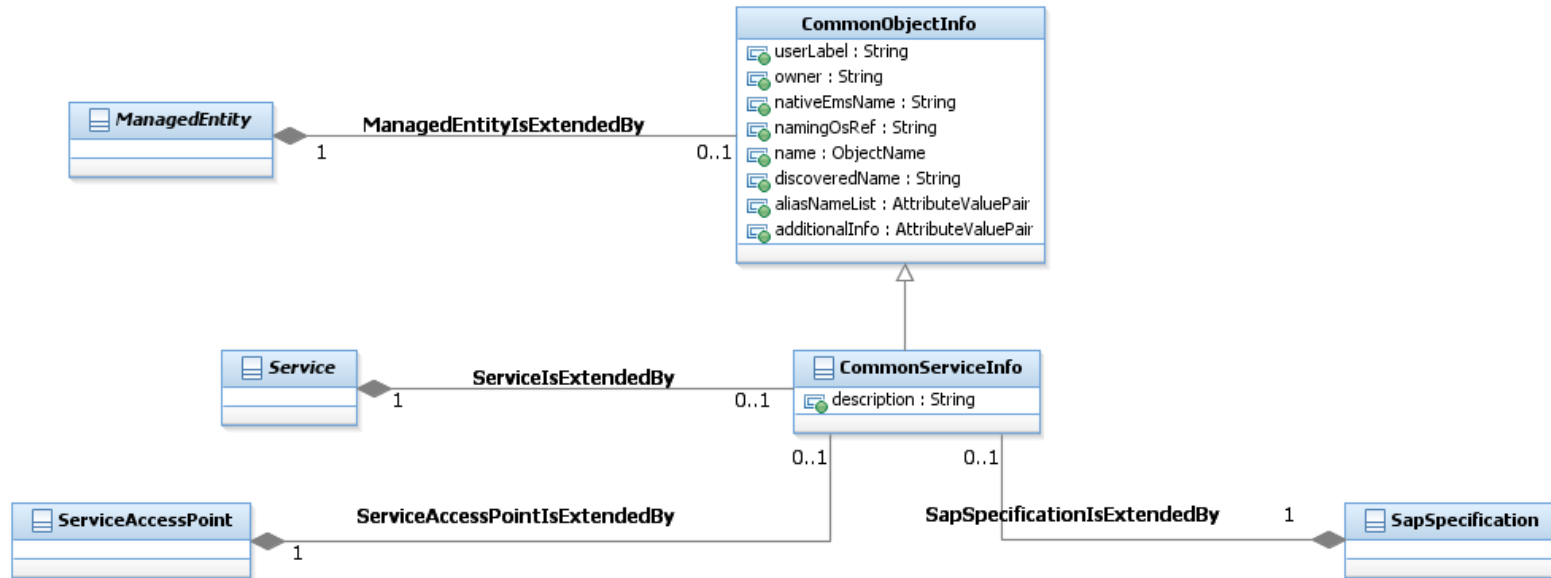


Figure SO.33a - TIP Service Management - CommonServiceInfo

1.2.5. Ethernet Services Example

The following example is based upon multiple service providers involved in the delivery of wholesale and retail services. The Network Service Provider (NSP) offers triple-play IP services to his residential customers, represented by an IP CFS. This CFS is instantiated over the **Service Activation Interface (SAI)**. The NSP Service Management OSS decomposes the CFS into different resource facing service components that require management, one of them being the Ethernet Connectivity RFS (which may also be split into Ethernet access, and Ethernet Aggregation and Backhaul).

The NSP purchases both Ethernet Access and Aggregation & Backhaul Products from the Broadband Ethernet Access (BEA) and Broadband Aggregation & Backhaul (BAB) providers respectively; these are represented in the BEA/BAB domains as “products”. They are instantiated as Ethernet Access Line CFS and Ethernet Backhaul CFS over the BEA/BAB **Service**

Activation Interfaces in their respective OSS/BSS domains. The BEA and BAB Service Management OSS decompose these CFS's into resource facing services that get instantiated over **Service Component Activation interfaces**. As a result, the underlying resources to support the component RFS are provisioned. The BEA and the BAB notify the NSP that their respective services are available and operational.

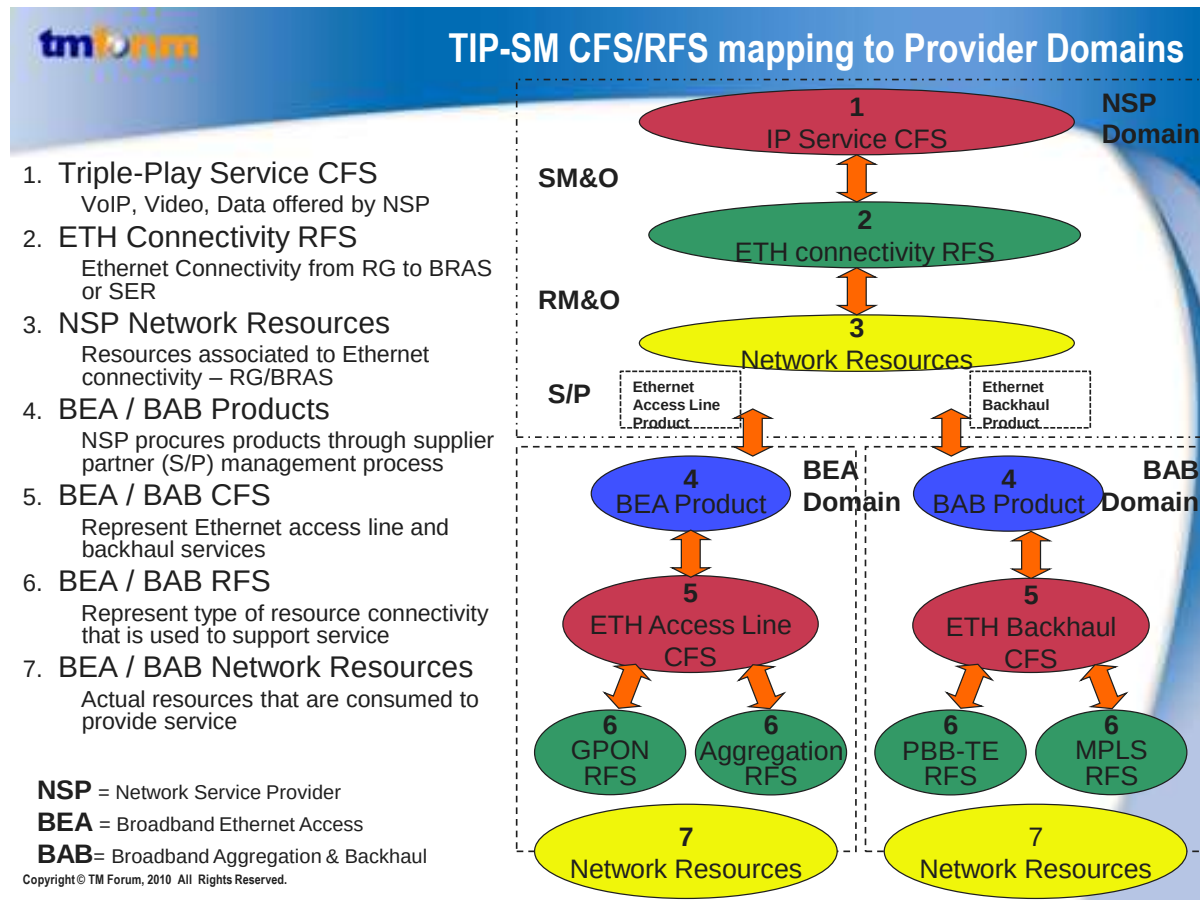


Figure SO.28 - Example of Ethernet Service CFS/RFS

1.3. Interaction With Other SID Business Entities

1.3.1. Introduction

This part of this Addendum will briefly describe the other SID business entities that have been identified that can interact with classes and relationships in the Service Domain. The principal types of entities that can interact with a Service are:

- Policy
- BusinessInteraction
- PartyRole
- Product
- Resource
- Location

1.3.2. Policy

Policy is used to define the semantics of how a Service is managed and controlled. Some of the tasks that Policy governs include the following:

- define what Services are needed for a particular task
- define how to manage, provision, activate, and monitor a set of Services
- define when a set of Services need to be reconfigured, and any additional semantics (e.g., order of changes)
- define which PartyRoles can manage a particular Service
- define which set of Services a particular PartyRole is authorized to use
- define where a Service can be installed or used

As can be seen, the interactions between Policy and Service are varied and complex. This will be one of the areas that will be explored in detail in In a later phase of the SID. The Policy model is defined in [Add 1Pol].

1.3.3. BusinessInteraction

A BusinessInteraction describes how different entities interact with each other. Business interactions take the form of requests, responses, notifications, and agreements. This set of relationships will be explored in detail in In a later phase of the SID after more of the Policy interactions for Service and LogicalResource have been defined.

1.3.4. PartyRole

A PartyRole defines the function that a Party takes on. PartyRoles can represent the ability to manage, configure, use, and perform other types of interactions with Services. A simple starting model is shown in Figure SO.29 below.

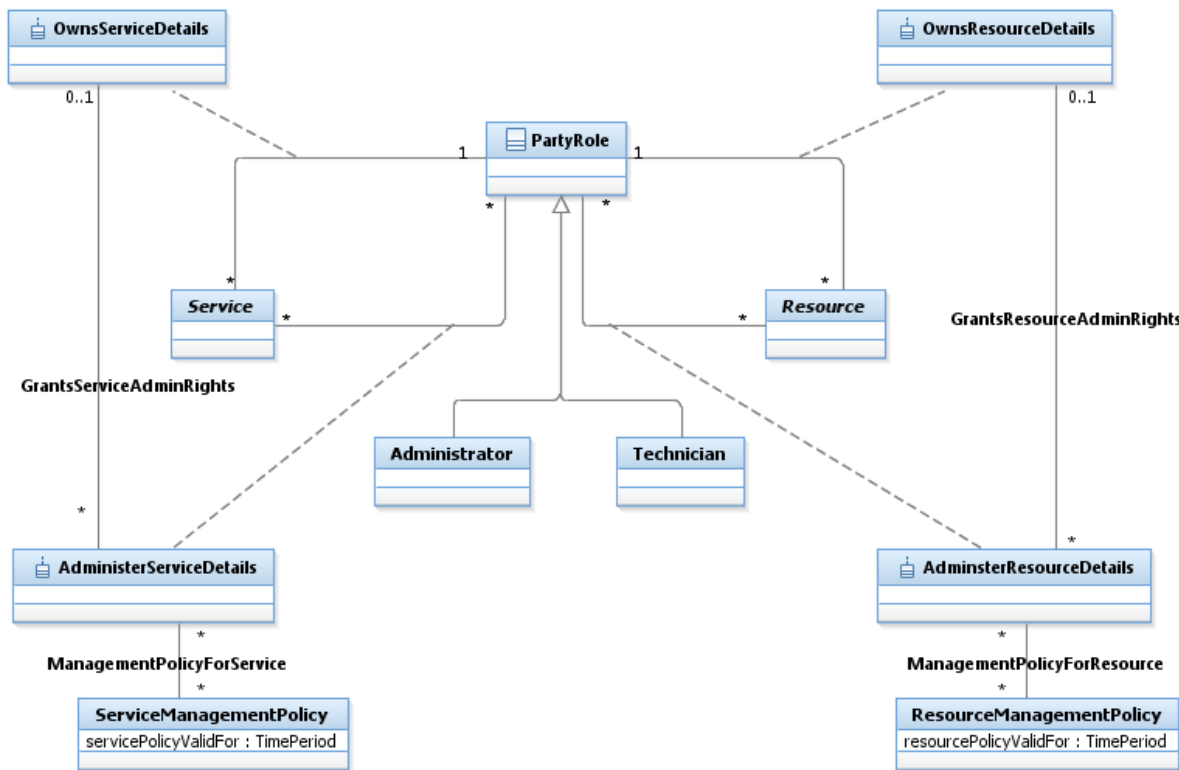


Figure SO.29 - Interaction Between PartyRole and Service

One can see that the Service and Resource management methods are symmetrical. Therefore, this section will only discuss the Service ownership and administration classes.

The *OwnsServiceDetails* association class defines a Service (CustomerFacing or ResourceFacing) that a particular PartyRole owns. Note that this is a very different relationship than the *AdministersServiceDetails* association class. The *OwnsServiceDetails* association class defines the PartyRole that is the owner of the Service. Here, “owner” is the person or group that is responsible (e.g., organizational, financial, and so forth) for the Service. Note that the cardinality of the *OwnsServiceDetails* association class on the PartyRole side is 1 – this means that a Service MUST have at least one PartyRole to manage it. Furthermore, the cardinality of this association on the Service side is *, which ensures that a PartyRole doesn’t have to own a Service.

The *AdministersServiceDetails* association class defines a Service (CustomerFacing or ResourceFacing) that a particular Party, playing a PartyRole, administers. From a business perspective, the Owner has to either appoint or give permission to the Administrator to administer the Service that is owned. This is done using the *GrantsServiceAdminRights* association class.

The *GrantsServiceAdminRights* association defines which PartyRoles are allowed to administer a particular owned Service. Different PartyRoles will in general have different administration rights for the same Service. For example, different types of Administrators will in general each have the ability to make more changes than different types of Technicians.

Policies can be defined for controlling the specific rights of administering a Service. This is the function of the *ServiceManagementPolicy* class. Only a simple single attribute is shown.

The *ServiceManagementPolicy* class defines the particular policies that are used to define how different aspects of the Service are managed and maintained. For example, assume that a Service is made up of several sub-Services. One of these sub-Services might require multiple PartyRoles to approve changes to its configuration, whereas another sub-Service might not require any approvals to change its configuration. Since Policy also uses the composite pattern, if a set of Policies are needed, these can be aggregated into a single Policy).

Note the power of this pattern – the *ManagementPolicyForService* association, along with its *ServiceManagementPolicy* class, provide a generic template for defining which set of PartyRoles can perform which type of management function for which type of Service. Figure SO.29 shows that Services and Resources can be managed using the same pattern, even though the contents of the classes will be different.

There is a lot more to the model, that may be developed further.

1.3.5. Product

For now, the basic relationship between Product and Service (as defined in Figure SO.03) are deemed sufficient. An additional relationship between *ProductSpecification* and *ServiceSpecification* is shown in Figure SO.36. **Error! Reference source not found..**

These relationships will be continuously re-evaluated as the above areas are more fully defined in In a later phase of the SID.

1.3.6. Interaction with Resources

Various diagrams throughout this Addendum have described different types of interactions between the Service and Resource Domains. Additional detail will be provided in the QoS guide book.

1.3.7. Place

In the SID, different types of Places have different types of semantics associated with them. For example, a Wireless Access Point has an associated coverage area. Only applications within that coverage area can use the services associated with the Wireless Access Point. As another example, subscribers will access different POPs (Points of Presence, the Service Provider term for an access network) as they travel nationally and internationally. Services may vary, and additional peering relationships may come into effect. A simple starting model relating Service to Place is shown in Figure SO.30 below.

Note that the ServicePlaceDetails association is implemented as an association class. Its purpose is to represent the semantics of this particular association, such as which specific Services are associated with this Place. Clearly, policies can be used to more fully define the semantics of a particular Place.

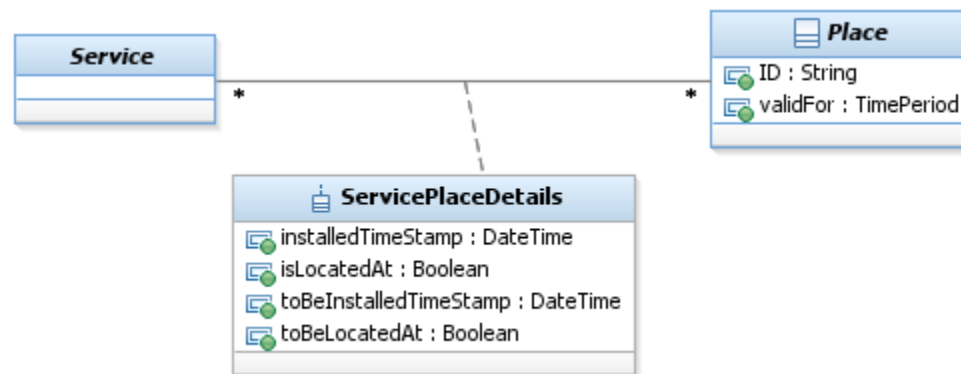


Figure SO.30 - Interaction between Service and Place

1.3.8. Service Level Specification Overview

It should be noted that this section is still a work in progress, and is subject to change.

A BusinessParticipant, such as a Customer (or other type of PartyRole), often expects a certain level of Service to be associated with a Product or Service procured from another BusinessParticipant, such as a Service Provider or some other Enterprise. Such expectations are defined by a Service Level Specification, which is associated to a Product, Service or their respective Specifications. Sometimes referred to as a Service Level Agreement Template, a Service Level Specification represents a pre-defined or negotiated set of Service Level Objectives. In addition, certain consequences are associated with not meeting the Service Level Objectives. Service Level Agreements are expressed in terms of Service Level Specifications.

This figure shows a simple, un-attributed view of a Service Level Specification. Note: the two subclasses of ServiceLevelSpecification are shown merely to emphasize the two different types of specifications.

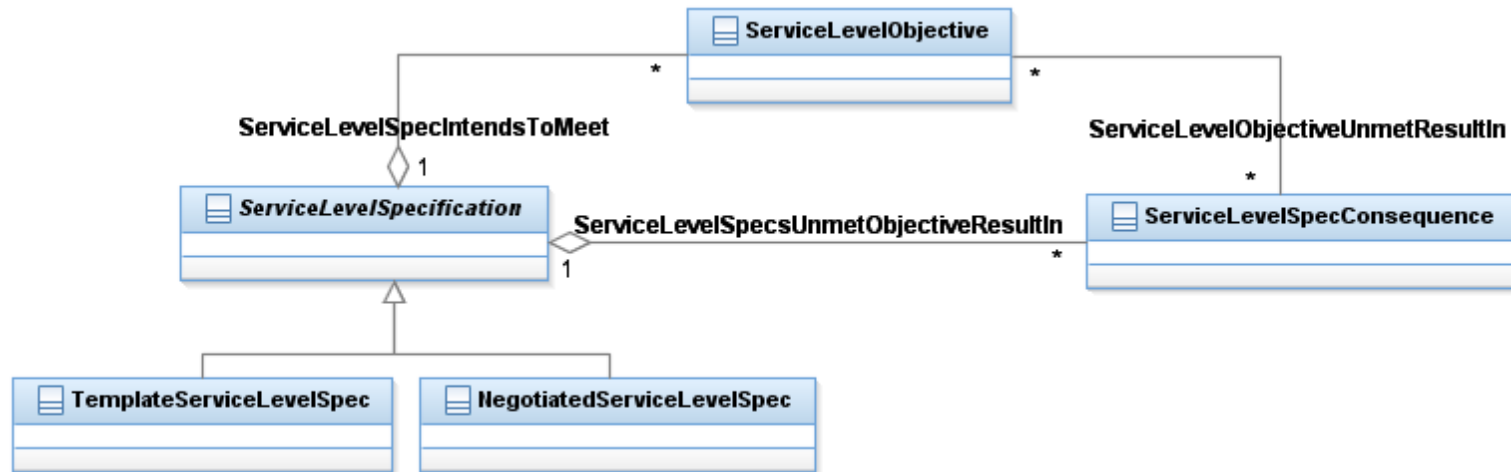


Figure SO.31 - ServiceLevelSpecification Overview

Examples of products and services, along with two different associated SLAs, are depicted in the diagram below. The diagram is a modified extract from the Service Level Agreement Handbook (GB917).

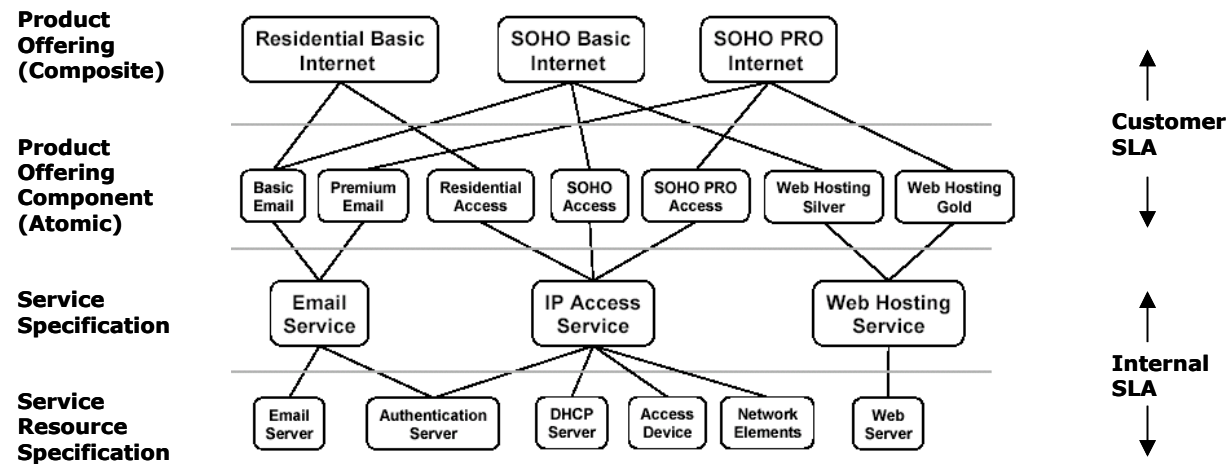


Figure SO.32 - Products, Services, and ServiceLevelAgreements

(Note: the above uses terminology from GB917. An Internal SLA is also known as a Service Provider SLA.)

Customer Service Level Agreements are expressed in terms that a Customer can understand. These Service Level Agreements are defined in terms of Service Level Specifications related to ProductOfferings and ProductSpecifications. ProductSpecifications are used in order to define a set of characteristics that the set of Service Level Specification will cover.

Internal Service Level Agreements are usually defined in terms of resource-facing Service Level Specifications, which are typically too technical to be of interest to a customer. Supplier/partner Service Level Agreements can be expressed in terms of either customer-facing or resource-facing Service Level Specifications. Figure SO.33 below shows a Service Level Specification's relationships to Products, Services, and their associated Specifications.

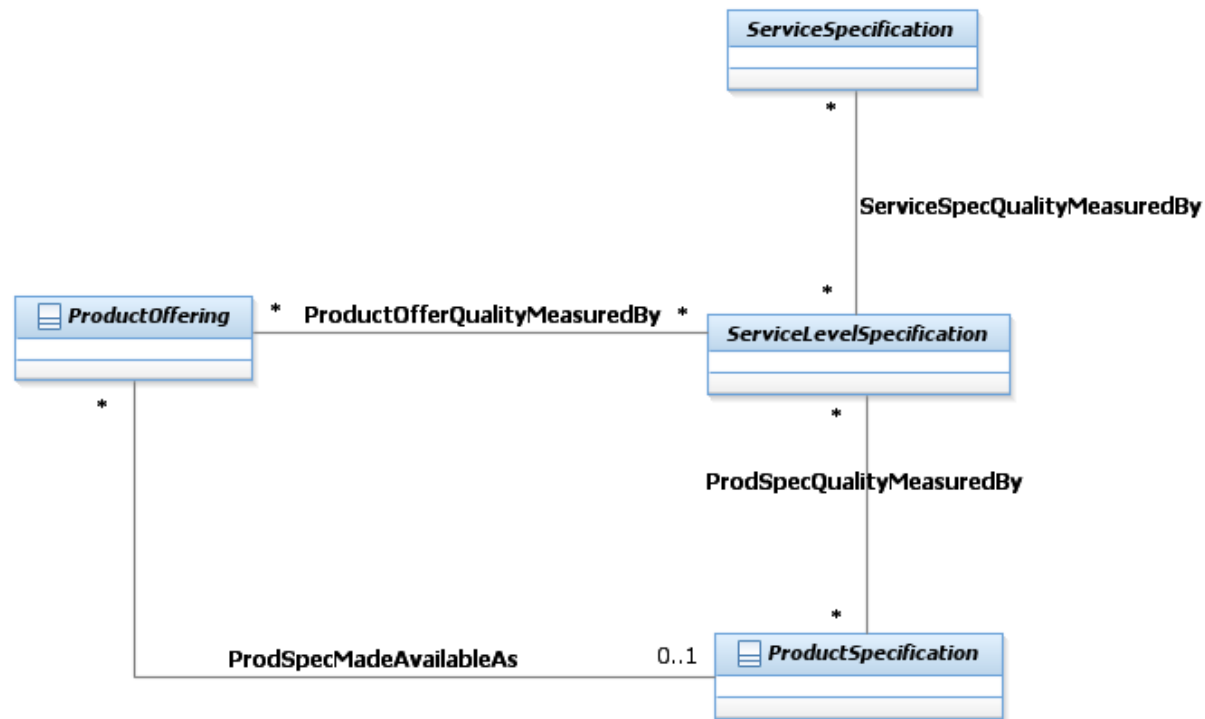


Figure SO.33 - Associating ServiceLevelSpecifications With Products and Resources

Service level objectives are defined in terms of parameters and metrics, thresholds, and tolerances associated with the parameters. Figure SO.34 below depicts Service Level Objectives, along with the Parameters that define them.

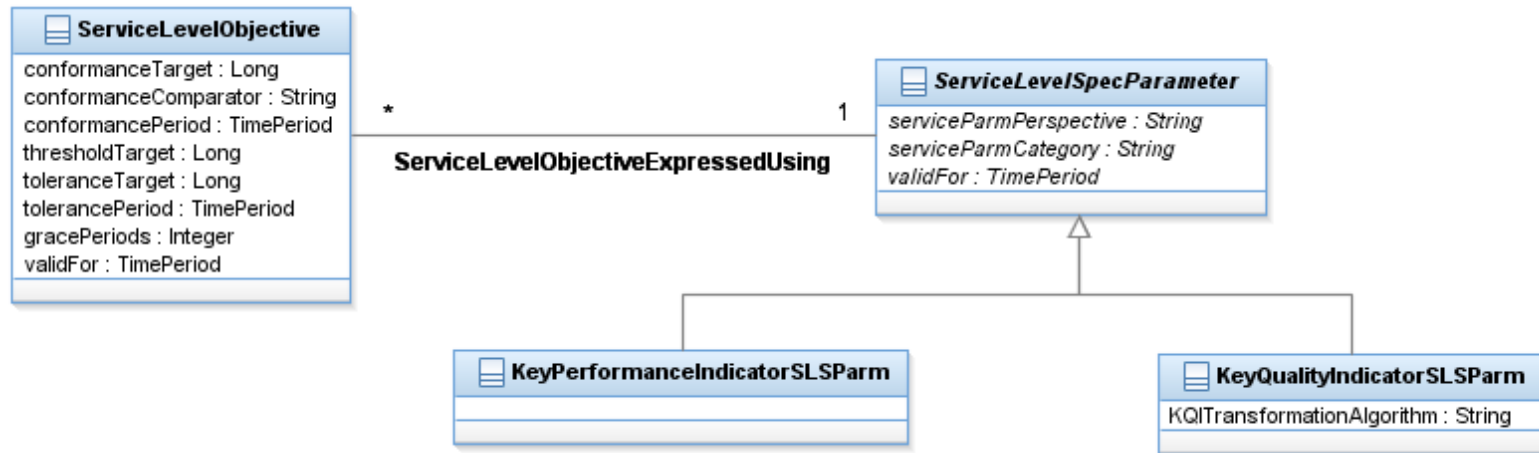


Figure SO.34 - Service Level Specification Parameters and Objectives

Service Level Specification parameters can be one of two types. A Key Quality Indicator (KQI) provides a measurement of a specific aspect of the performance of a Product (i.e., ProductSpecification, ProductOffering, or Product) or a Service (i.e., ServiceSpecification or Service). A KQI draws its data from a number of sources, including Key Performance Indicators (KPIs). A KPI provides a measurement of a specific aspect of the performance of a Service (whether it is a network- or a non-network-based Service) or a group of Services of the same type. A KPI is restricted to a specific resource type.³ The figure below extends Figure SO.32 by showing how KPIs and KQIs are related to products and services.

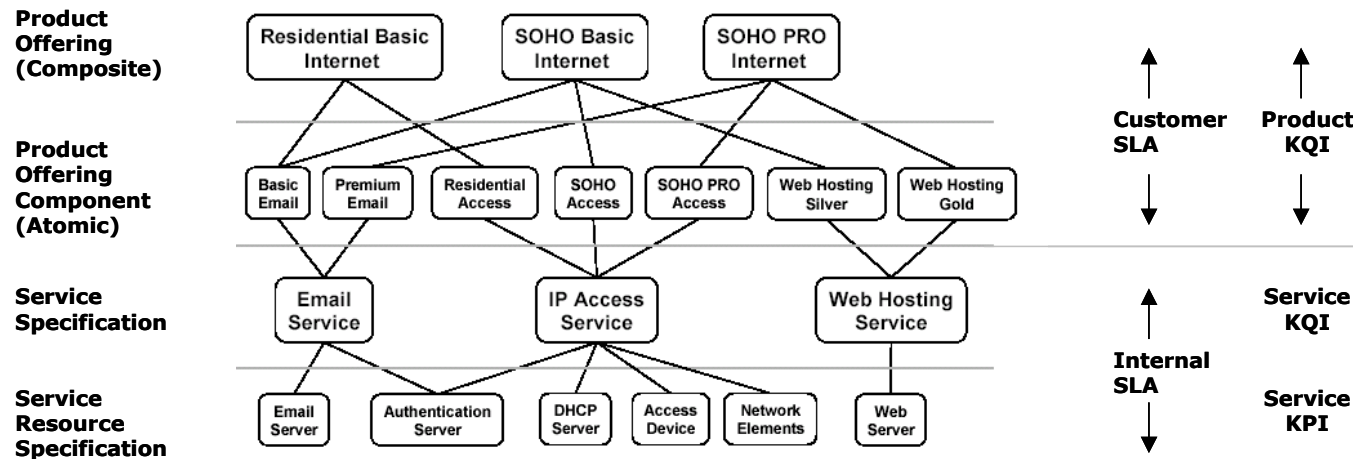


Figure SO.35 - KPIs, KQIs, Products, and Services

³ Definitions excerpted from Wireless Services Quality Handbook, GB923.
© TM Forum 2016. All Rights Reserved.

Customer-facing Product KQIs, such as the number of trouble tickets opened for a Product, characterize Customer Service Level Agreements. Service KQIs, such as the maximum web server unavailability time that is permitted, represent an aggregation of Service KPIs or a simple one-to-one mapping to a Service KPI, and characterize internal and engaged party Service Level Agreements. Atomic Service KPIs, such as the maximum IP packet loss ratio, do not characterize Service Level Agreements, but form the basis for the determination of KQIs. Please see the SLA Management Handbook (GB917) for additional examples and the Wireless Services Measurement Handbook (GB923) for more details about KPIs and KQIs.

Violations to Service Level Agreements usually result in some consequence for the provider of the Service. The consequence could be a general one that applies to any objective violation, or the consequence could be associated with a specific objective. An example of a consequence is a discount applied to a customer's bill if the number of trouble tickets for a Product exceeds some number over a specified period of time. The figure below shows how consequences of violating a Service Level Agreement fit into the Service Level Specification model.

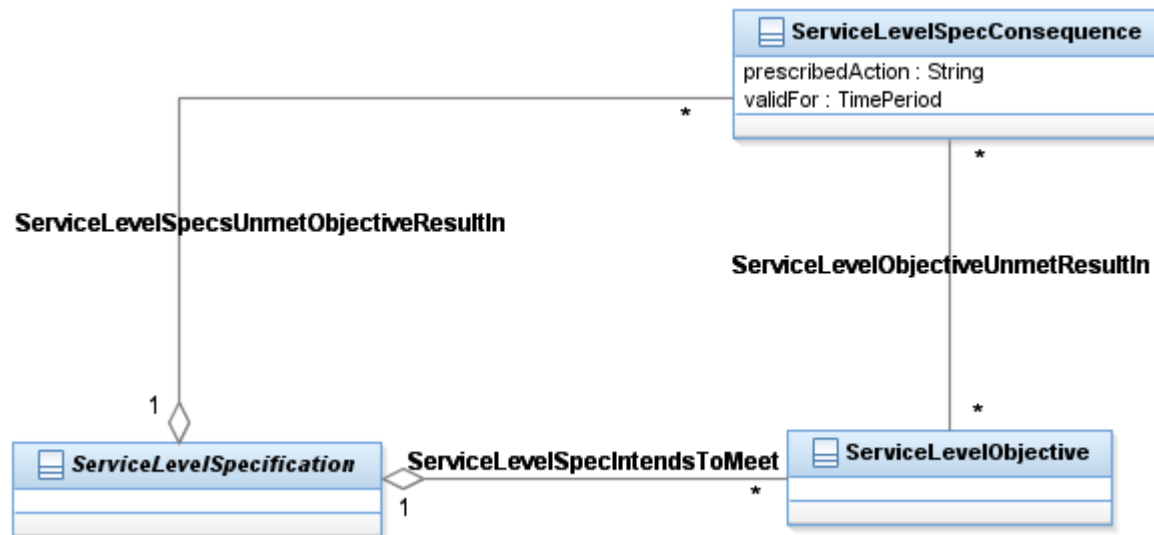


Figure SO.36. - ServiceLevelSpecConsequences

No attempt was made to build a detailed model that represents the prescribed actions. Rather, the above model is intended to be an extensible framework that enables each adopter of the model to customize their extensions for this business entity.

The applicability of Service Level Specifications, Service Level Objectives, and Service Level Specification consequences may vary by day and/or time of day. For example, a Service Level Objective may be represented by one set of metrics during weekdays and by another set during weekends. Thus, we may need to define a set of consequences to associate the proper effect of violating a Service Level Objective. For example, one consequence might be the result of a Service Level Violation during normal working hours, while a second consequence might be used to represent Service Level Violations that occur during non-working hours. This is handled by introducing a Service Level Specification applicability business entity as shown in the figure below.

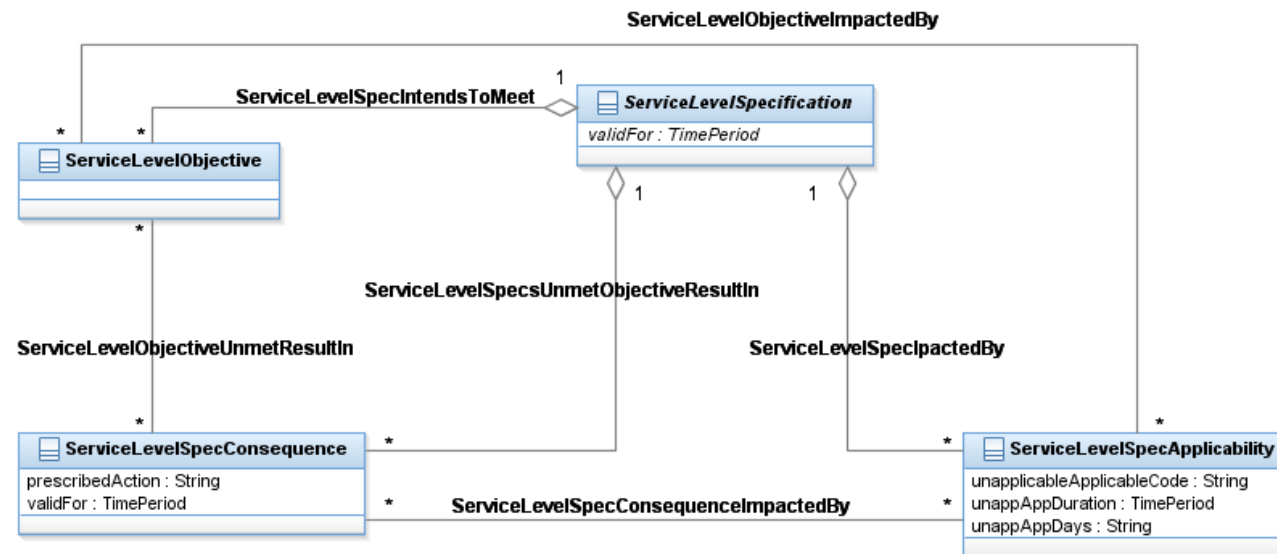


Figure SO.37 - ServiceLevelSpecApplicability

The ServiceLevelSpecExpression entity is intended to describe strict relations between ServiceLevelSpecification subordinates (ServiceLevelSpecParameter, ServiceLevelObjective, ServiceLevelSpecApplicability, and ServiceLevelSpecConsequence). The baseline idea of ServiceLevelSpecExpression entity resolves the multiple associations among ServiceLevelSpecification subordinate entities. So in this case it is possible to have variety of ServiceLevelSpecification subordinates combined using ServiceLevelSpecExpression. Figure 4SO.43a – ServiceLevelSpecificationExpression shows this entity.

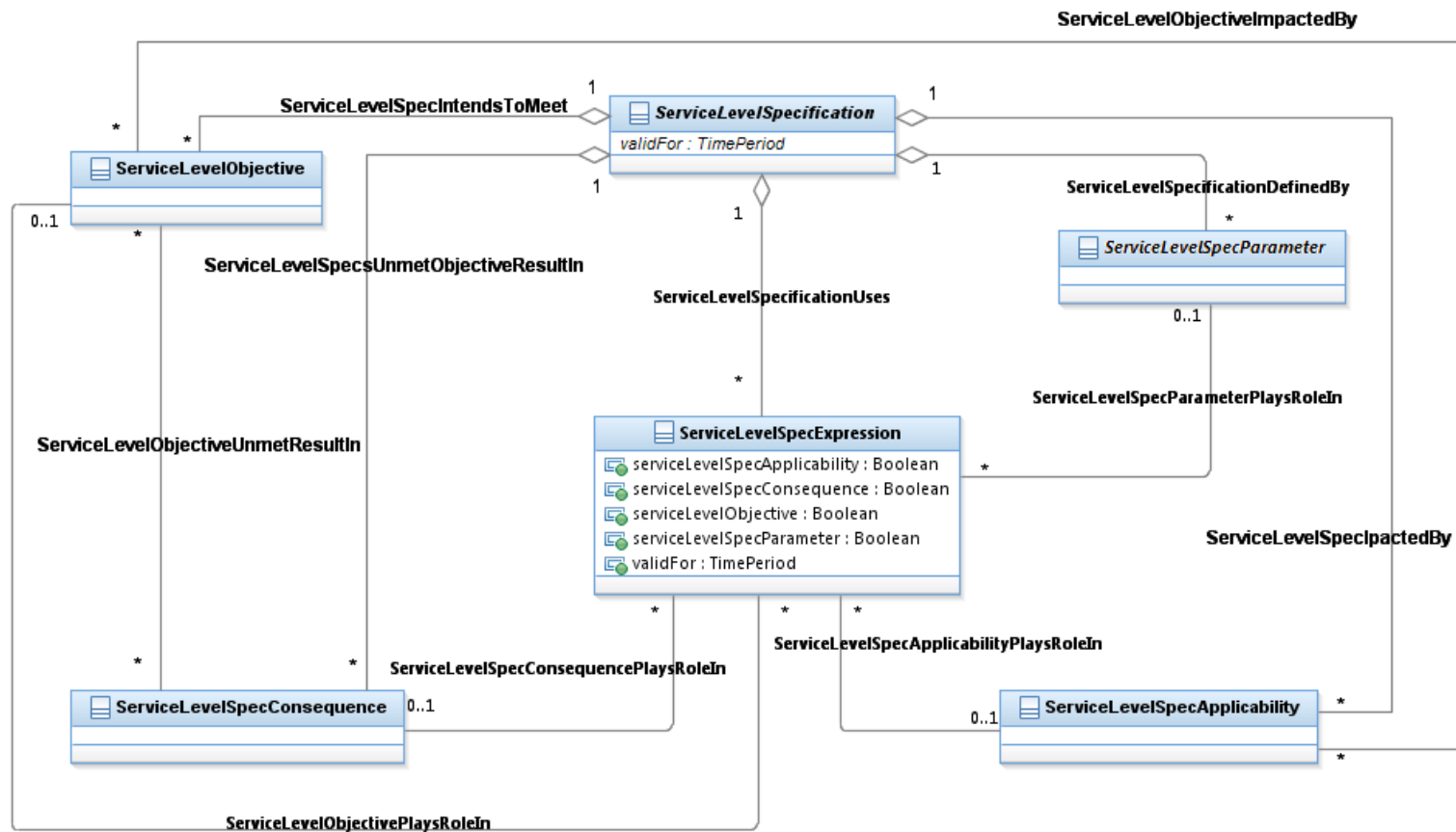


Figure SO.43a - ServiceLevelSpecificationExpression

Use Case example

When it is needed to calculate Time-To-Resolve of a customer's` issue and the system must notify different departments upon reaching different Time-To-Resolve values.

In this case first ServiceLevelObjective_A with thresholdTarget = 15 min and conformanceTarget = 45 min will be created and associated with ServiceLevelSpecConsequence_A (this consequence will raise notification action: Send letter to the user about existed issue when threshold target was reached and apologies letter when conformance target was reached).

On the other hand second ServiceLevelObjective_B with thresholdTarget = 5 min and conformanceTarget = 30 will be created and associated with ServiceLevelSpecConsequence_B (this consequence will raise notification action: Send letter to the engineer about existed issue when threshold target was reached and escalation letter to the management when conformance target was reached)

So in this case it is crucial to understand which ServiceLevelObjective is related to which consequence and this could be done with in Expression in following way:

- Create ServiceLevelSpecExpression_A were
- serviceLevelSpecApplicability = **Applicability_A**
- serviceLevelSpecConsequence = **Consequence_A**
- serviceLevelSpecParameter = **Parameter_A**
- serviceLevelObjective = **ServiceLevelObjective_A**
- **and**
- create second ServiceLevelSpecExpression_B were
- serviceLevelSpecApplicability = **Applicability_A**
- serviceLevelSpecConsequence = **Consequence_B**
- serviceLevelSpecParameter = **Parameter_A**
- serviceLevelObjective = **ServiceLevelObjective_B**

This section has defined a ServiceLevelSpecification, and its associated related objects, to be compatible with the SLA Handbook and the work done by the WSMT. The next version of this Addendum will seek to align this work with other industry work.

1.4. References

The following table provides a list of references used in this Addendum.

Normative References	
UML 1.5	OMG UML Resource Page http://www.uml.org/
Non Normative References	
ACIA	Alliance Common Information Architecture (AT&T, BT & Concert) version 3.1, 15 Dec 2000
Fowler-AP	Analysis Patterns – Reusable Object Models by Martin Fowler ISBN 0-201-89542-0 http://martinfowler.com/articles.html
Fowler-Role	Dealing with Roles http://www.awl.com/cseng/titles/0-201-89542-0/apsupp/roles2-1.html
eTOM	GB921 version 7.0
Zachman	Zachman Framework http://www.zifa.com/frmwork2.htm
Larman	Applying UML and Patterns, Second Edition, ISBN 0-13-095004-1 http://www.craiglarman.com/book_applying_2nd/Applying_2nd.htm
Baumer	The Role Object (Design) Pattern. Download PDF from http://www.riehle.org/computer-science-research/1997/plop-1997-role-object.pdf
DEN	Directory Enabled Networks by John Strassner ISBN 1-57870-140-6
DEN-ng	Directory Enabled Networks New Generation by John Strassner, et. Al., to be published
CIM	Common Information Model, DMTF http://www.dmtf.org/standards/standard_cim.php
CIMPROB	Mining Information from the DMTF CIM into the TMF SID, ed by John Strassner (This was a deliverable from the TMF-DMTF liaison)
MetaSolv	The MetaSolv Object Model, contribution to the TMF
SLA Handbook	GB917 version 1.5
Wireless Services Measurement Handbook	GB923 version 1.0 (member evaluation)
XML Specification of SLAs for Telecom Services	Stephen Cross, University of Oxford, Masters Thesis, Feb 2002.

2. Administrative Appendix

This Appendix provides additional background material about the TM Forum and this document. In general, Sections may be included or omitted as desired; however, a Document History must always be included.

2.1 About this document

This is a TM Forum Guidebook. The guidebook format is used when:

- The document lays out a 'core' part of TM Forum's approach to automating business processes. Such guidebooks would include the Telecom Operations Map and the Technology Integration Map, but not the detailed specifications that are developed in support of the approach.
- Information about TM Forum policy, or goals or programs is provided, such as the Strategic Plan or Operating Plan.
- Information about the marketplace is provided, as in the report on the size of the OSS market.

2.2 Document History

2.2.1. Version History

Version Number	Modified by:	Date Modified	Description of changes
0.1	J. Strassner	September 2002	First rough draft
0.2	J. Strassner	October 2002	Final population using DEN-ng high-level service model, which contains mined information from DEN, OSS/J and MetaSolv
0.3	J. Strassner	October 2002	Final draft ready for modeling team to review
0.4	J. Strassner	November 2002	Updated in response to detailed comments from John Reilly and Chris Hartley
0.5	J. Strassner	November 2002	Updated to incorporate additional comments from John Reilly and Cliff Faurer.

1.0	TMF	November 2002	Formatted for TMF Publishing
1.1	J. Strassner	April 2003	Added ServicePackage and ServiceBundle, which link to the QoS model. Minor corrections to get Service model better in sync with other models
1.2	J. Strassner	May 2003	Incorporated comments from the SID team, including detailed comments from Cliff and John Reilly. Excised ProductItem, and enhanced the link to the QoS Addendum.
1.3	J. Strassner	May 2003	Incorporated final comments from Cliff, updated for consistency with other Addenda
1.4	J. Strassner	June 2003	Updated PartyRole interaction section
2.0	TMF	July 2003	Formatted for Member Review with NGOSS R3.5
2.1	TMF	Aug 2004	To reflect TMF Approved Status
3.0	J Reilly	Mar 2008	Updates to Characteristics and miscellaneous updates to synchronize addendum with UML model.
7.5	John Reilly	Jan 2008	Updated for ITU-T submission
7.6	Tina O'Sullivan	May 2008	Converted to correct template and other corrections prior to posting.
7.7	John Reilly	July 2008	Updates to Characteristics and miscellaneous updates to synchronize addendum with UML model.

7.8	John Reilly	July 2008	Updates after new template applied
7.9	Tina O'Sullivan	July 2008	Final changes prior to posting.
8.0	Alicja Kawecki	May 2009	Minor updates to reflect TM Forum Approved status
9.0	John Reilly	January 2010	Updates to reflect SID Phase IX Change Requests
9.1	Alicja Kawecki	April 2010	Minor cosmetic corrections for web posting and ME
9.2	Alicja Kawecki	June 2010	Updated Notice
9.3	Alicja Kawecki	Oct 2010	Updated to reflect TM Forum Approved status
9.4	Josh Salomon	January 2011	Updates to reflect SID release 9.5 change requests
9.5	John Reilly	March 2011	Correct figure references and corrupt table.
9.6	Alicja Kawecki	March 2011	Updated Notice, minor formatting corrections prior to web posting and ME
9.7	Alicja Kawecki	September 2011	Updated to reflect TM Forum Approved status
9.8	Cécile Ludwichowski	June 2013	Incorporation of CR: Artf2381 (Figure SO.45) Artf1431 (Figure SO.31, SO-33) Artf1788
9.9	Alicja Kawecki	July 2013	Updated cover & notice, corrected footer prior to posting
9.10	Alicja Kawecki	Sep 2013	Added IPR Mode to cover, updated notice to reflect TM Forum Approved status
9.10.1	Alicja Kawecki	Nov 2013	Updated cover, header, footer & Notice to reflect TM Forum Approved status
14.5.0	John Reilly	October 2014	Updated Figure SO.7

			to align with model
14.5.1	Alicja Kawecki	March 2015	Updated cover, header, footer and Notice to reflect TM Forum Approved status
16.0	John Reilly	May 2016	Aligned model and guide book, removed data dictionary, and aligned with changes in other domains.
16.0.1	Alicja Kawecki	26 May 2016	Minor cosmetic edits prior to publication for Fx16
16.0.2	Alicja Kawecki	8 Sep 2016	Updated cover, header and Notice to reflect TM Forum Approved status

2.2.2. Release History

Release Number	Date Modified	Modified by:	Description of changes
8.0	17/July/2008	John Reilly	Updates to Characteristics and miscellaneous updates to synchronize addendum with UML model.
9.0	John Reilly	January 2010	Updates to reflect SID Phase IX Change Requests
9.5	Josh Salomon	January 2011	Updates to reflect SID release 9.5 change requests
13.0	Cécile Ludwiczowski	June 2013	Incorporation of CR: Artf2381 (Figure SO.45) Artf1431 (Figure SO.31, SO-33) Artf1788
14.5.0	John Reilly	October 2014	Updated Figure SO.7 to align with model and added ServiceLevelSpecExpression entity and figure.
16.0	John Reilly	May 2016	Aligned model and guide book, removed data dictionary, and aligned with changes in other domains.

2.3 Acknowledgments

This document was prepared by the members of the TM Forum Information Framework (SID) team.

The Shared Information/Data Model is a genuinely collaborative effort. The TM Forum would like to thank the following people for contributing their time and expertise to the production of this document. It is just not possible to recognize all the organizations and individuals that have contributed or influenced the introduction. We apologize to any person or organization we inadvertently missed in these acknowledgments.

Key individuals that reviewed, provided input, managed, and determined how to utilize inputs coming from all over the world, and really made this document happen were:

Name	Affiliation
Cliff Faurer	TM Forum
Ram Garg	MetaSolv Software
Chris Hartley	Telstra
Helen Hepburn	British Telecom
Jenny Huang	AT&T
Matt Izzo	Agilent Technologies
Jessie Jewitt	Ciena
Cécile Ludwichowski	France Telecom -Orange
Mike McLoughlin	British Telecom
Dave Raymer	Motorola
John Reilly	TM Forum
Wayne Sigley	Telstra
Yaroslav Oratovsky	NetCracker
John Strassner	Intelliden (original editor)